Software Engineering

How To Create Software Products Everyone Loves!

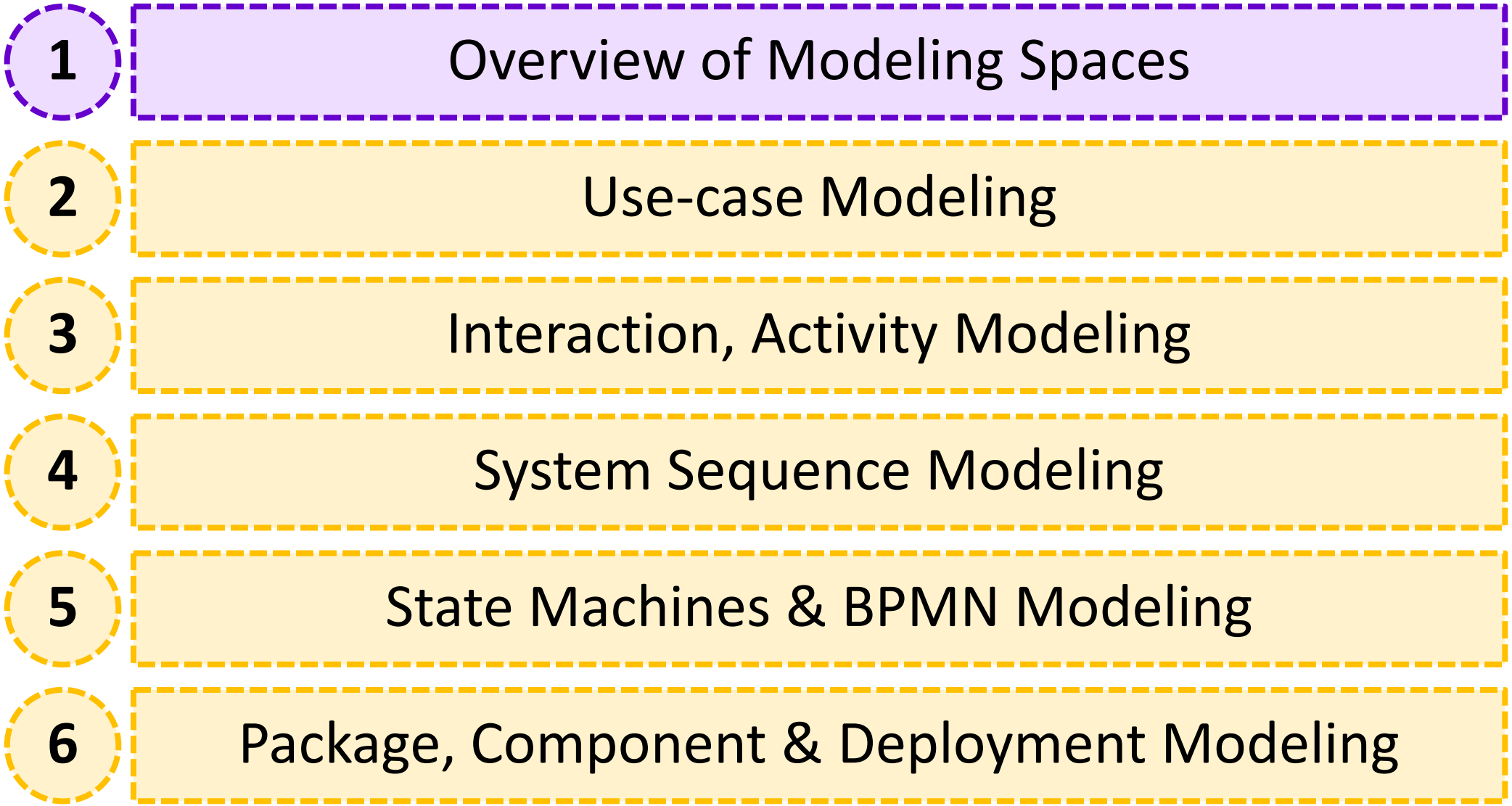
Module 16

Software Architecture and Technical Design: Modeling & Design With UML

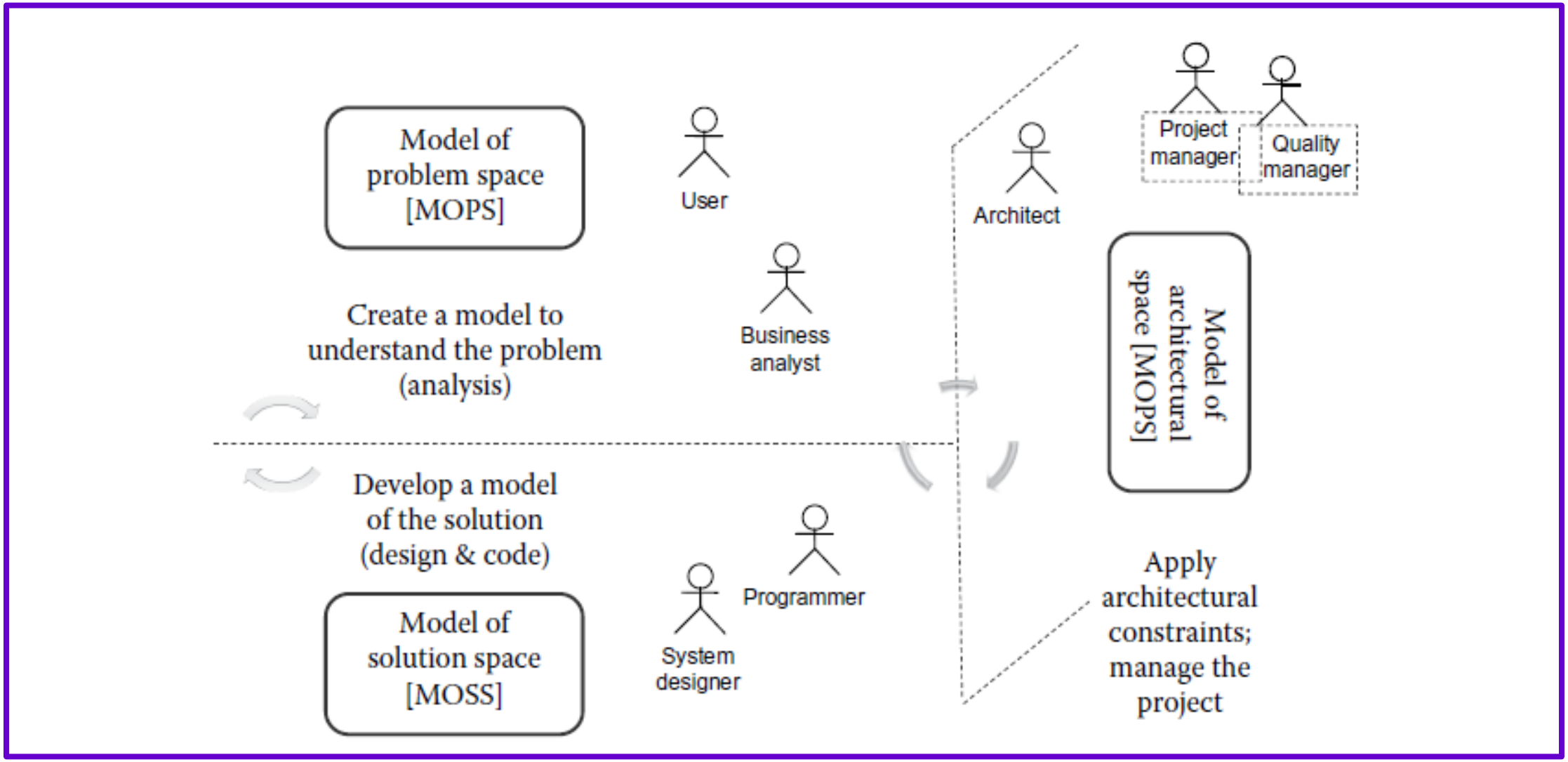
Ali Samanipour

May. 2023

Ali Samanipour
linkedin.com/in/Samanipour


1
Overview of Modeling Spaces
2
Use-case Modeling
3
Interaction, Activity Modeling
4
System Sequence Modeling
5
State Machines & BPMN Modeling
6
Package, Component & Deployment Modeling

RemeModeling Spaces in Software Engineering



Importance of UML Diagrams to Respective Models

With this understanding of the three modeling spaces, it is now easier to understand how each of the 14 UML diagrams can play a role in these different modeling spaces with varying degrees of importance and relevance.

UML Diagrams	*MOPS (Business Analyst)*	*MOSS (Designer)*	*MOAS (Architect)*
Use case	*****	**	*
Activity	*****	**	*
Class	***	*****	**
Sequence	****	*****	*
Interaction overview	****	**	**
Communication	*	***	*
Object	*	*****	***
State machine	***	****	**
Composite structure	*	*****	****
Component	*	***	*****
Deployment	**	**	*****
Package	***	**	****
Timing	*	***	***
Profile	*	**	****

Actor

An actor is a role played by a person or a thing that is external to the software system. An actor (user of a system) interacts with the system in order to achieve business goals.

- A role played by a typical user of the system
- A role that initiates an interaction with the system
- Time is considered an actor because time-triggered events initiate an interaction or a process within a system
- A role that derives benefit (achieves goals) from the system
- An “external system” with which the system under development will interact
- An external device with which the system under development will interact
- Anything that sends a message to the system
- Anything that receives a message from the system

How to Find Actors?

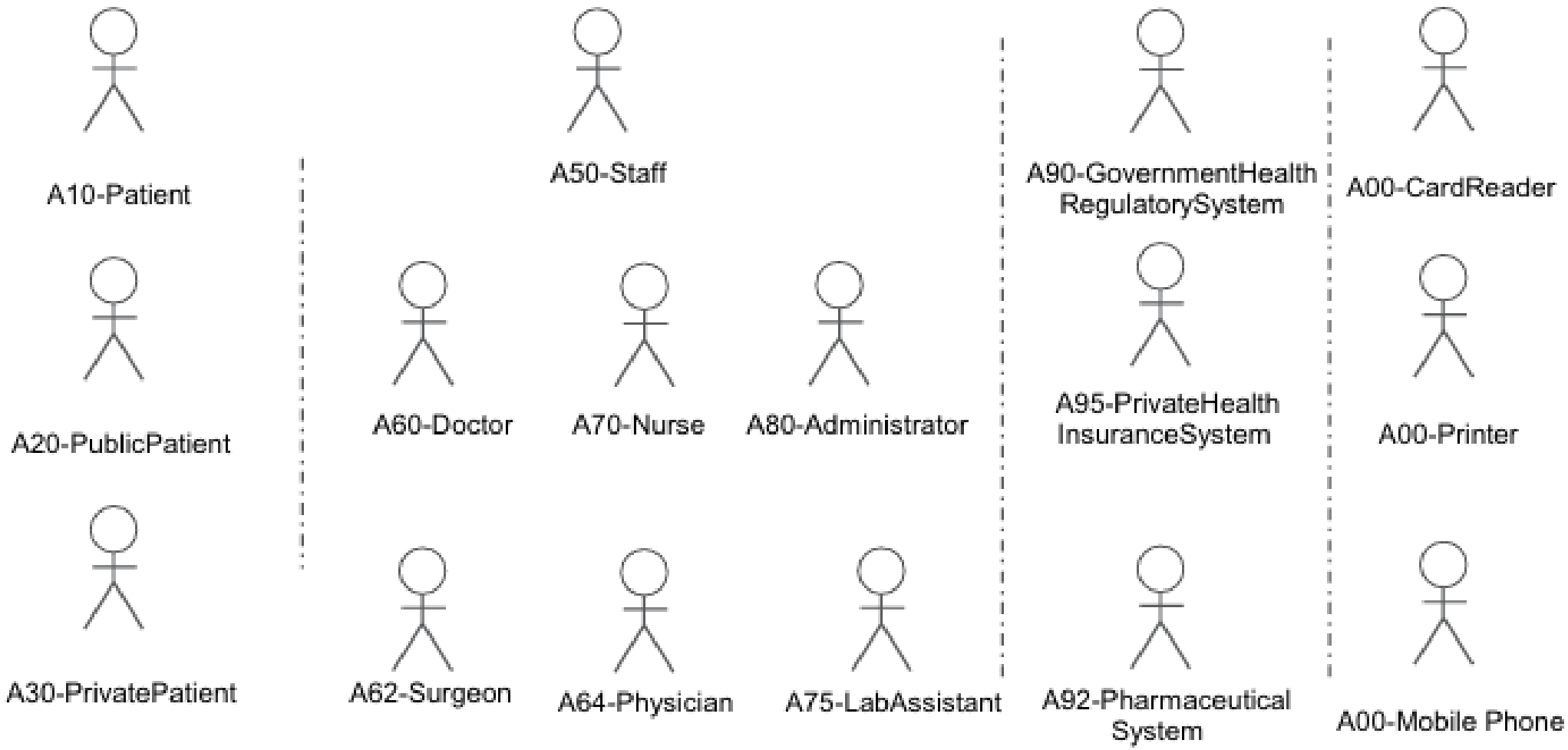
- Who will be the main and secondary users of the system?
- Who will be the primary beneficiaries of the interactions with the system?
- Who will be the primary initiators of interactions with the system?
- What external systems and devices will the system under development need to interface with?
- Is there a time-based process in the system?

Actor Variations

The primary actors are those for whom the system exists. These are the main actors who benefit from the system—for example, a patient, a doctor, or a nurse in a HMS

The secondary actors are roles of indirect relevance in the HMS. For example, if a medical assistant is involved in processing a blood sample, but is not involved in the actual execution of any of the use cases.

Potential list of actors for HMS



Direct versus Indirect Actors

Direct actors are those who actually use the system. For example, an administrator keying in the details of a patient is a direct actor, whereas the patient, standing across the counter and providing her details, is an indirect actor.

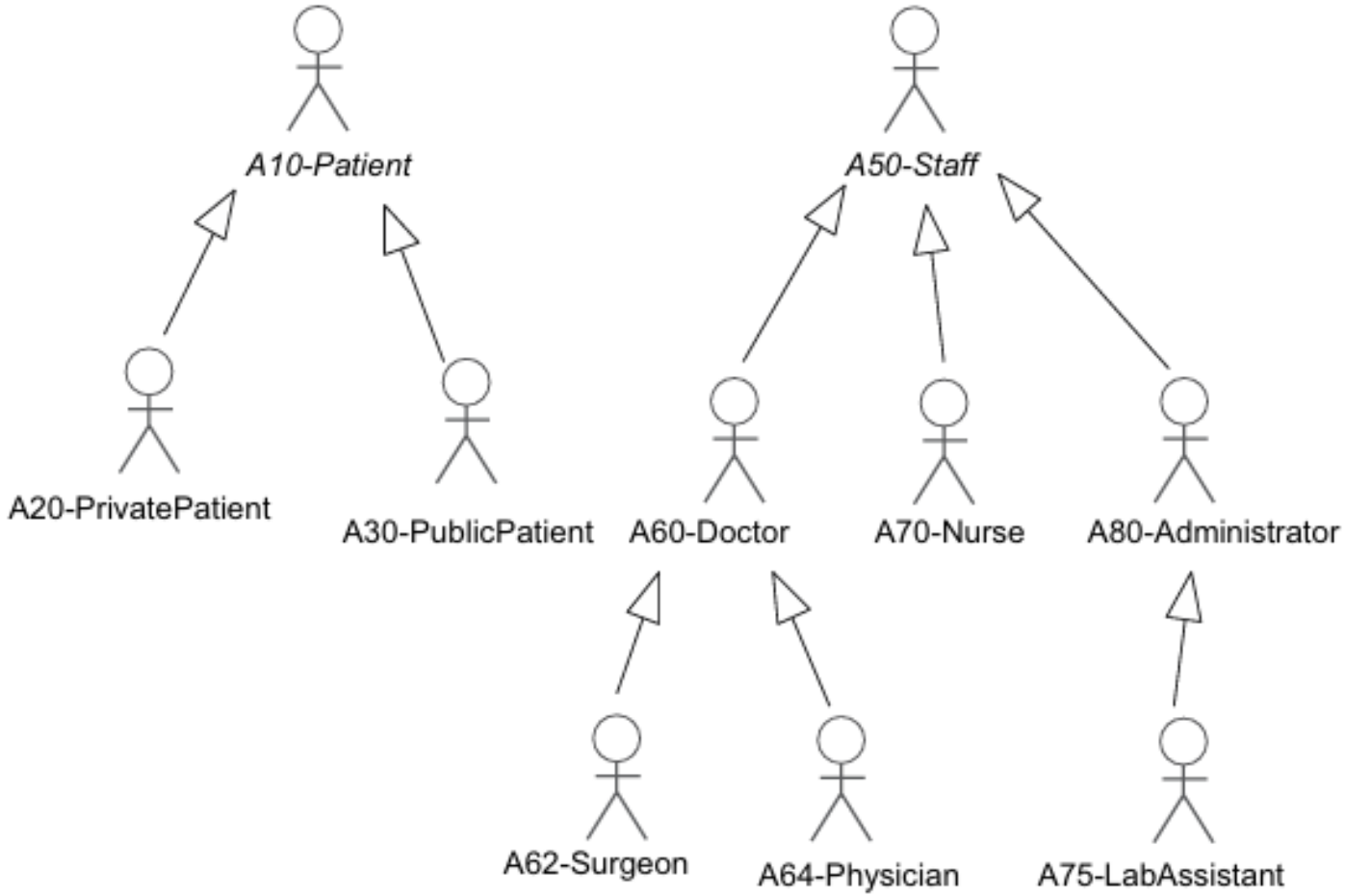
T he understanding of whether an actor is primary or secondary, direct or indirect, is entirely dependent on the context of the actor's use of the system. For example, a patient that is indirect while standing across the counter would become a direct actor when accessing her details on the Internet.

Abstract versus Concrete Actors

The UML permits actors to be generalized. This means an actor can inherit the definition of another actor.

T he understanding of whether an actor is primary or secondary, direct or indirect, is entirely dependent on the context of the actor’s use of the system. For example, a patient that is indirect while standing across the counter would become a direct actor when accessing her details on the Internet.

Abstract versus concrete actors and a corresponding actor hierarchy in HMS



Actor Documentation

Actor Thumbnail
<Name of the actor. Optionally, a number prefixes the name to facilitate grouping of actors. The prefix also differentiates the actor from a possible class with the same name.>

Actor Type and Stereotype
<Describes the type of actor. This can include whether it is a primary or secondary actor, person or external system or device, or if the actor is "abstract" or concrete. The type of actor may be described in general or it may be a formal stereotype.*>

Actor Description
<Short description of the actor and what he/she/it does. Together with the actor thumbnail, this might be the only thing documented for the actors in the first iteration.>

Actor Relationships
<Thumbnails of other relevant actors or use cases in the system with whom *this* actor is interacting. If there is an inheritance hierarchy, thumbnails of generalized/specialized actors can also be noted here.>

Interface Specifications
<Since, by definition, the actor has to interact with the system, we note here the details of the interface through which the actor performs this interaction. This will be a list of the numbers and names of GUI specifications related to this actor—including specifications of Web interfaces. For external systems and devices, it may be a description of the interface that may not be graphic.>

Author and History
<Original author and modifiers of this actor description>

Reference Material
<Relevant references, as well as sources from where material is inserted/available for *this* actor.>

Actor Documentation for “A10-Patient”

Actor Thumbnail
Actor: A10-Patient

Actor Type and Stereotype
This is an abstract actor representing all types of patients in the HMS.

Actor Description
The actor patient is the primary role interacting with the HMS in order to carry out all functions related to the patient. This actor will primarily use the system to update her details, check for the availability of doctors, schedule consultations with doctors, and seek follow-up advice. In order to carry out these functions, this actor will have to register herself, and also identify herself every time the system is accessed. This actor can be a private patient or a patient belonging to the public health system (a public patient). This private versus public distinction is made only during the registration process by the patient providing either her private insurance details or her Medicare details.

Actor Documentation for “A10-Patient”

Actor Relationships
Two different types of concrete actors are derived from this actor:
A20-PublicPatient
A30-PrivatePatient
The actor will interface with the following use case (examples):
UC01-LogsIn
UC10-RegistersPatientDetails

Interface Specifications
UI010-LogIn*
UI020-PatientDetails
I900-GovernmentHealthCareSystem

Author and History
Colleen Berish

Reference Material
Government rules regarding patient registration in the hospital can be found on the government health regulatory system website.

What Is a Use Case?

A use case documents a series of interactions of an actor with a system. This interaction is meant to provide some concrete, measurable results of value to the actor.

Use cases describe what a system does, but they do not specify how the system does it.

Finding Use Cases

A use case documents a series of interactions of an actor with a system. This interaction is meant to provide some concrete, measurable results of value to the actor.

- Interviews and discussions with users and domain experts in a workshop session
- Play-acting of various scenarios or “stories” told by users in terms of how they would use the system
- Identifying and documenting actors, leading to an understanding of their goals or purpose in using the system
- Revisiting the output of requirements analysis

Finding Use Cases …

- Formal and informal problem statements (such as presented in Appendix A)
- Executing existing systems (especially legacy applications), if available
- Investigating existing user documentation, if available
- Investigating existing “help” for the system, if available
- Researching the problem domain, especially on the Internet for relevant analysis models
- Researching and using published literature, such as Analysis Patterns (Fowler)

Use Case Documentation Template

Use Case Thumbnail:
<This is the number and name of the use case and, optionally, a version number. The numbering may take the form UC10-, UC20-, where UC stands for use case and the numbering provides a common grouping mechanism, similar to the actors.>

Use Case Description:
<This is a short description of the use case. This description ranges from a brief "one liner" to a paragraph describing its purpose and usage. Sometimes, for a small, single-iteration project, this might be the only description of the use case.>

Stereotype and Package:
<Description of the stereotype and the package to which this use case belongs. This is optional information and may not always be documented, although it will be easily entered in a modeling tool.>

Actors:
<A list of the actors involved in this use case is documented here. >

Preconditions:
<Preconditions are the conditions that need to be satisfied before the execution described by the use case can commence.>

Postconditions:
<Postconditions are conditions that must be met at the end of a use case.>

Use Case Relationships:
<Thumbnails of other use cases that are included, extended, or inherited. These three use-case-to-use-case relationships are discussed in >

Use Case Documentation Template

Use Case Text (Basic Flow):
1.0 <description of step>
2.0 <description of step> (A1, E1, E2)
3.0 <description of step> (A2, E3)
≪include≫ <Thumbnail of use case(s) included>
≪extends≫ <Thumbnail of use case(s) extended>

Alternative Flow:
<A1—The optional descriptions here are alternative flows under conditions specified in the steps in the basic flow.>

Exceptions:
<E1 The optional descriptions here specify actions taken under "exception" conditions encountered during the basic flow of the use case. Technically, this can represent actions to be taken in case of an error.>

Constraints:
<Here are the documented special constraints or limitations that are relevant to the use case.>

User Interface Specifications:
<Number/s and name/s of UI specifications related to the use case, including Web screen specifications, as and when available. Note that these are not user interface designs but simply references to the likely screens/forms that will be used by the actor in interfacing with the system.>

Metrics (Complexity):
<Anything that needs to be measured that is related to the use cases will be put here—for example, complexity of the use case: simple/medium/complex. This information can be helpful in highly mature organizations, typically CMM level 4 and 5, where artifacts are measured for the purpose of optimization. Complexity may be based on the number of actors, relationships with other use cases, and even technical issues for each use case. However, the actual discussion on metrics is beyond the scope of this book.>

Use Case Documentation Template

Priority:
<The importance of the functionality described by this use case: high/medium/low. This could be based on analysis and understanding of risks and importance of the use cases.>

Status:
<The state of completeness of the documentation of this use case: initial/major/final. This will indicate the level of maturity of the use case.>

Author & History:
<Original author and modifiers of this use case.>

Reference Material:
<Relevant references, as well as sources from where the use case has been derived. Any large documentation and material that does not form part of the "flow" in a use case (such as mathematical formulas, legal documents, and policy materials).>

Use Case Documentation Example

Use Case Thumbnail: UC52-IssuesBill

Actors: A50-Staff

Use Case Description
This use case describes the process of issuing a bill (or invoice) for hospital services. This bill is either posted to the patient or electronically sent to the patient or his health insurance company.

Use Case Thumbnail: UC62-ReordersMedicines

Actors: A75-LabAssistant, A50-Staff

Use Case Description
This use case describes the process for reordering medicines (drugs) in the laboratories and in the pharmaceutical section of the hospital. The actual ordering will require an authorized A50-Staff member together with help from A75-LabAssistant.

Use Case Thumbnail: UC60-ChecksInventory

Actors: A75-LabAssistant

Use Case Description
This use case describes the process for A75-LabAssistant to check the hospital's inventory of drugs. The lab assistant obtains the inventory level from the system and can then check the physical inventory to verify that the system is up to date. If the system's inventory does not match the physical inventory, then the lab assistant can adjust the system's inventory amount (with appropriate authorization).

Use Case Documentation Example 2

Use Case Thumbnail: UC10-RegistersPatient

Use Case Description: This use case describes the process of registering a new patient in the hospital system. The patient must be registered and her details verified with the government health system before any of the hospital's services can be provided.

Stereotype and Package: ≪Patient≫

Preconditions: Patient must have not been already registered with the hospital.

Postconditions: Patient is registered in the hospital system.

Actors: A10-Patient; A80-Administrator; A90-GovernmentRegulatoryHealthSystem, A95-Private Health Insurance System

Use Case Relationships: Associated with actors: A10-Patient, A80-Administrator.

Basic Flow (Text):

1. A patient arrives at the hospital for some medical treatment.
2. The administrator asks the patient if she had been previously treated at this hospital.
3. Patient provides the answer (A1).
4. Administrator asks patient for her personal details such as name, address, telephone, date of birth, and emergency contact.
5. Patient provides details as requested.
6. Administrator enters details into system.
7. System verifies details (A2).
8. Administrator asks patient whether she is a public or private patient.
9. Patient provides the answer. [Public or Private][Public]
   - 9.1a Administrator asks public patient for Medicare number.
   - 9.2a Patient provides Medicare number.
   - 9.3a Administrator enters Medicare number into system.
   - 9.4a System verifies patient identity with the government health regulatory system (A3).

Use Case Documentation Example 2 …

[Private]

9.1b Administrator asks private patient for insurance details.

9.2b Patient provides insurance details.

9.3b Administrator enters insurance details (insurance company, patient's insurance number) into system.

9.4b System verifies patient identity with private health insurance company system (A4).

10. System saves patient details.
11. System confirms patient registration.

Use Case Documentation Example 2 …

Alternative Flow:
<A1 – If the patient has visited the hospital previously, the details are registered in the hospital's system. The administrator informs the patient of existing registration.>
<A2 – If insufficient or incorrect details have been provided, the patient is requested to provide the details again.>
<A3 – The patient cannot be verified in the government health regulatory system, so the administrator asks the patient for her Medicare card details again. If no verification is possible, the patient is conditionally registered as either a full fee–paying patient or one who will provide Medicare details later.>
<A4 – The patient cannot be verified in the private health insurance company's system, so the administrator asks the patient for the private health insurance details again. If no verification is possible, the patient is conditionally registered as either a full fee–paying patient and her details with the health insurance company are verified later.>

Exceptions: None.

Constraints: None.

User Interface Specifications: UI10-PatientRegistrationForm*

Metrics: Complex **Priority:** High **Status:** Major

Author and History: Vivek E.

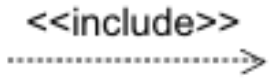
Reference Material: Details of patient that are required by law are specified in the hospital's patient policy document available from the administration department. See <patientpolicy.doc>.†

Use Case Diagram

Actor

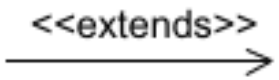
Inheritance

Use case



Inclusion of a
use case by another

Actor to use case
association

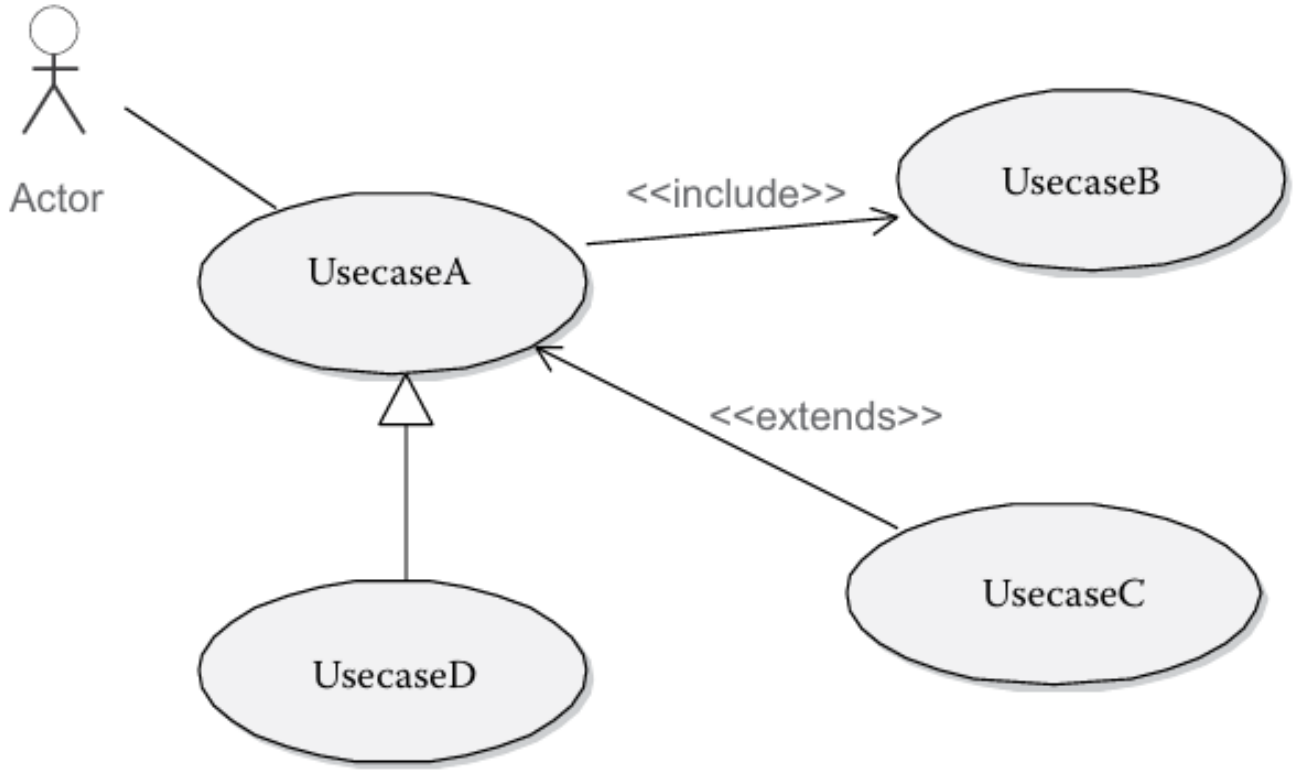


Extension of a
use case by another

System
boundary

Notes

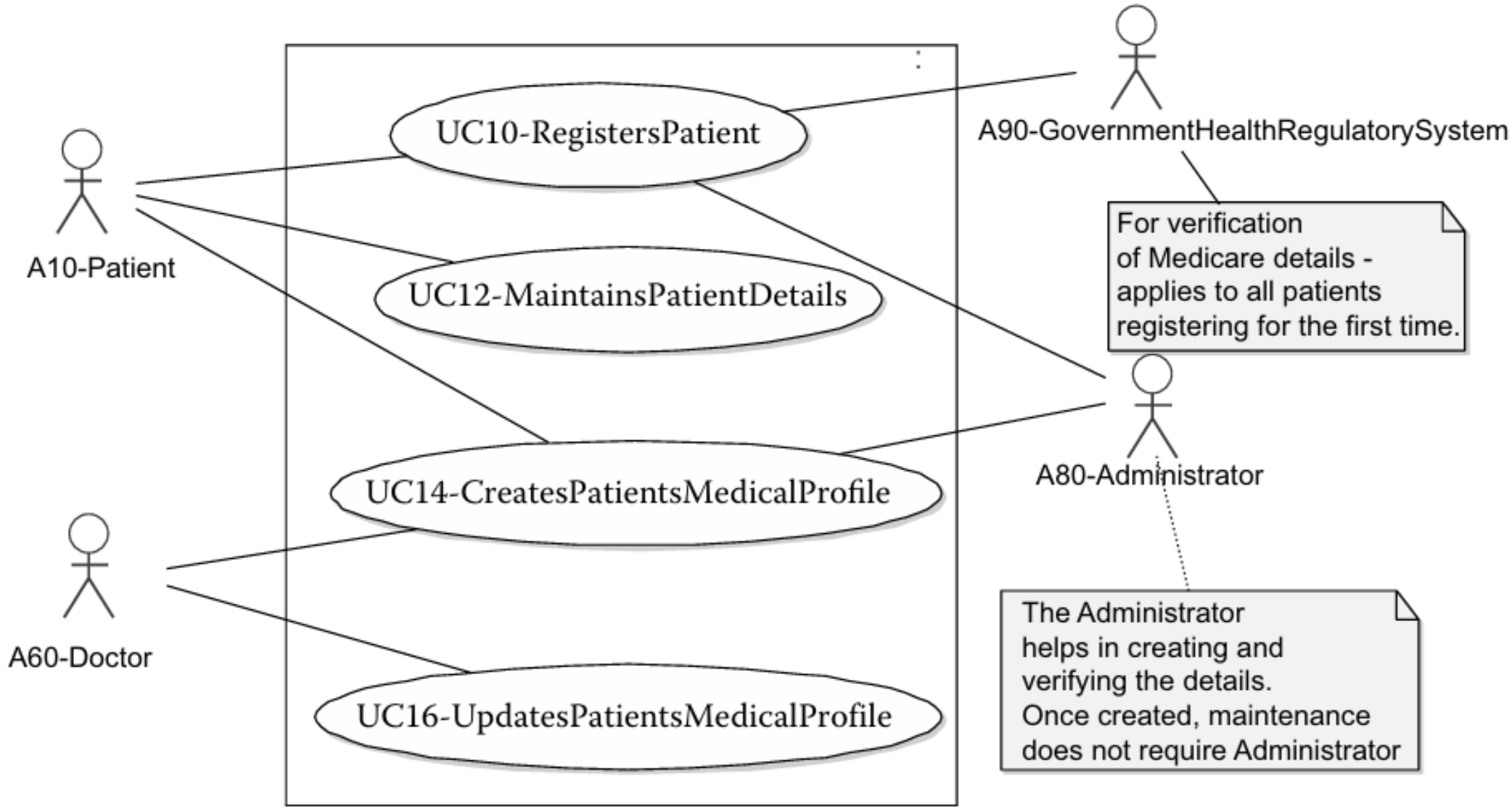
Use Case Relationships



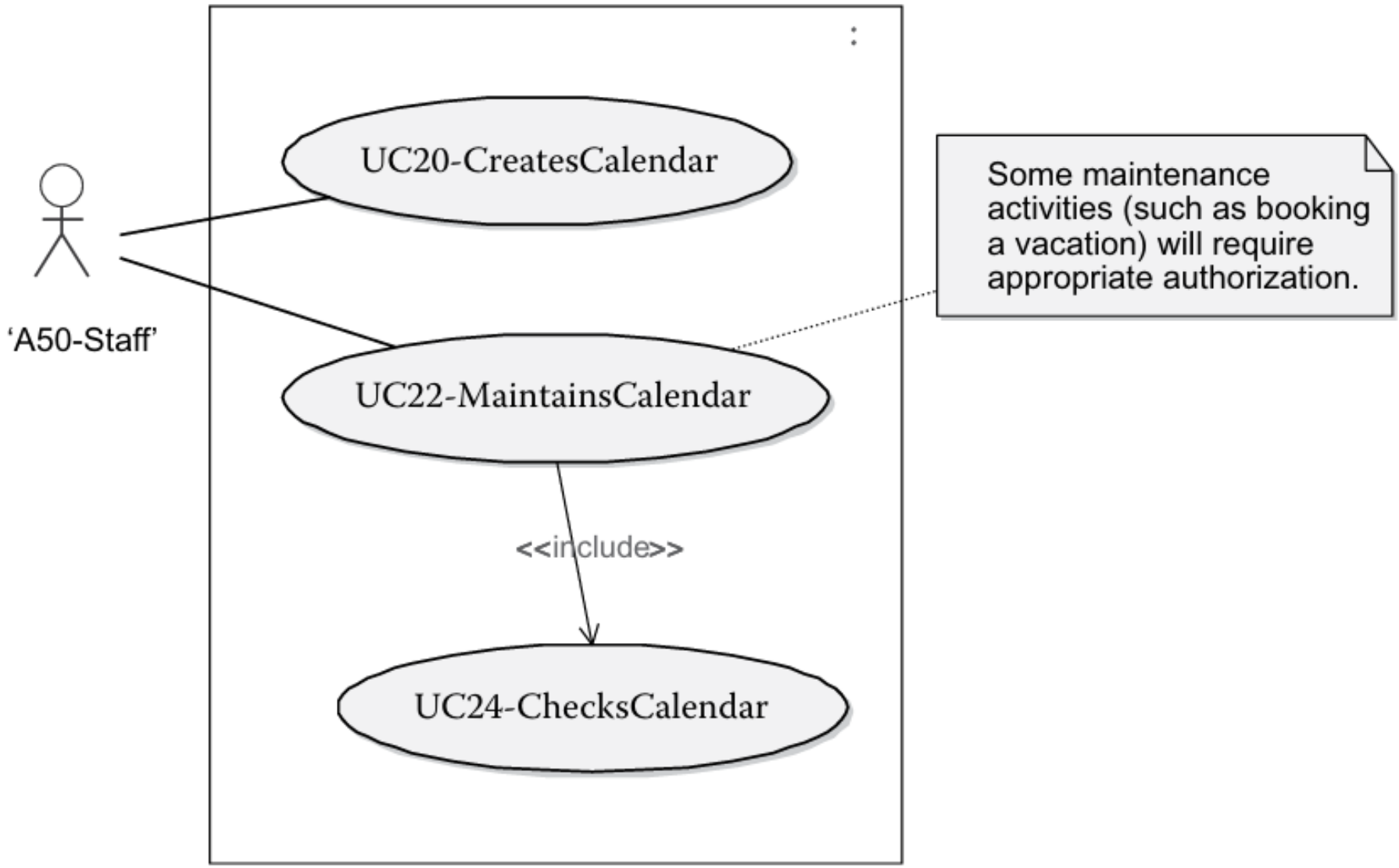
UsecaseA <<include>> UsecaseB
UsecaseC <<extends>> UsecaseA
UsecaseD <<inherit>> UsecaseA
Labeling include and extends is mandatory

Practical Tips:
Use cases are IDEAL for requirements modeling rather than system design; they DON'T crash systems; include and extends are indicative relationships only – don't be pedantic about them but add notes for explanations

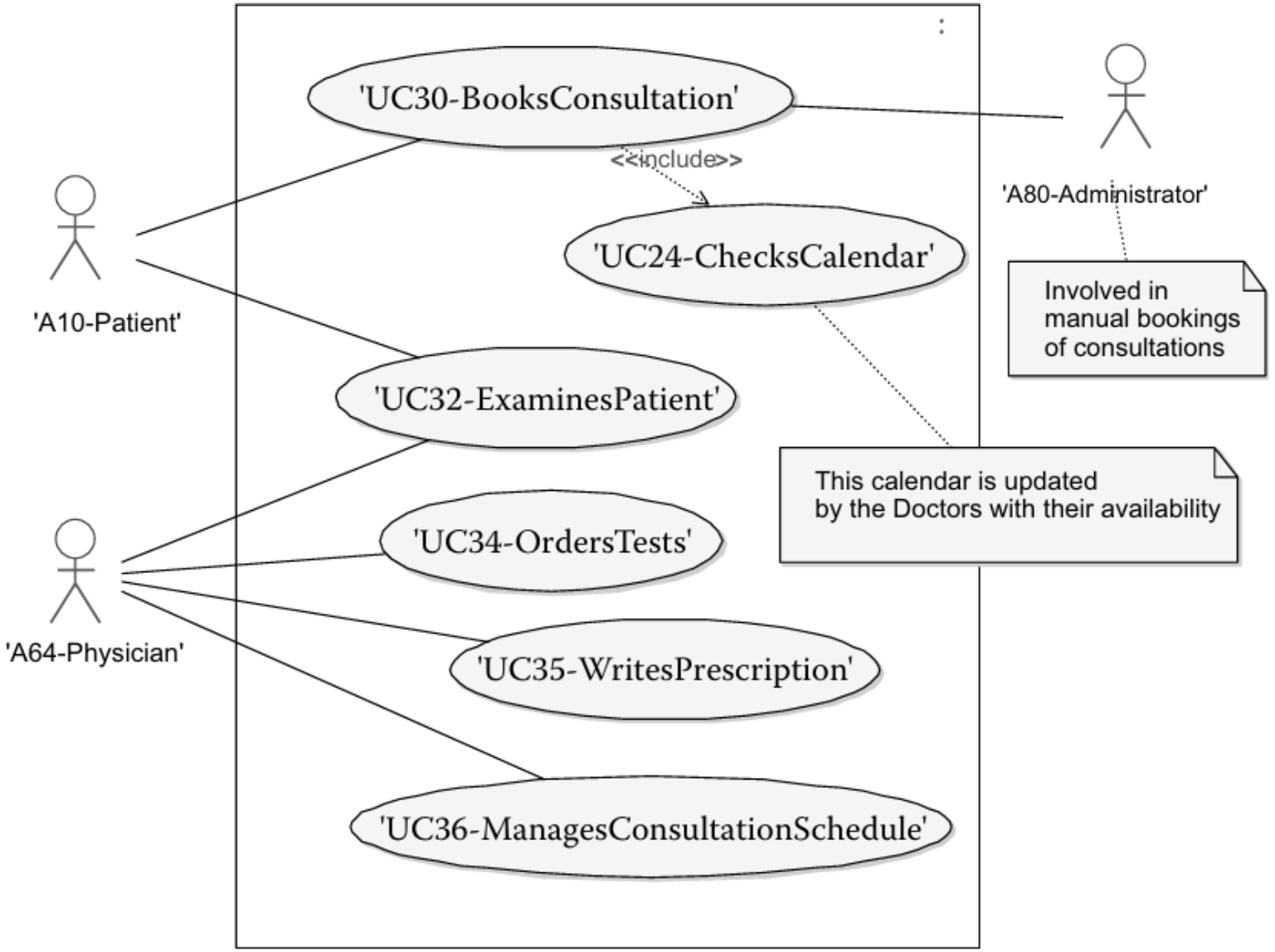
Patient maintenance “use case diagram.”



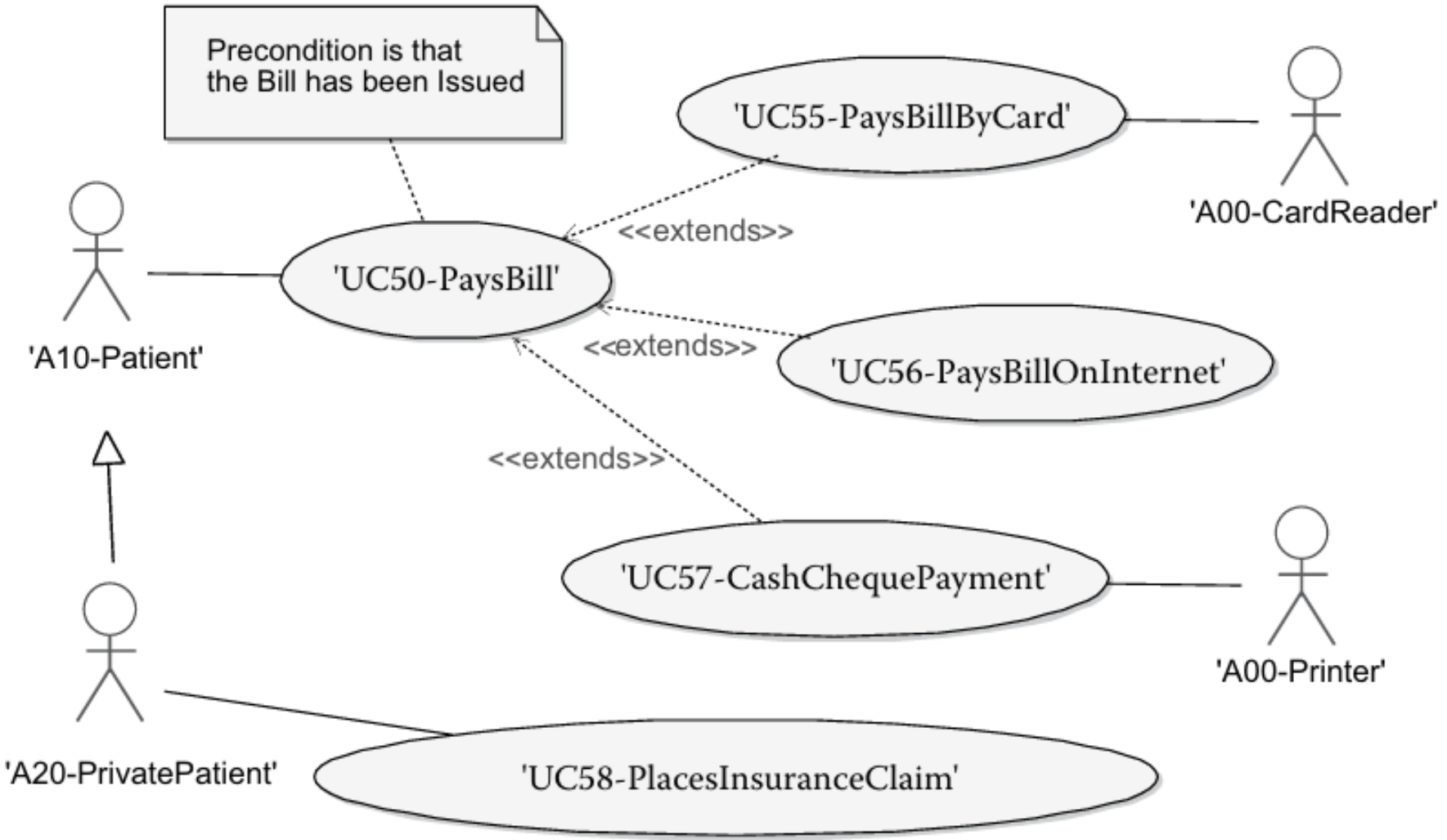
Calendar maintenance “use case diagram.”



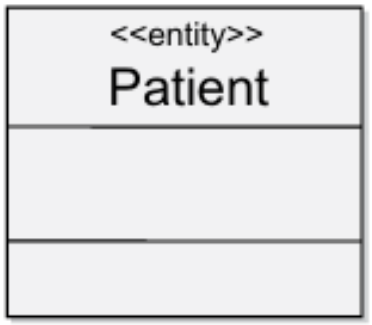
Consultation details “use case diagram.”



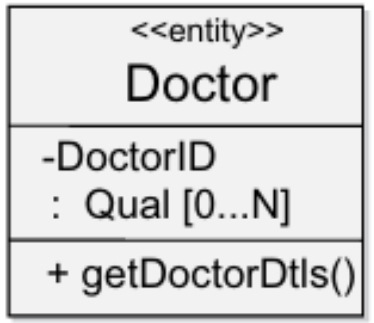
Accounting “use case diagram.”



Major notations of a class diagram.



Class:
With 3
compartments



Class
description
contains
attributes
and
operations

Inheritance

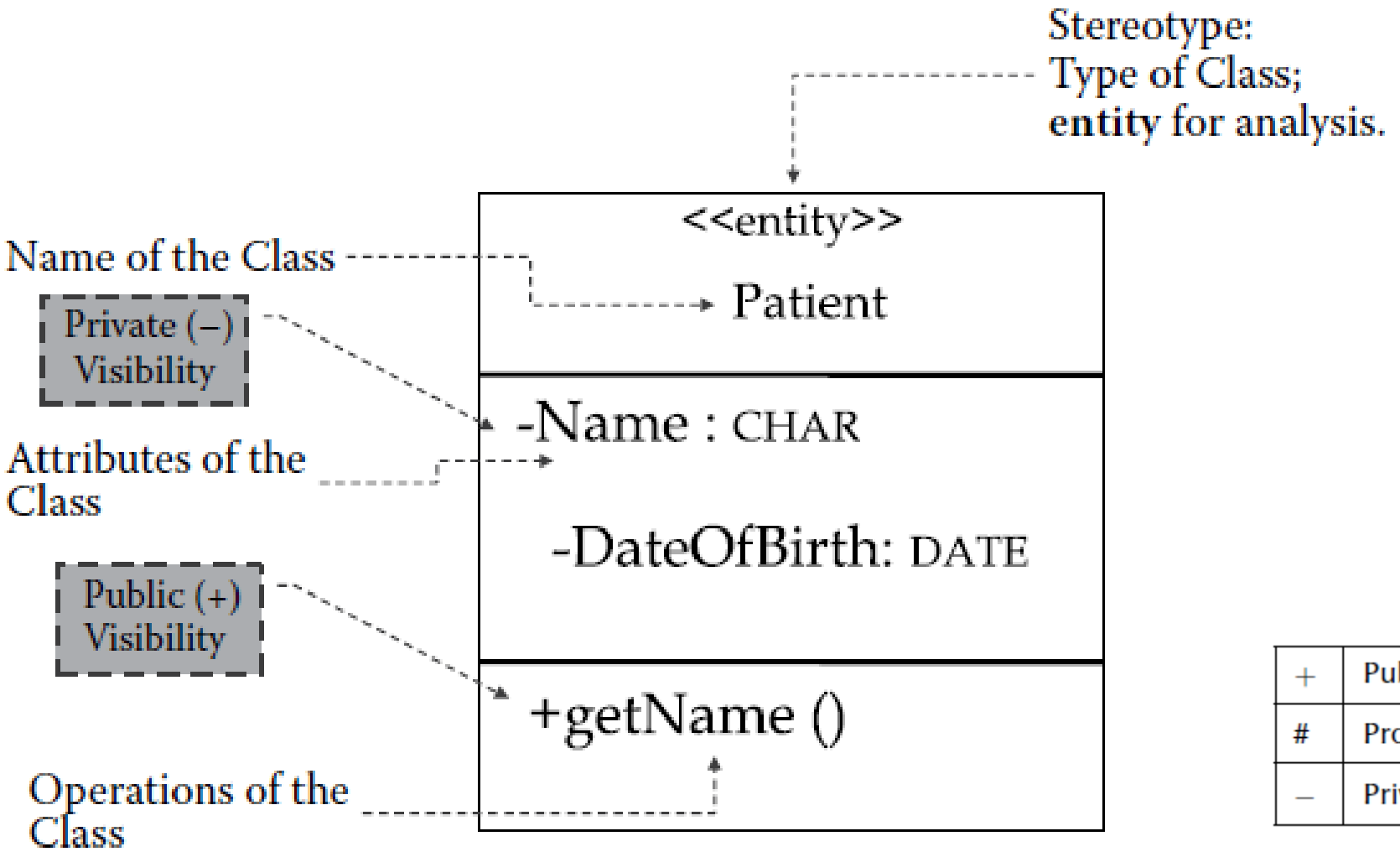
Association

Multiplicity
at each end
of association

Aggregation

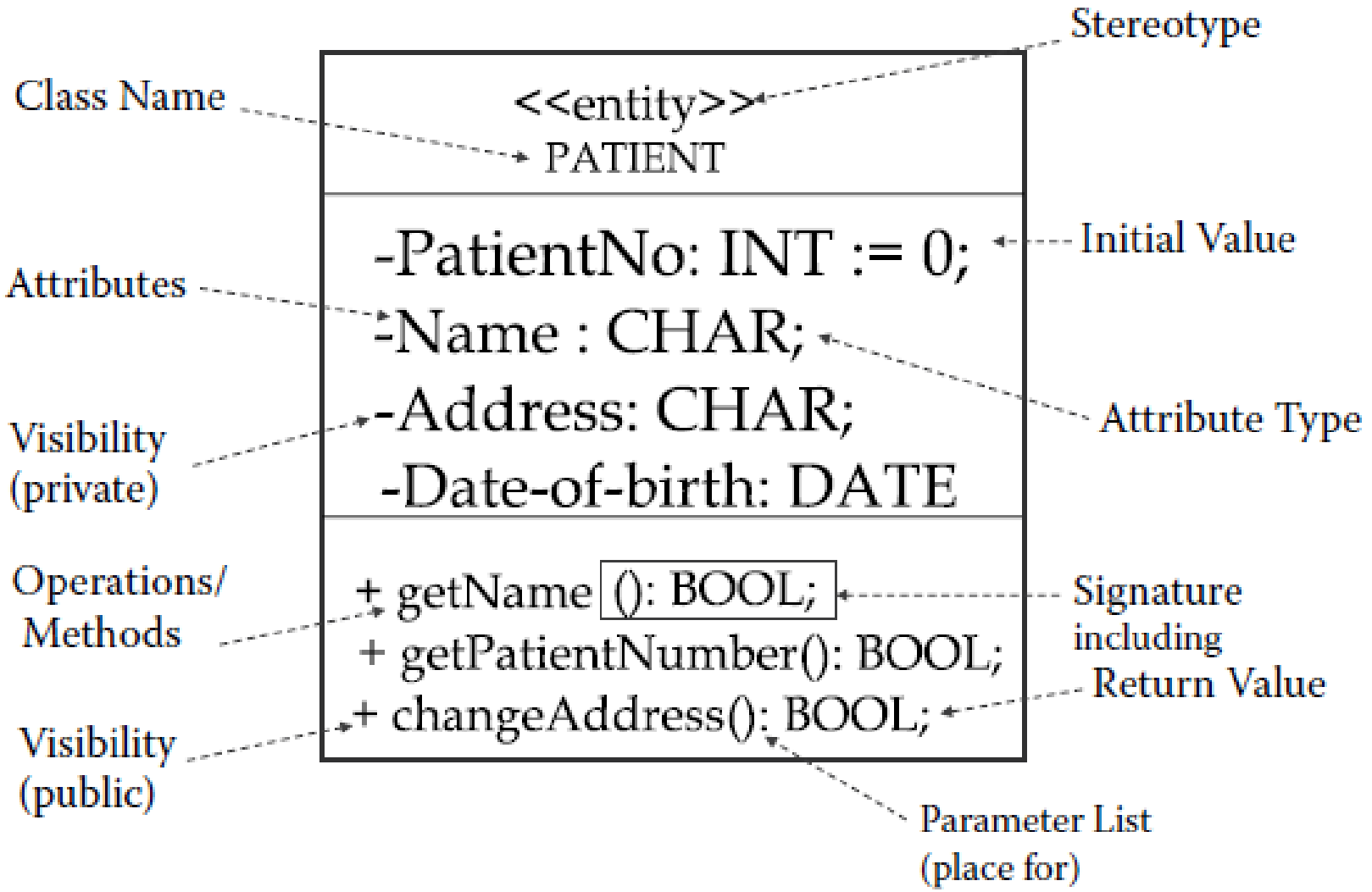
Notes

Representation of a class (notation).

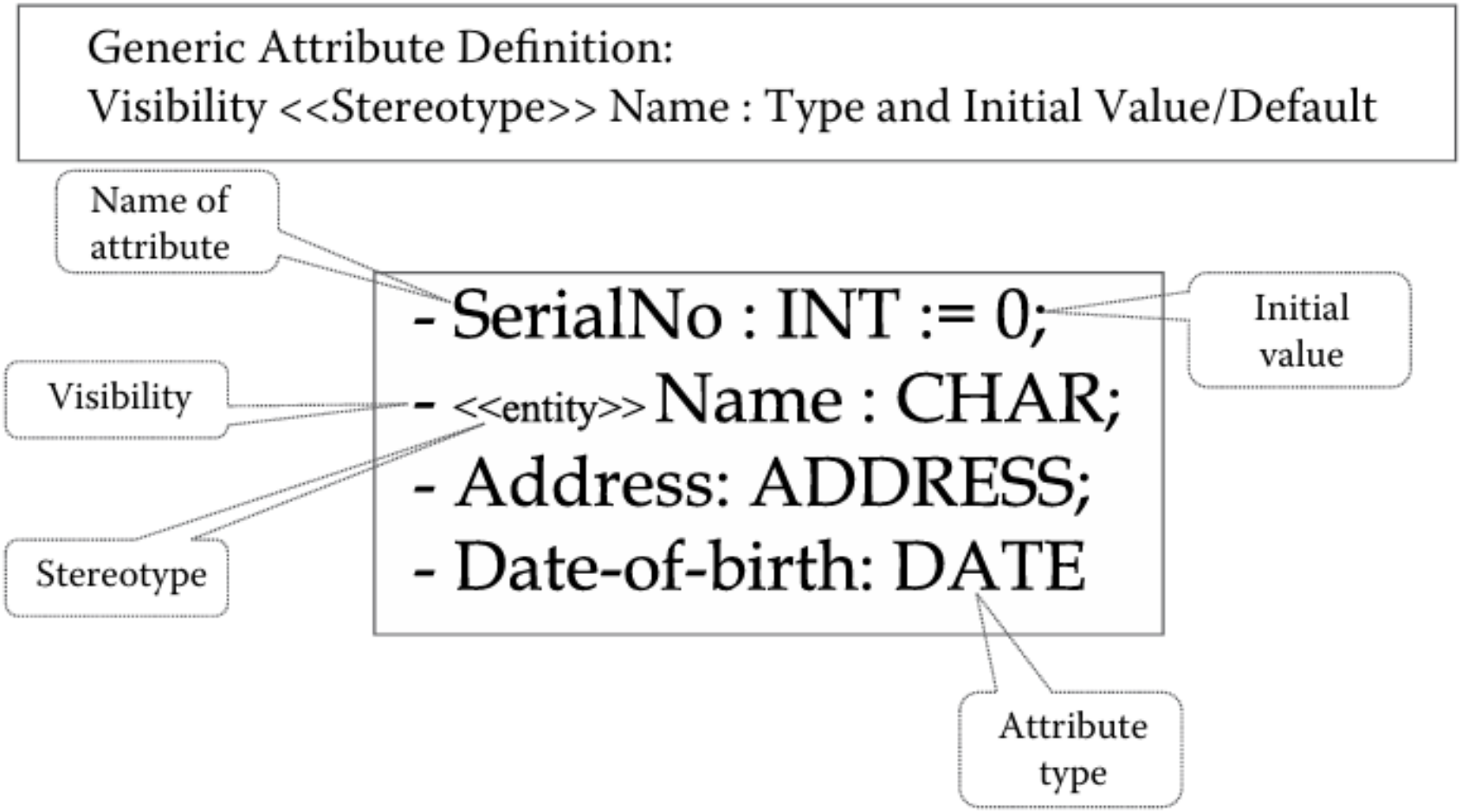


+	Public access
#	Protected access
–	Private access

Designing a Class in the Solution Space

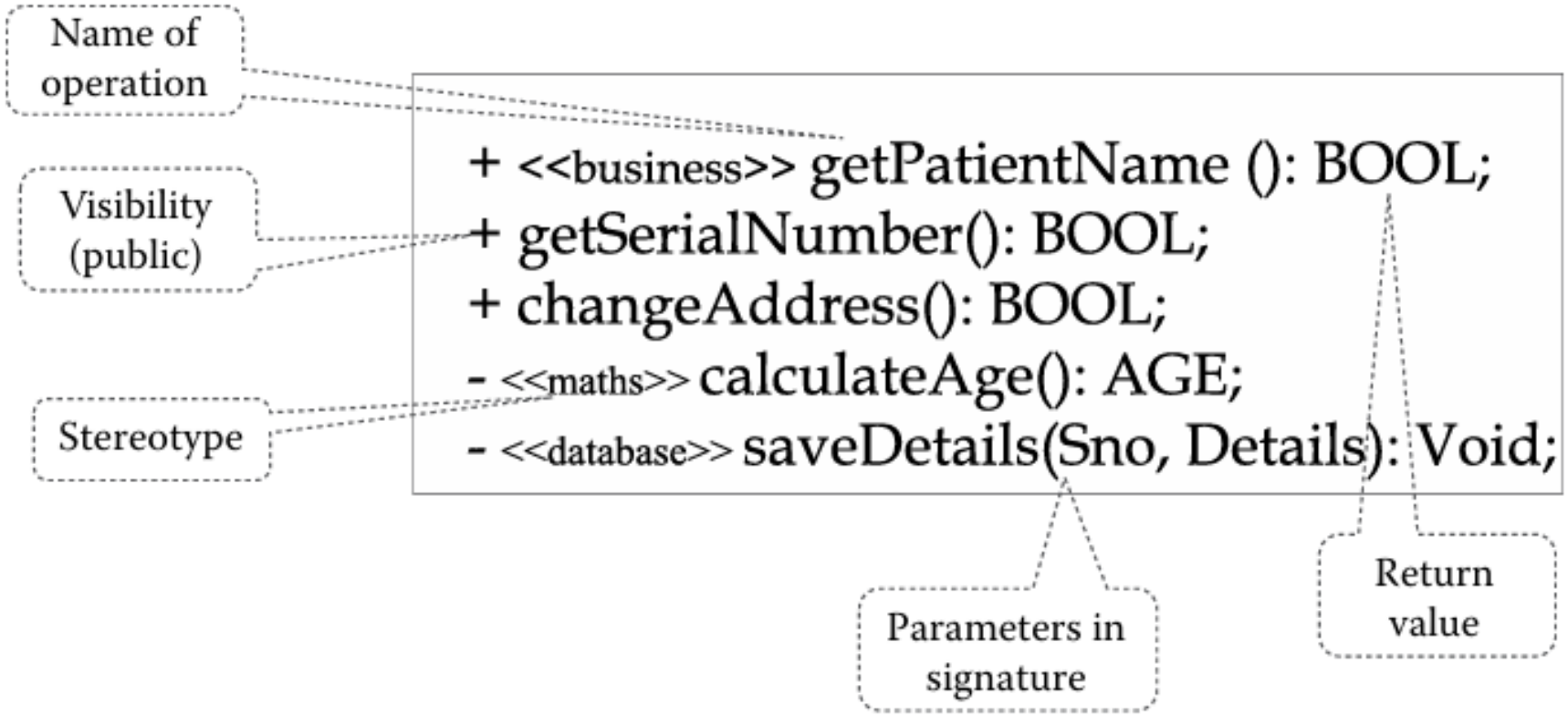


Defining and displaying attributes in design (showing visibility, types, and initial values)

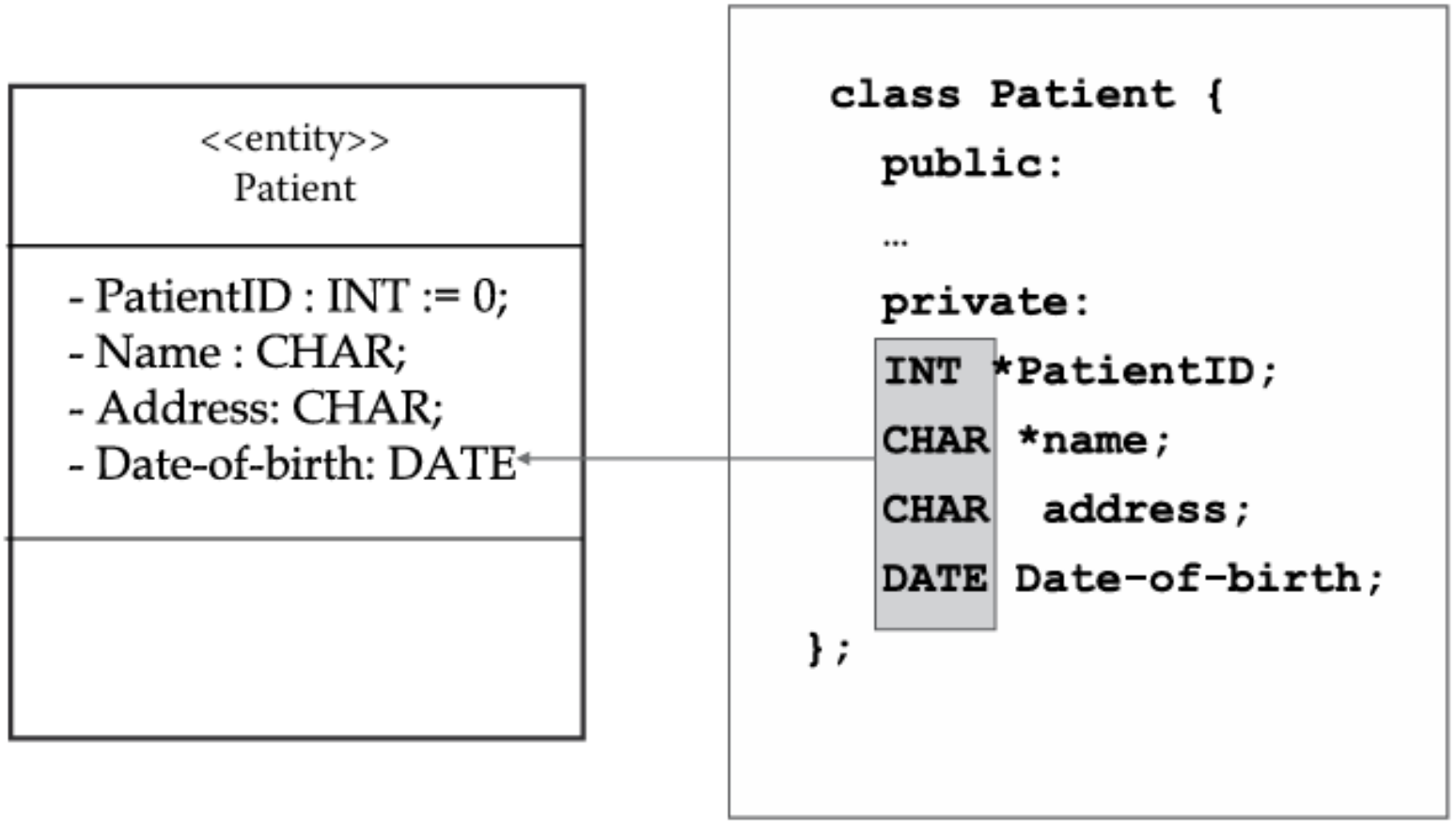


Defining and displaying operations in design

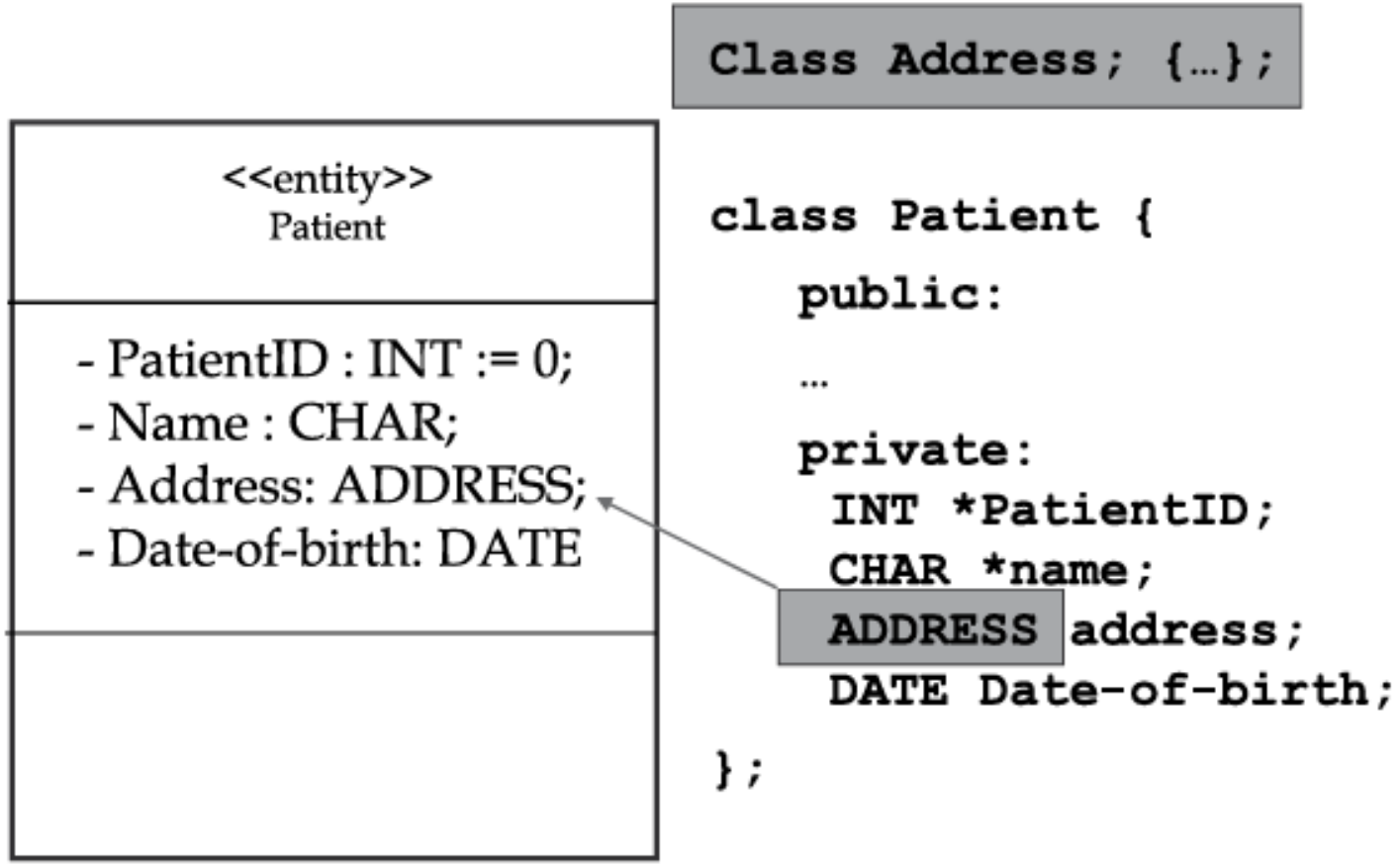
– Shown in the third compartment of a class



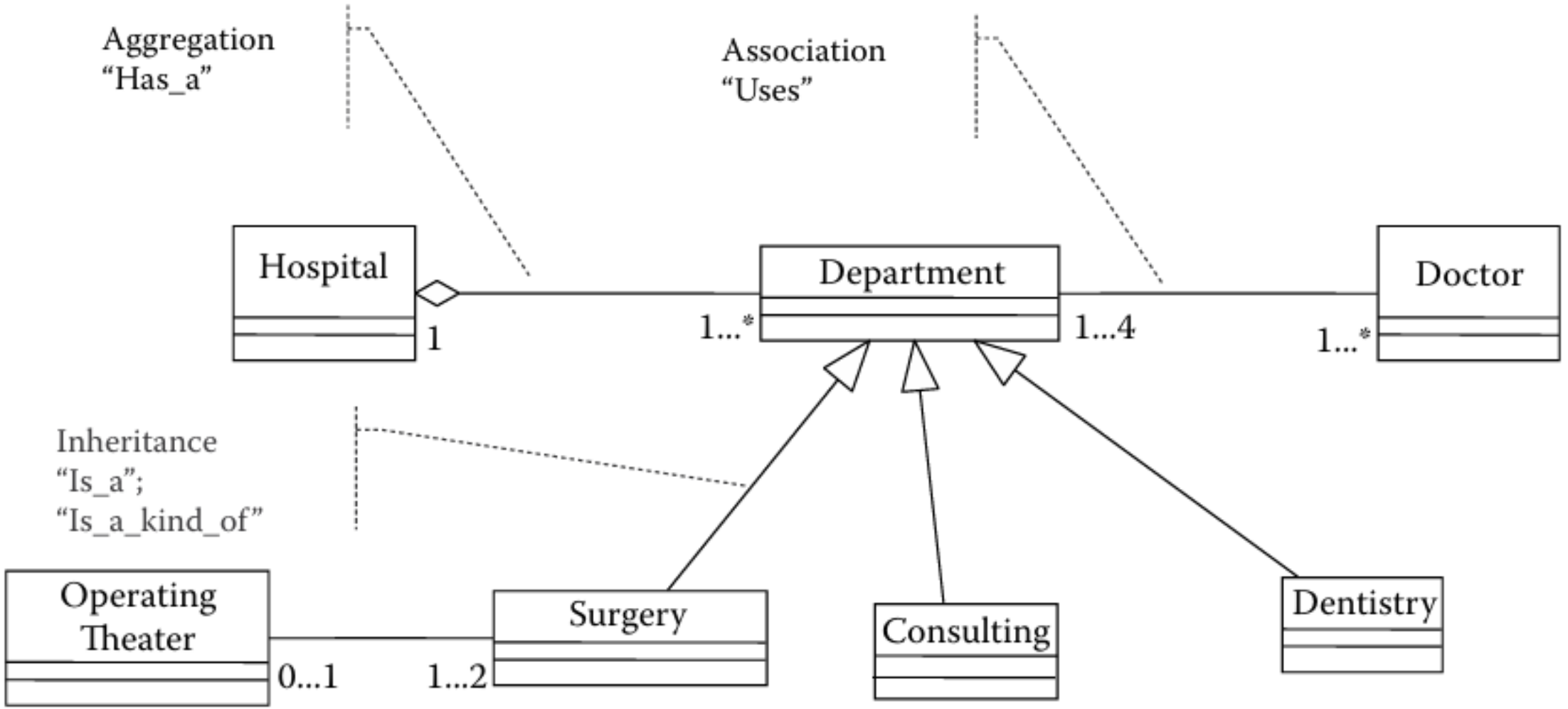
Attribute types and classes provided by implementation language



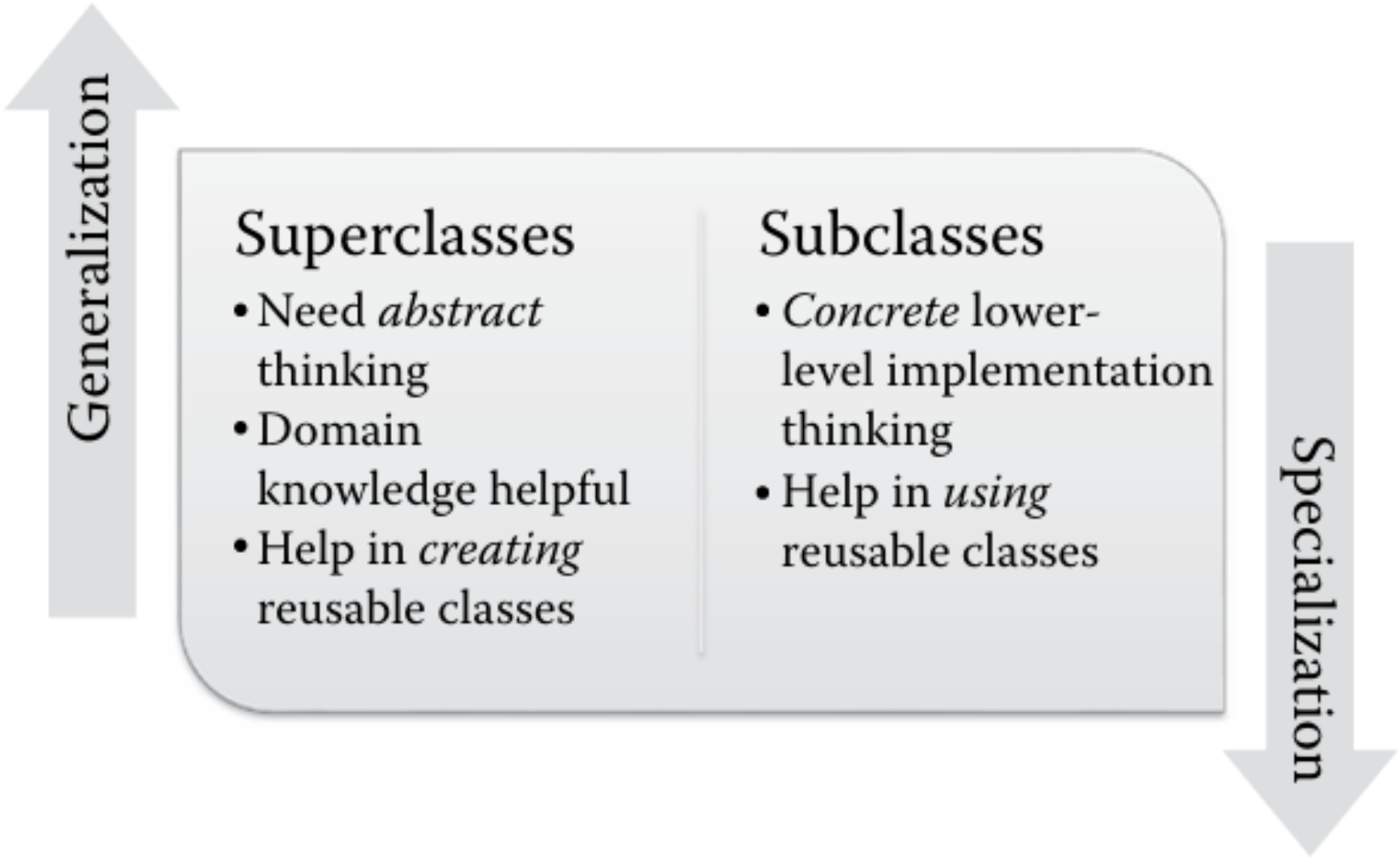
Attribute types and classes created by developers and designers.



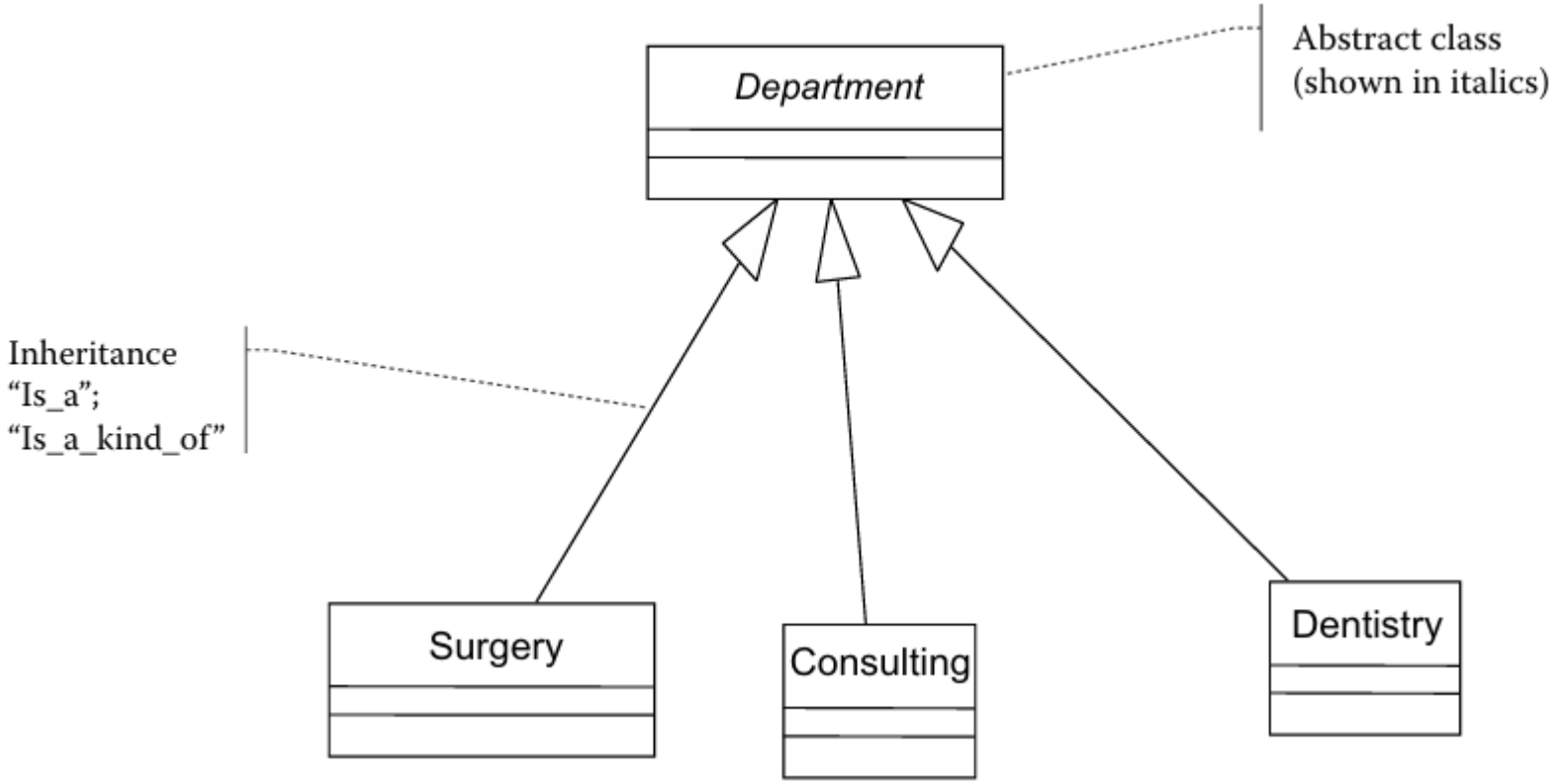
Relationships on a class diagram.



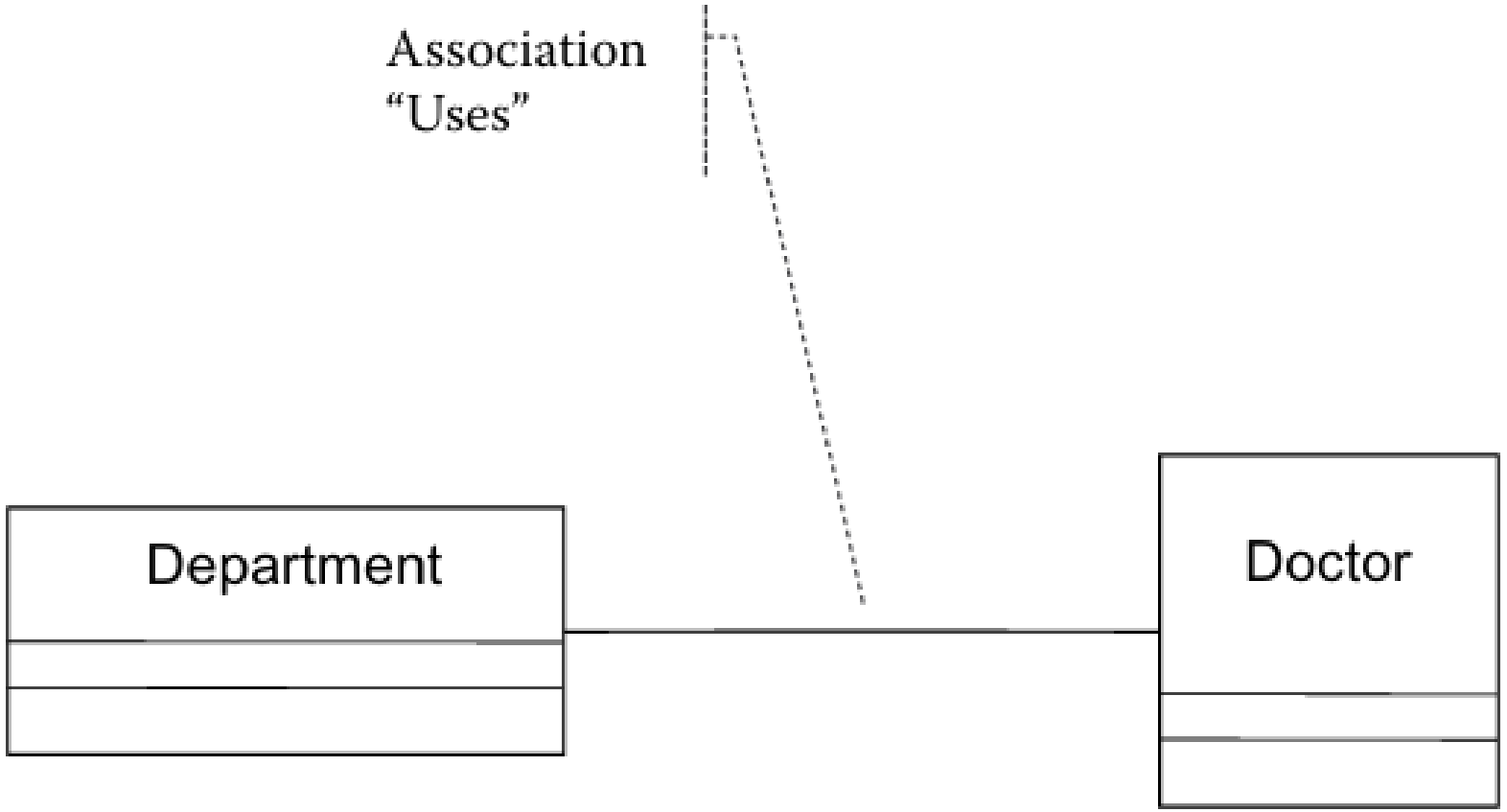
Generalization versus specialization



The inheritance relationship.

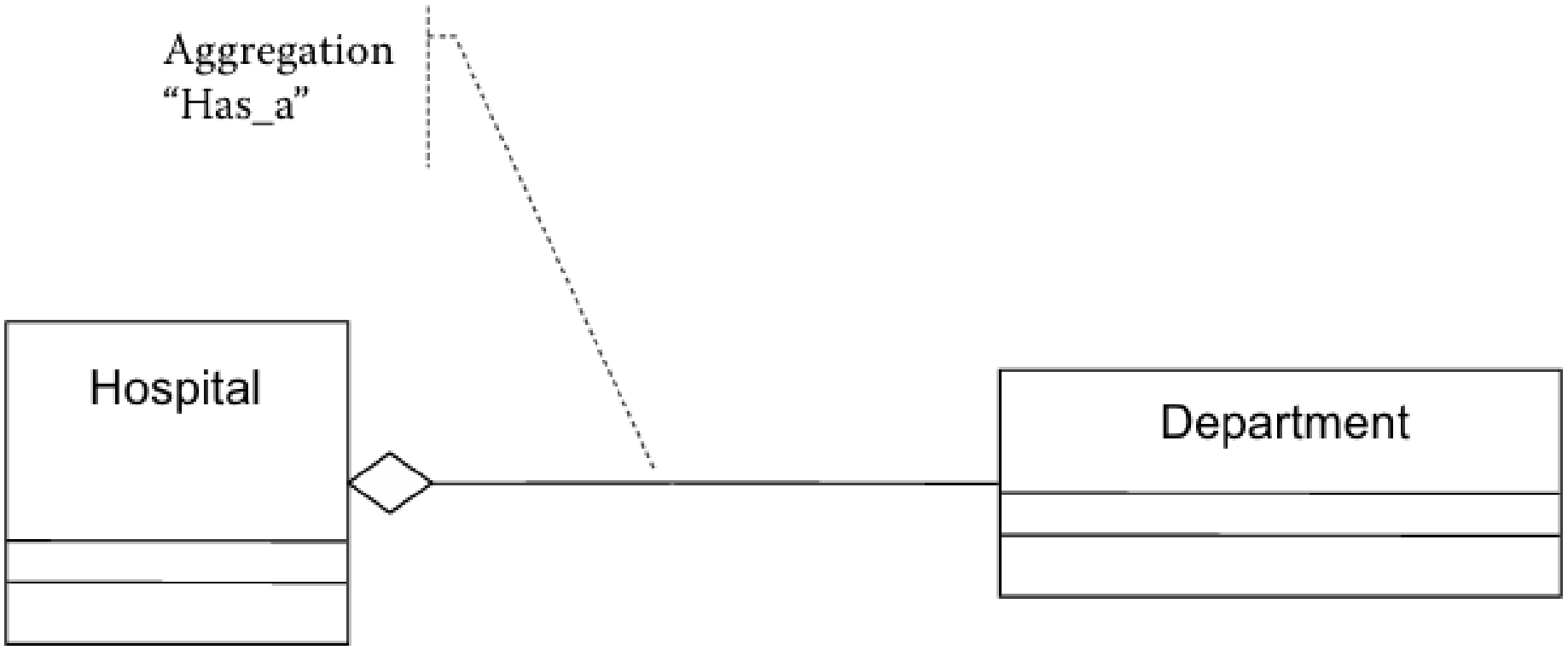


Association relationship on class diagrams



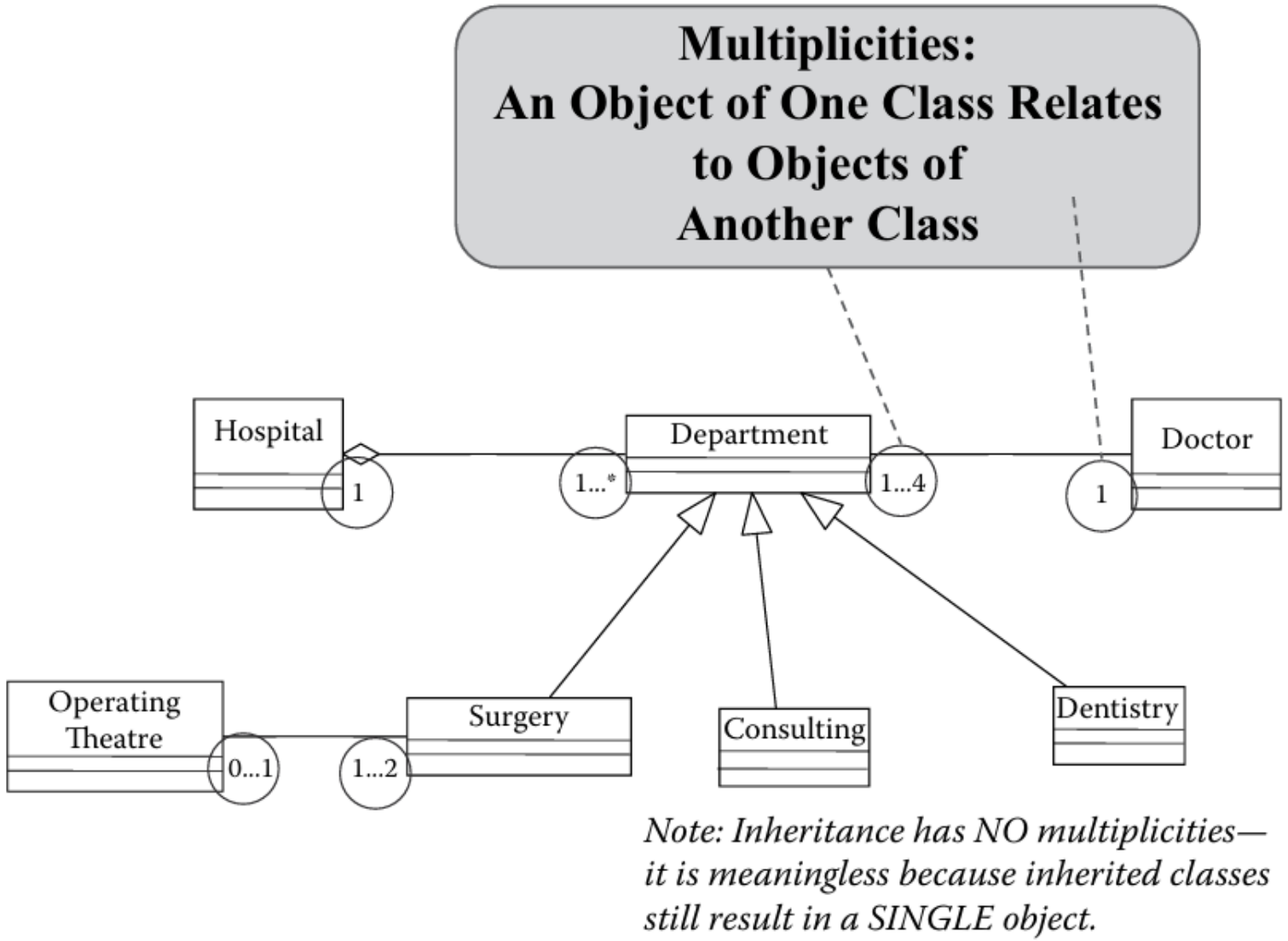
Doctor *uses* Department; Department also uses Doctor; The Relationship is *loose*.

Aggregation Relationship in a Class Diagram

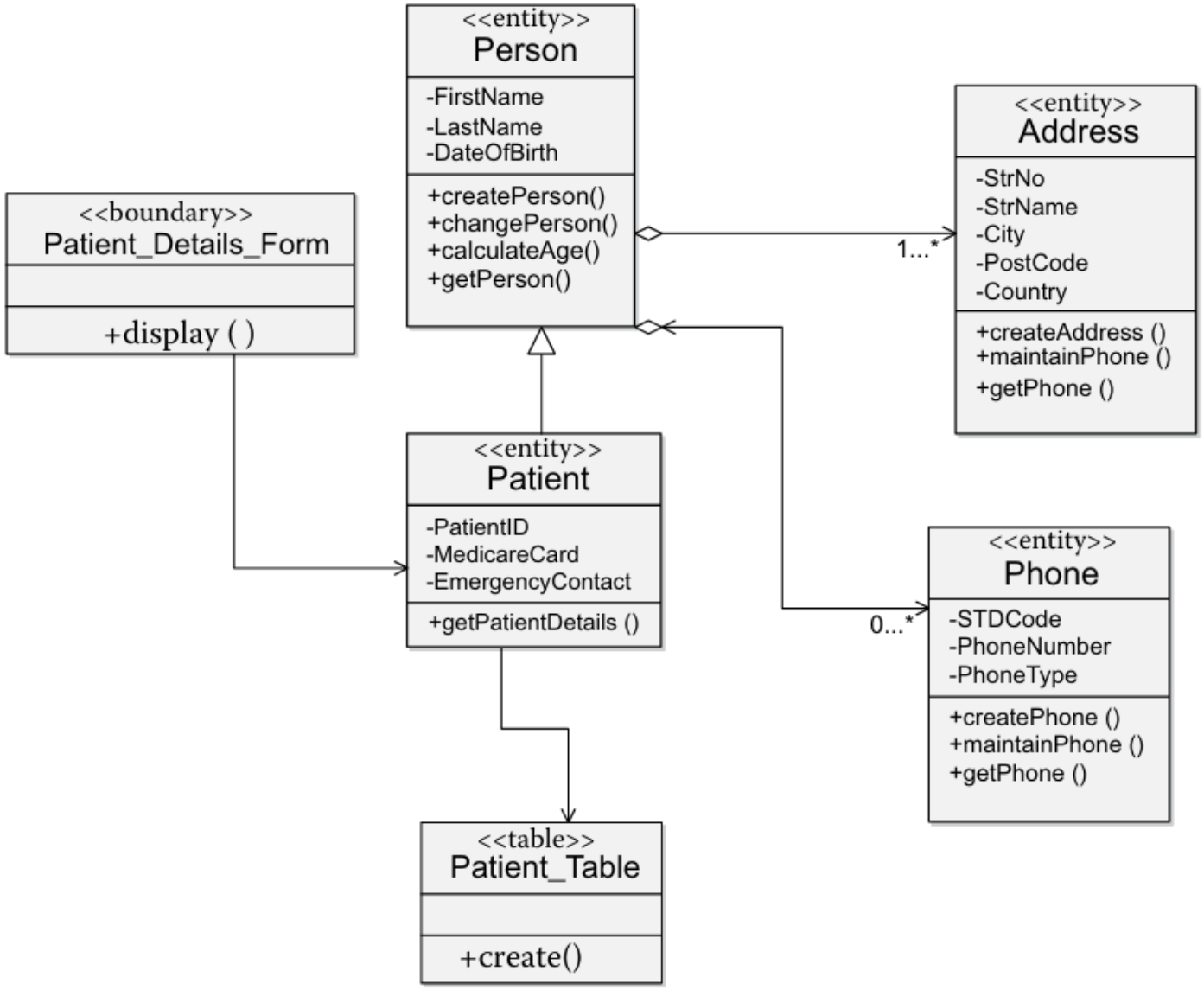


Hospital *has* Department;
Closer/Tighter Relationship

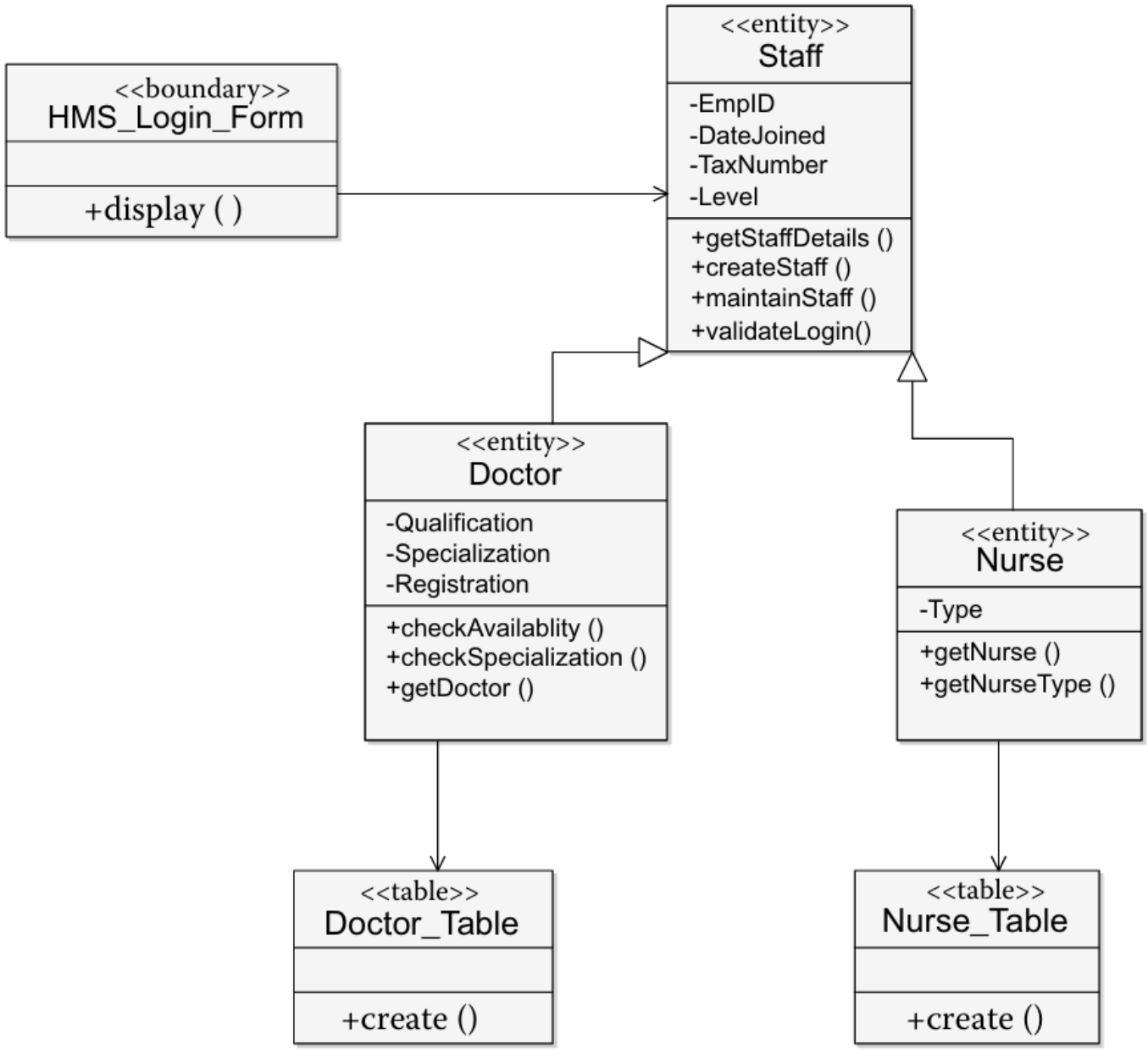
Multiplicities in class diagrams



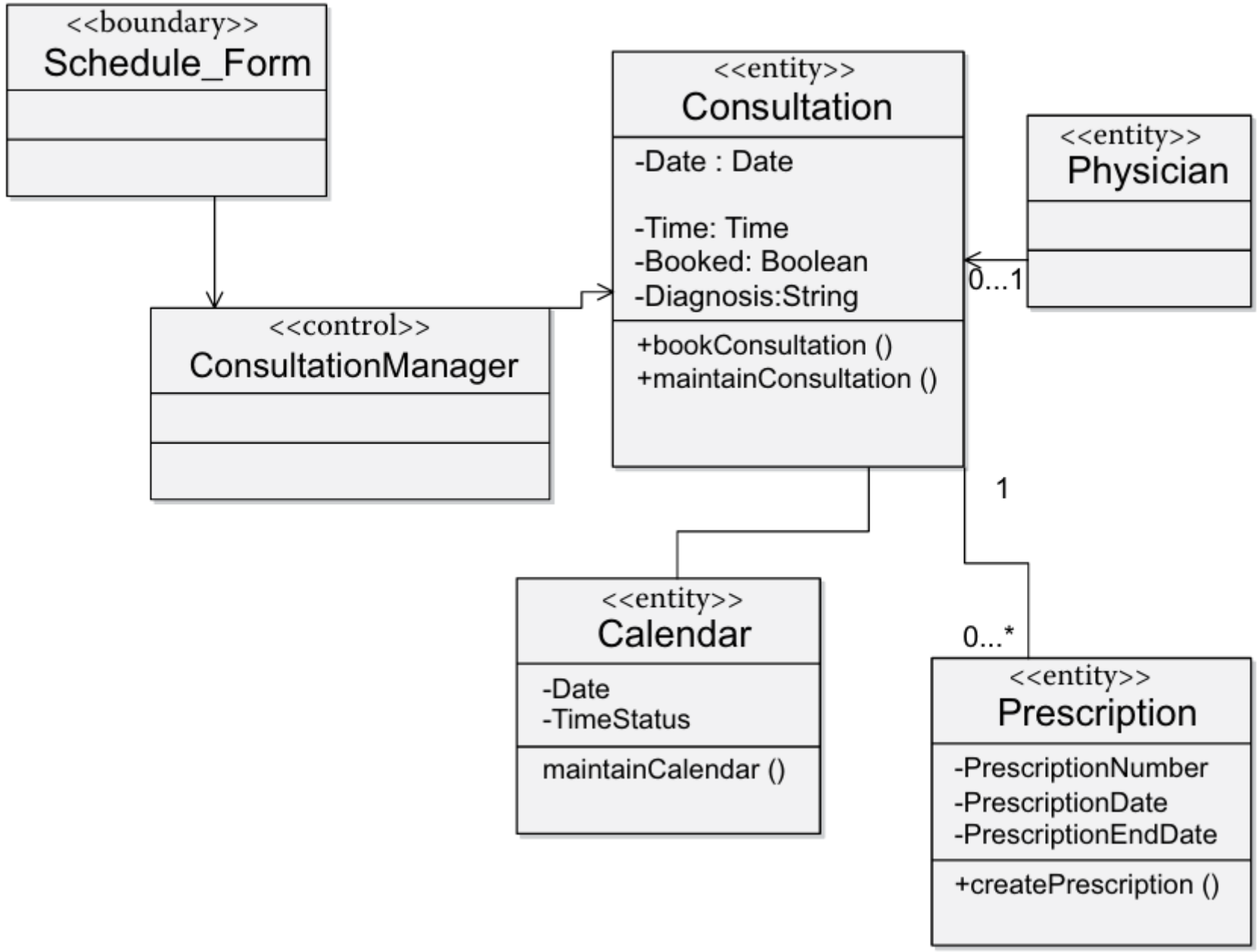
“Patient Details” class diagram



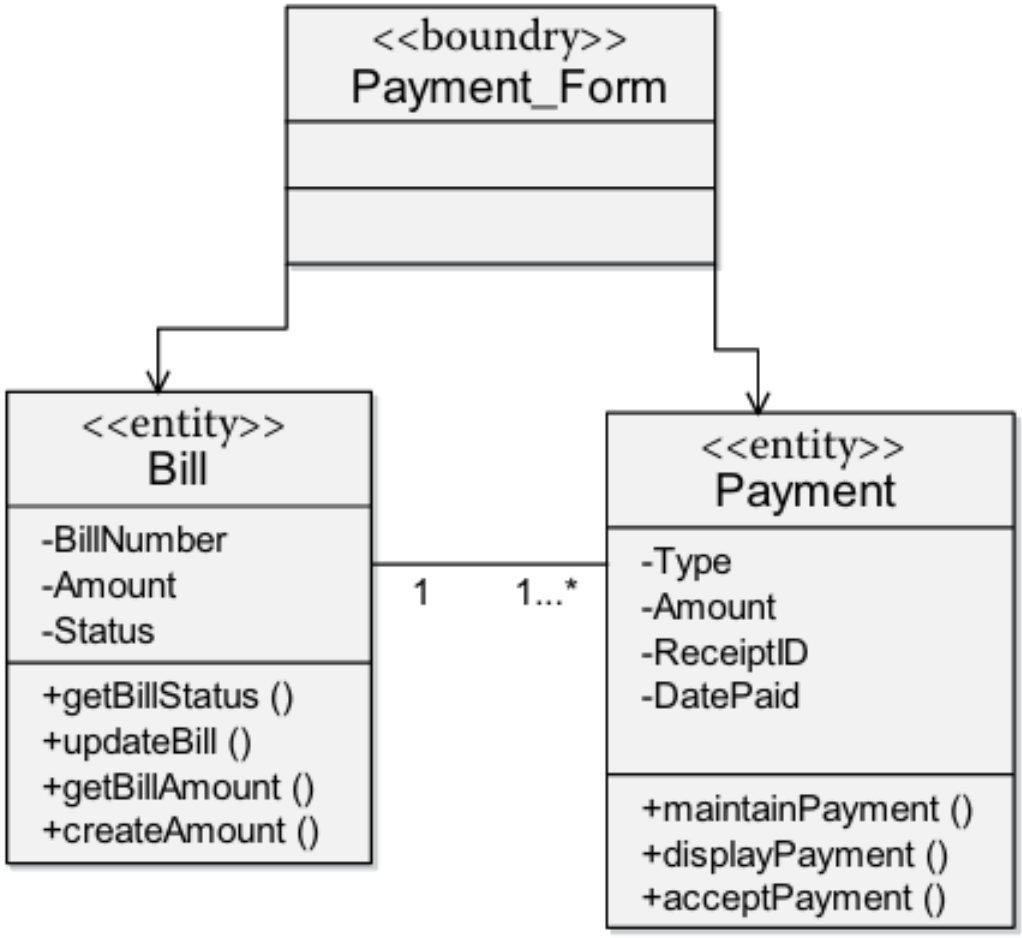
“Staff Details” class diagram



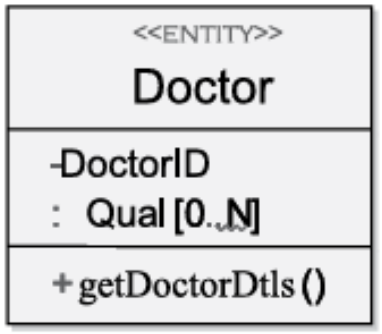
Consultation Details” class diagram.



“Accounting” class diagram



Notations of a class diagram in solution space.



Class description contains attributes & operations

Multiplicity at each end of association

Notes

Inheritance

Association

Aggregation

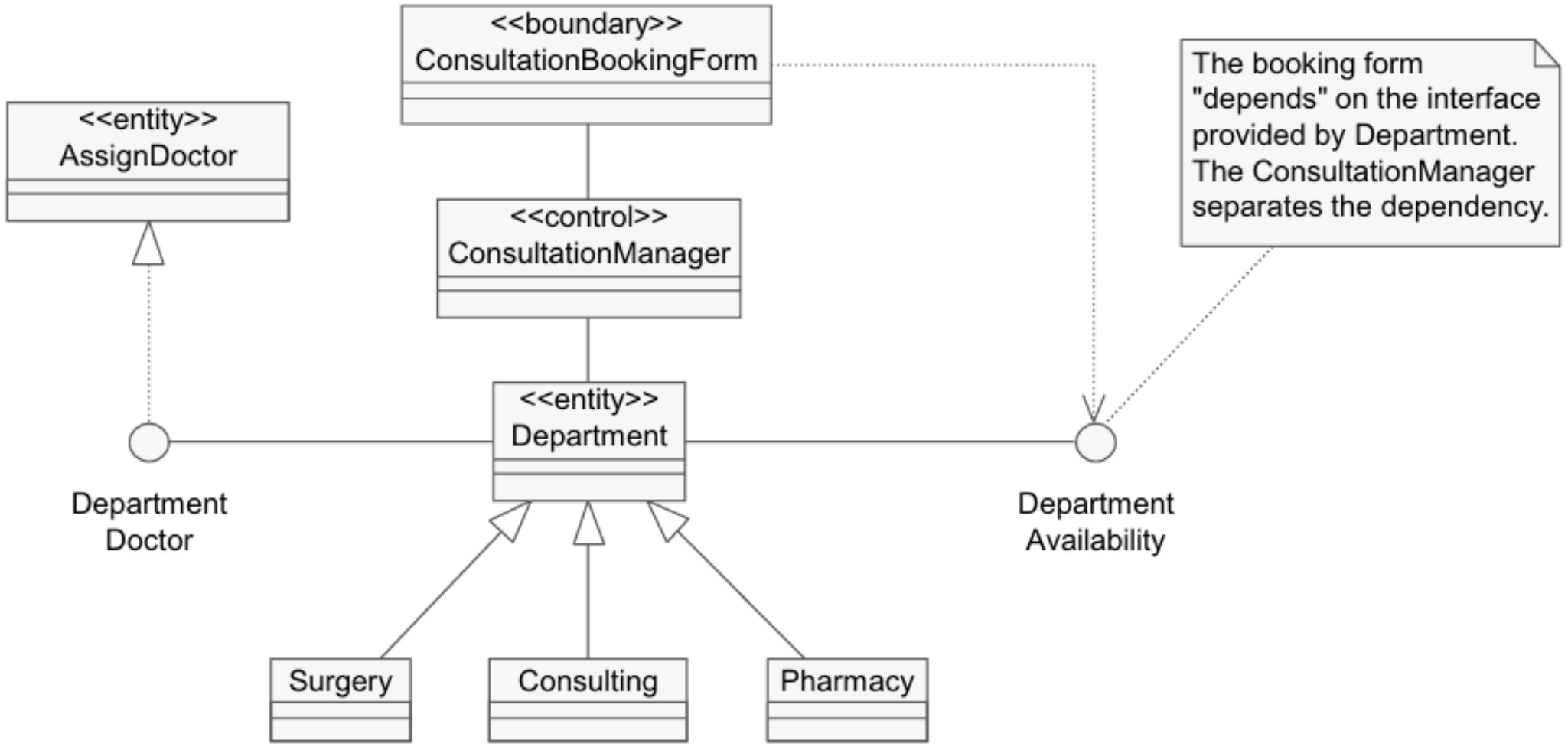
Composition

Realization

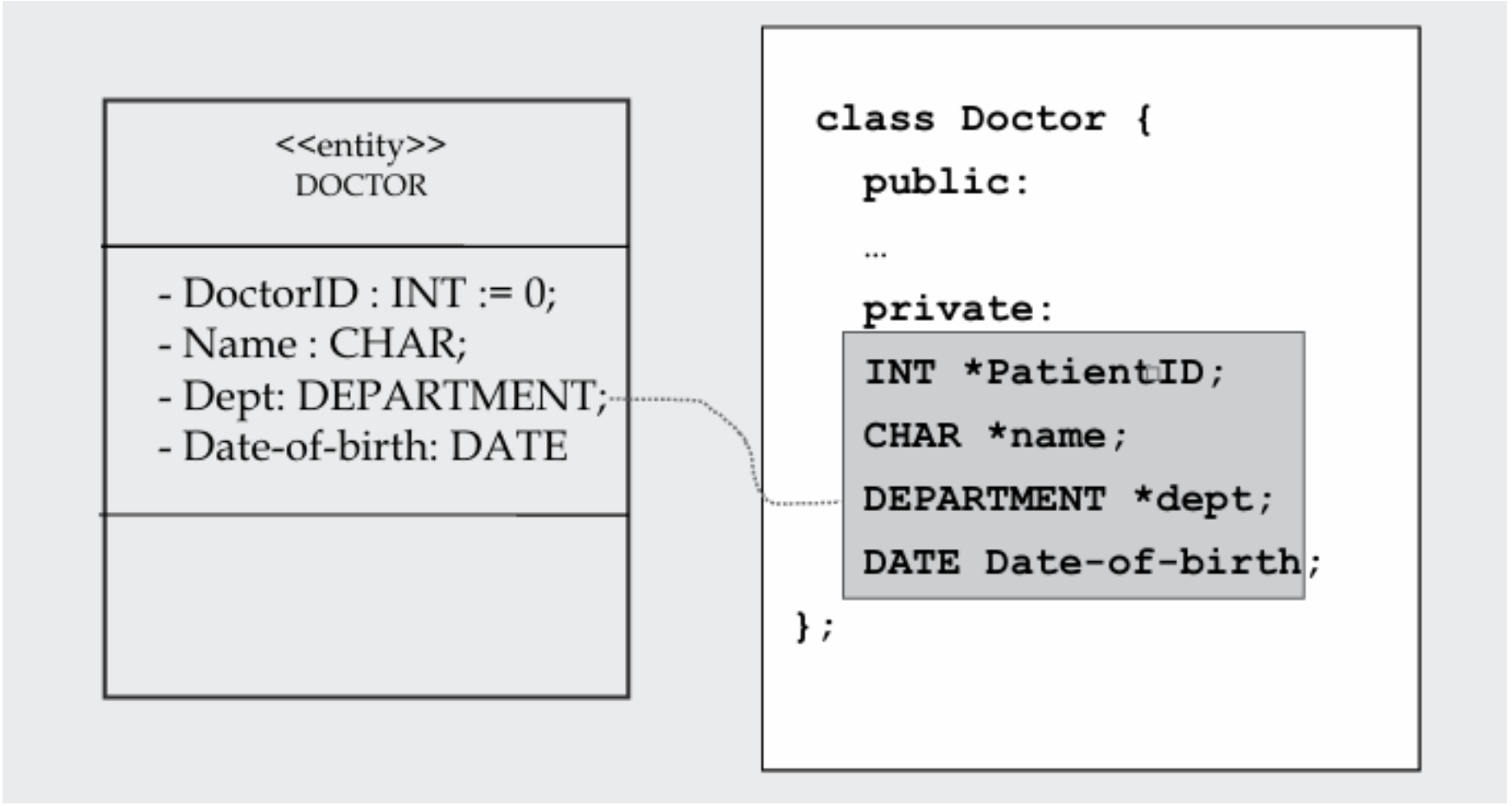
Interface

Dependency

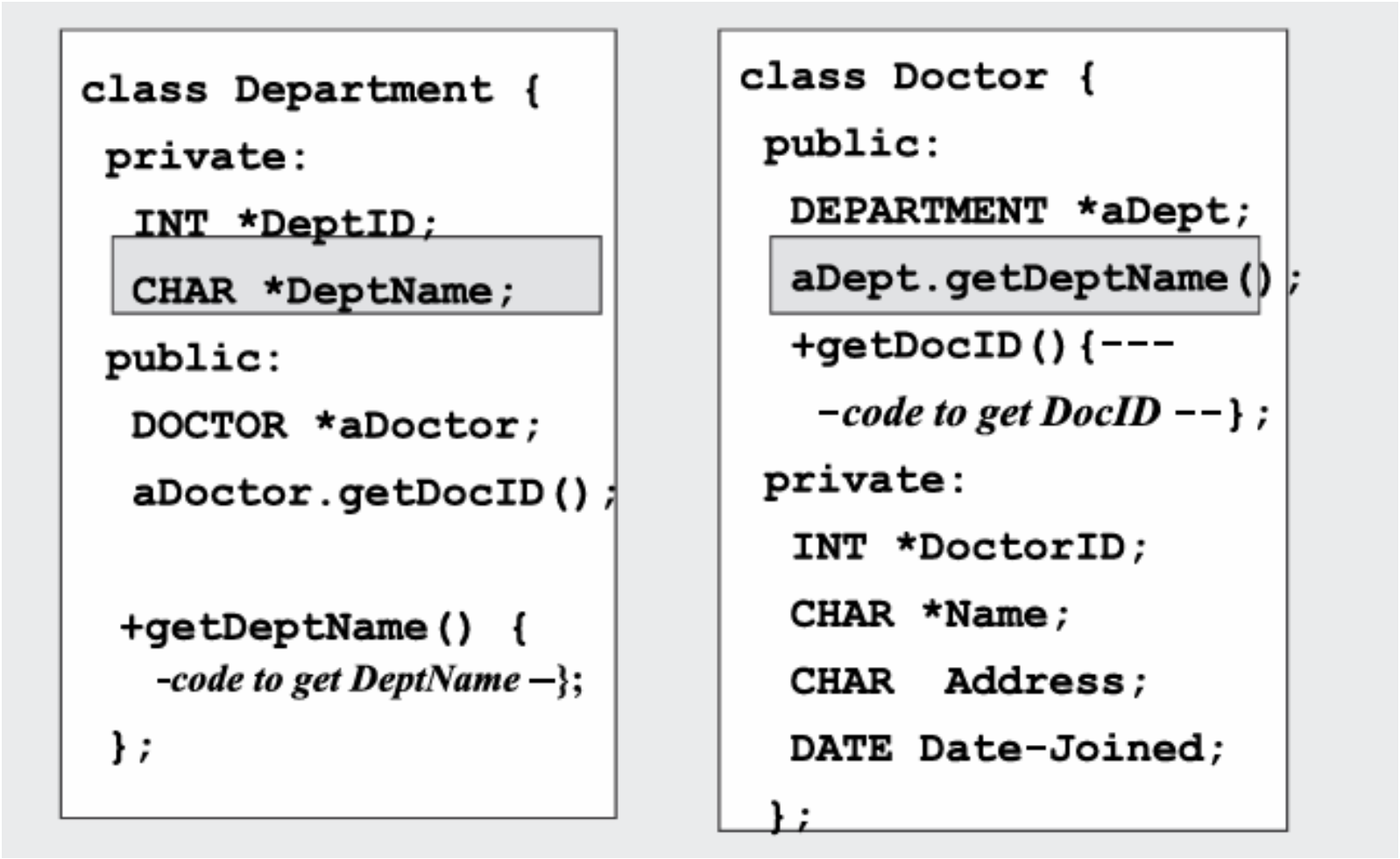
Advanced relationships in a class diagram in design



CODE EXAMPLE : FOR ASSOCIATION RELATIONSHIP: IN DOCTOR CLASS FOR ASSOCIATION RELATIONSHIP WITH DEPARTMENT



CODE EXAMPLE : FOR BIDIRECTIONAL ASSOCIATION RELATIONSHIP IN BOTH DEPARTMENT AND DOCTOR CLASSES FOR BIDIRECTIONAL ASSOCIATION RELATIONSHIP

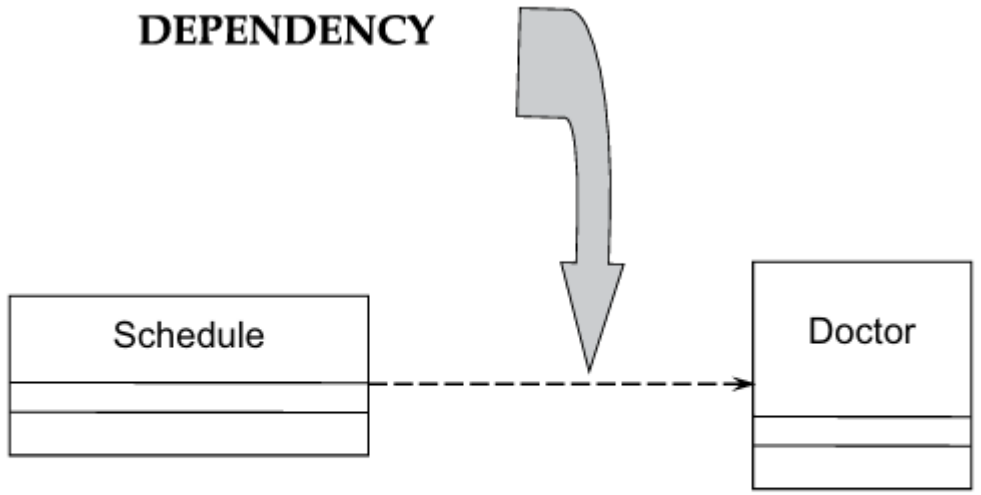


CODE EXAMPLE : FOR DEPENDENCY RELATIONSHIP

```
class Schedule {
  public:
  DOCTOR aDoctor;
  private:
  CALENDAR calendar;
 };
```

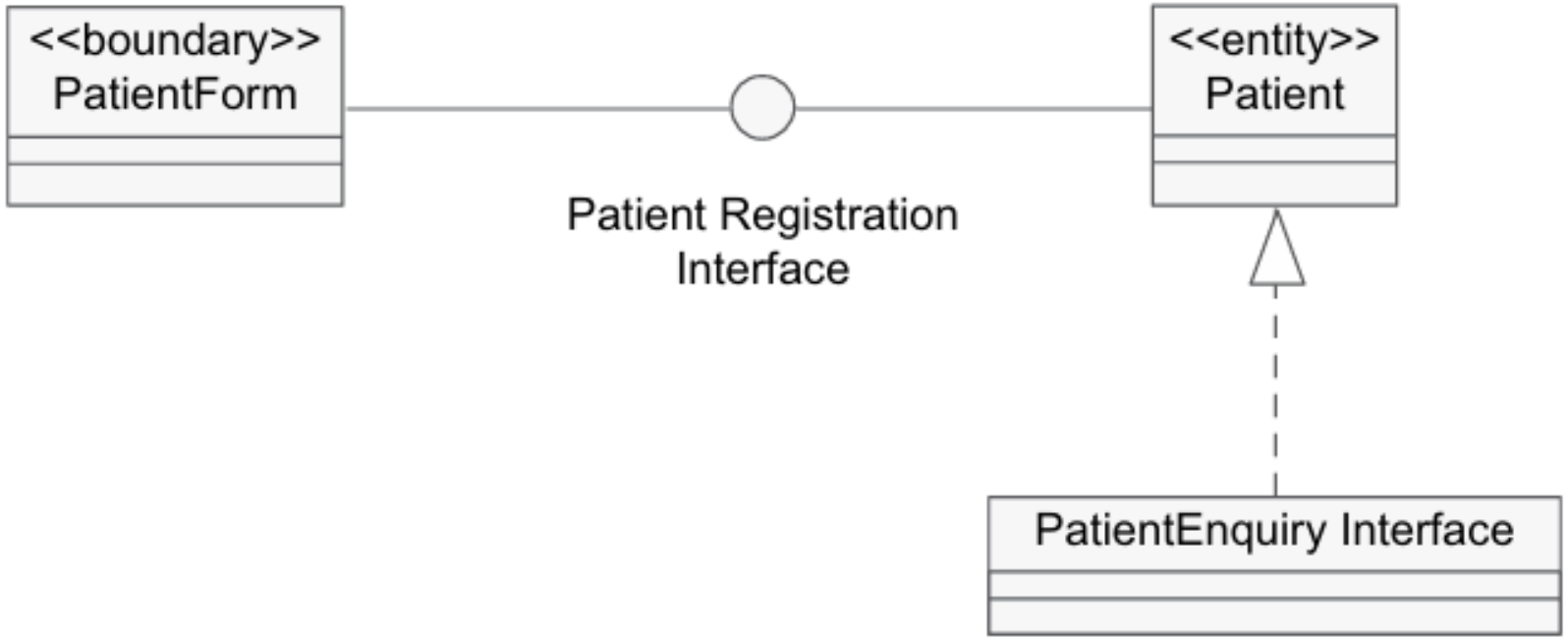
```
class Doctor {
  public:

  private:
  INT *DoctorID;
  CHAR *name;
  CHAR  address;
  DATE Date-Joined;
 };
```



Schedule here depends on Doctor;
changes to Doctor object will affect the Schedule object,
but not the other way around.

Relating two classes through interface

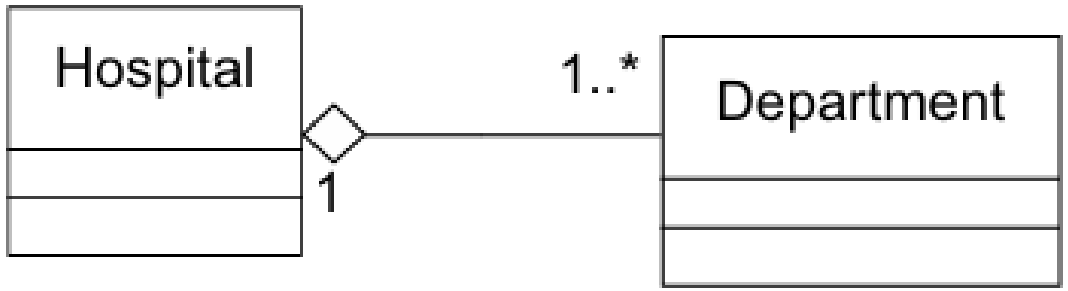


“By reference” aggregation relationship in design

“By Reference” implies only a reference (or pointer) to the object (e.g., Department). Therefore, it is possible for other objects to point to the same Doctor object.

Note: Association is also implemented “By Reference”

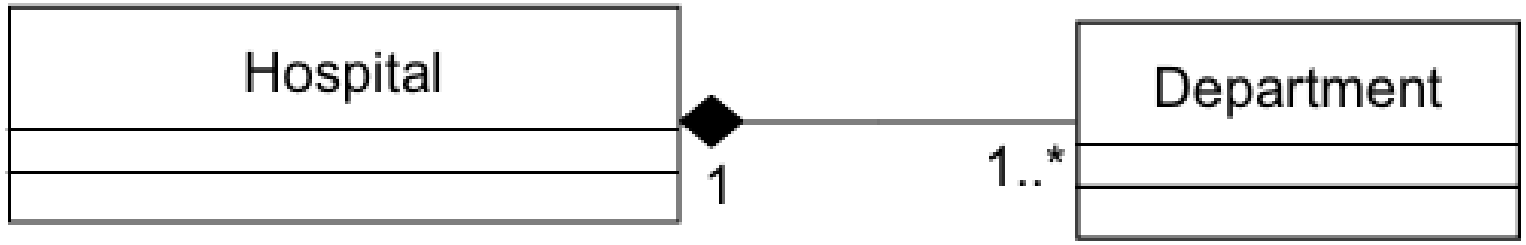
By reference



“By value” aggregation relationship in design.

In implementing “by value,” a copy of the aggregate parts (e.g., Department) is made. The “contained” Department is not shared by other objects in the system

By Value

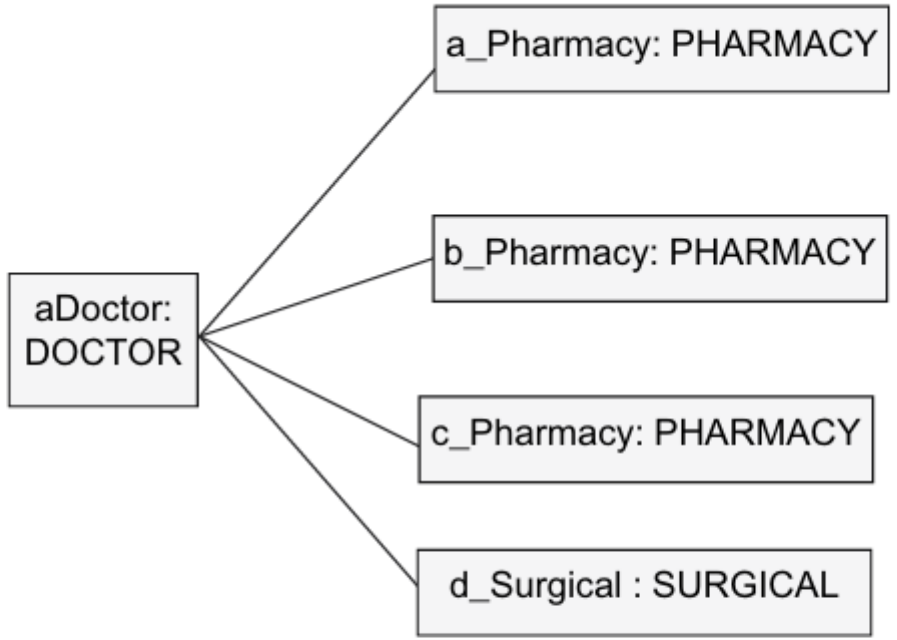


CODE EXAMPLE: FOR AGGREGATION RELATIONSHIP IN JAVA

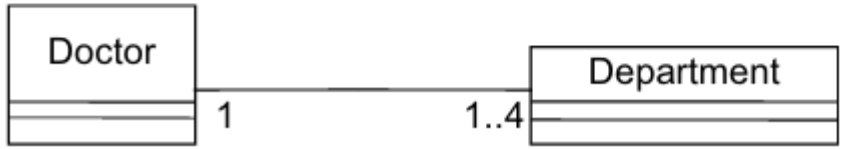
```
class Department
{
  int DeptID;
  String DeptName;
  int[] AllDoctors;
  int HospitalID;
  public String getDeptName()
  {
    //code to get DeptName
  }
  public int getHospitalID()
  {
  }  public String getHospitalName()
  {
  }
  public int getDoctors(Doctor DoctorArray[])
  {
  }
}class Hospital
{
  int HospitalID;
  String HospitalName;
  Department[] AllDepartments;
  Address HospitalAddress;
  public String getHospitalName()
  {
  }
  public int getDepts(Department DepartmentArray[])
  {
    //get list of departments in the hospital
  }
}
```

Object diagrams to interpret multiplicities

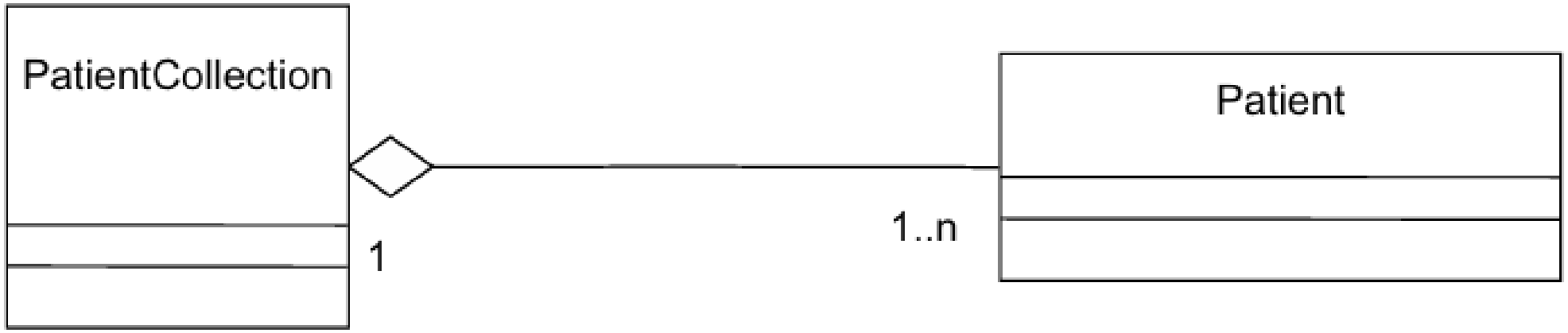
An object diagram



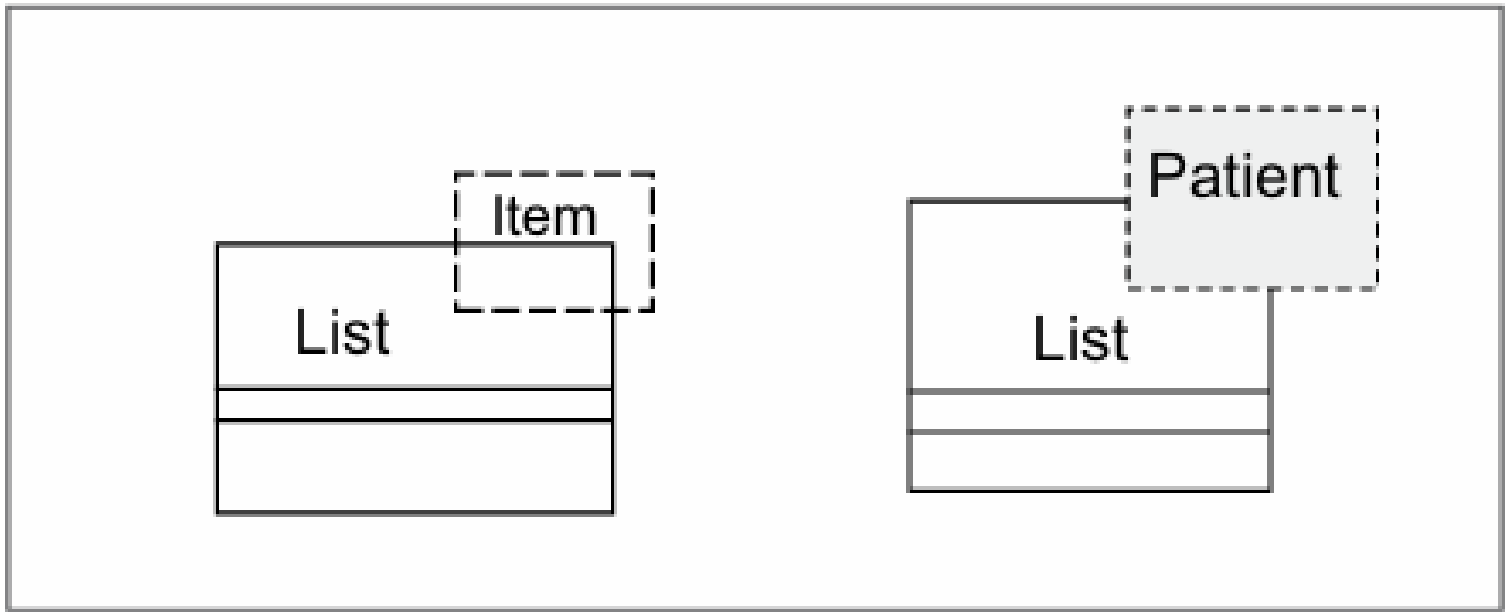
Corresponding to this class diagram



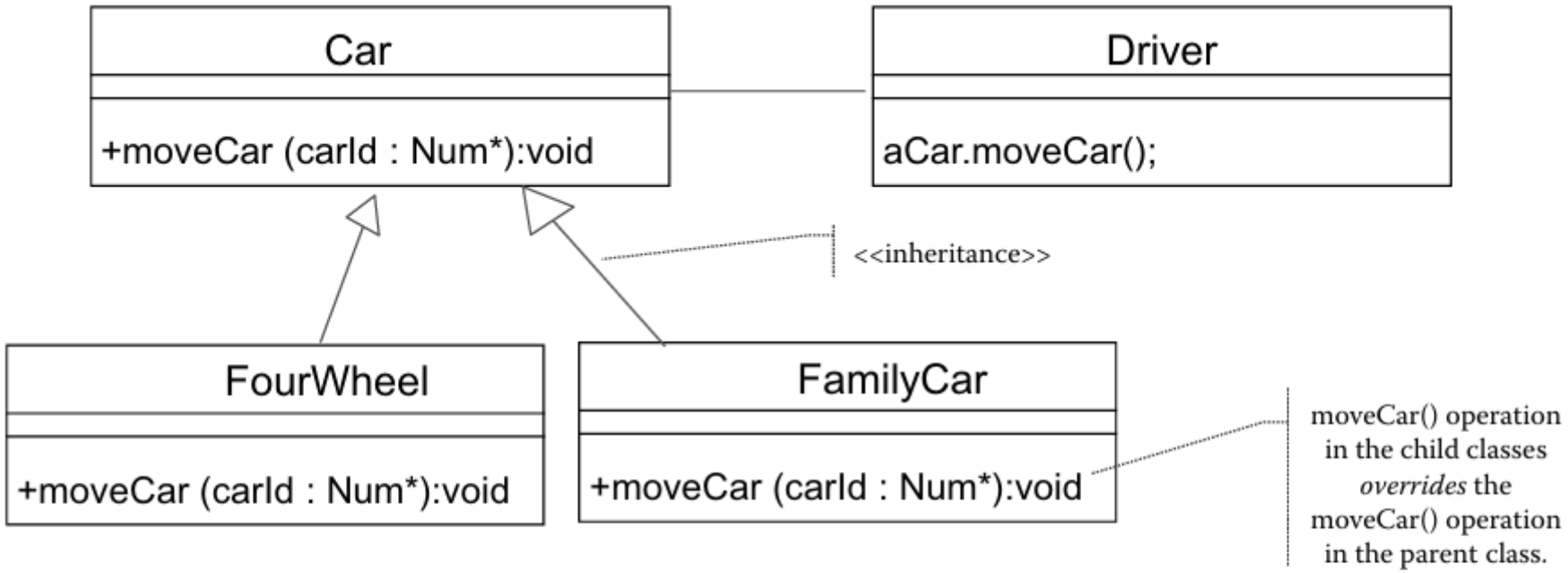
A collection class.



A parameterized class for designing a multiplicity of one or more



A parameterized class for designing a multiplicity of one or more



Opportunity for polymorphism at RUN TIME is created due to DESIGN time:

- Inheritance

- Operator Overloading

CODE EXAMPLE: FOR INHERITANCE RELATIONSHIP

```
public class Department {
   public void setDeptDtls() {   };
   public void changeDeptDtls() {   };

   private char DeptID;
   private double Capacity;
  }
-------------------------------------
public class Surgery extends Department {
   public Surgical (); {
   }
   public void operate(); {  }
   public void bookOperation (); {  }
   public void releaseTheatre(); {  }

   private char Name;
   private int NoOfRoom;
   }
-------------------------------------
```

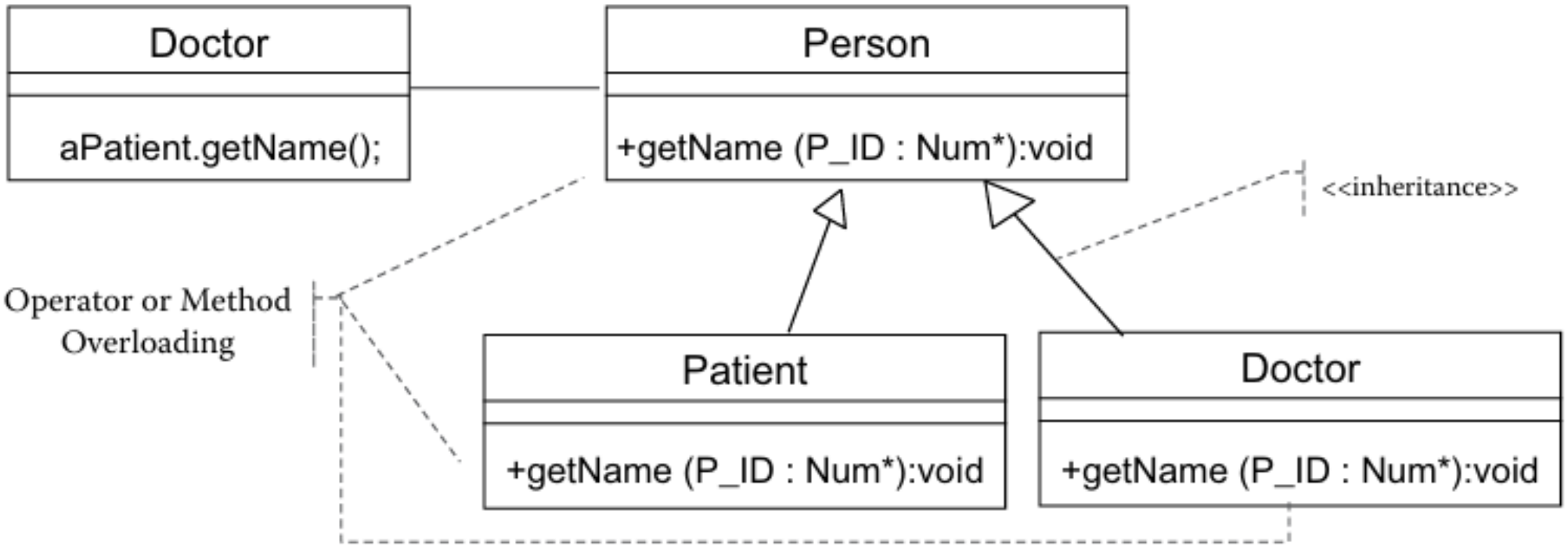
CODE EXAMPLE: SIMPLE CODE EXAMPLE FOR POLYMORPHISM

```
Class Driver:
 Car myCar;
 myCar := new FourWheel;
 myCar.moveCar();

 myCar := new FamilyCar;
 myCar.moveCar();
```

This pseudo-code indicates that when the driver object issues a call to moveCar, whichever object has been instantiated (either FourWheel or FamilCar) will move.

Another example of polymorphism and operation overloading



CODE EXAMPLE: ANOTHER SAMPLE CODE EXAMPLE IMPLEMENTING POLYMORPHISM USING JAVA

```
class Person //this is the Super class
{
public void getName()
{
System.out.println("This is super class");
}
}
class Doctor extends Person //Sub-class of the Person
{
public void getName() //Inherited Method which overridees the
getName() of Person
{
System.out.println("This is sub-class Doctor");
}
}
class Patient extends Person //another sub class of the Person class
{
public void getName() //Inherited Method which also overrides the
getName() of Person
{
System.out.println("this is sub-class Patient");
}
}
```

CODE EXAMPLE: SAMPLE CODE EXAMPLE TO EXECUTE POLYMORPHISM

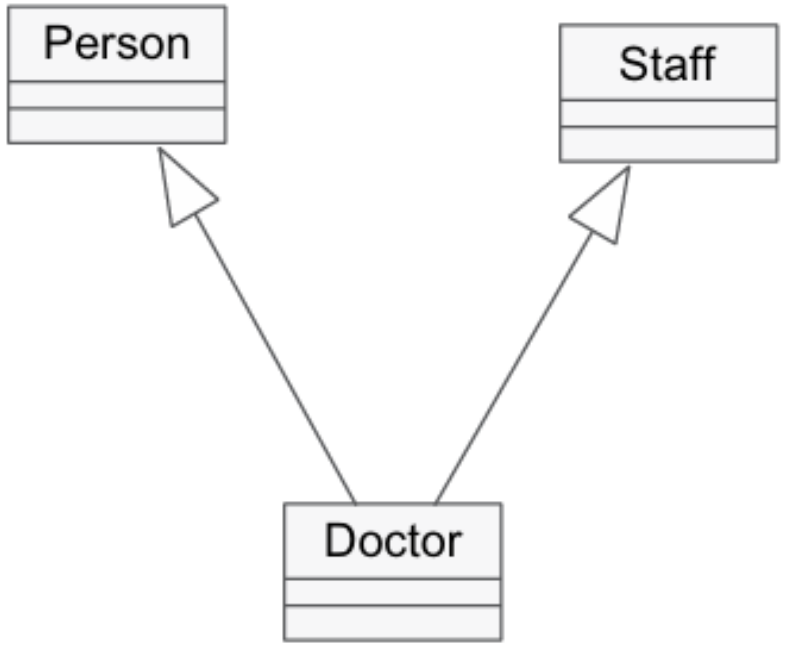
```
class Execute
{
public void ExecuteObject(Person p) // This is where the polymorphic
object is passed as a parameter!
{
p.getName();
}
}
class PolyMorphic // this is to execute the polymorphic class
{
public static void main(String args[])
{
Execute e = new Execute(); //Polymorphic class
Doctor aDoctor = new Doctor();
e.ExecuteObject(aDoctor); //executes Doctor's method
Patient aPatient = new Patient();
e.ExecuteObject(aPatient); //executes Patient's method
}
}
```

Multiple inheritance

Multiple Inheritance
IS_A_KIND_OF *both;*

Doctor inherits properties and behavior of both Person and Staff.

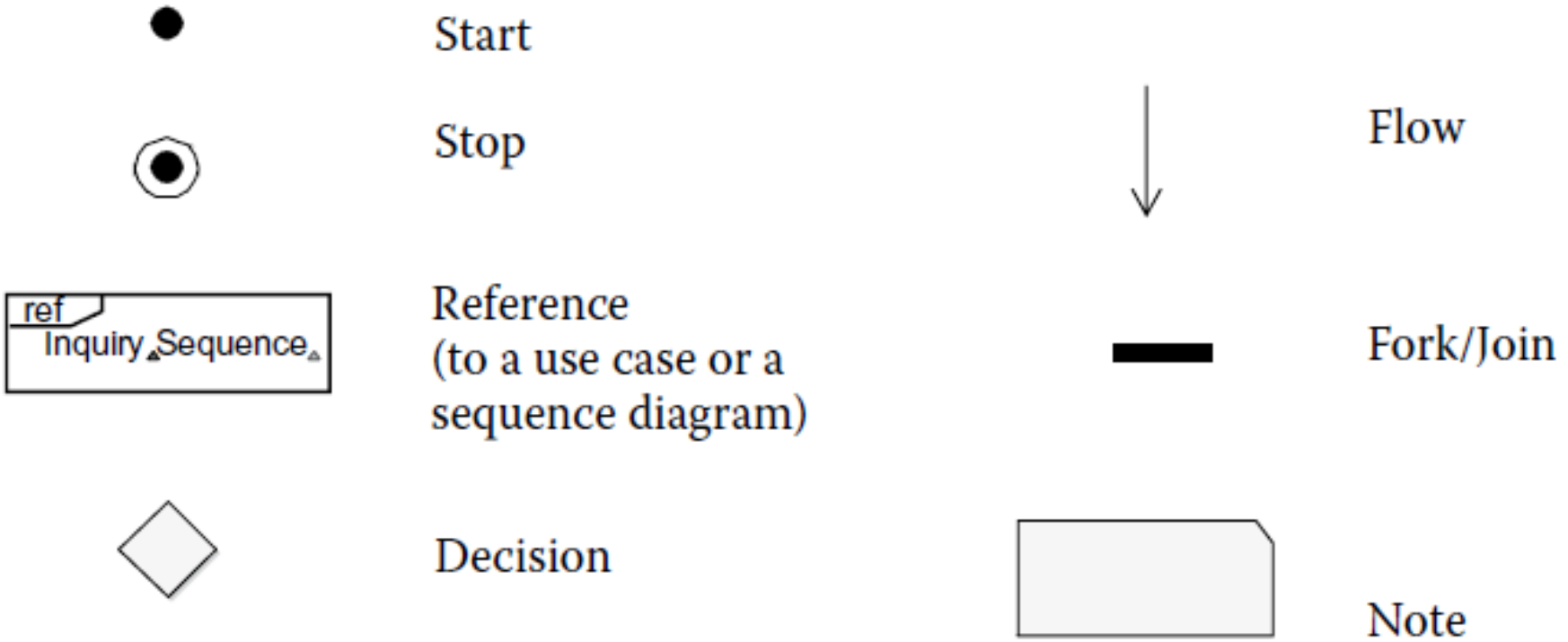
Potential for further confusion should Staff further inherit from Person.



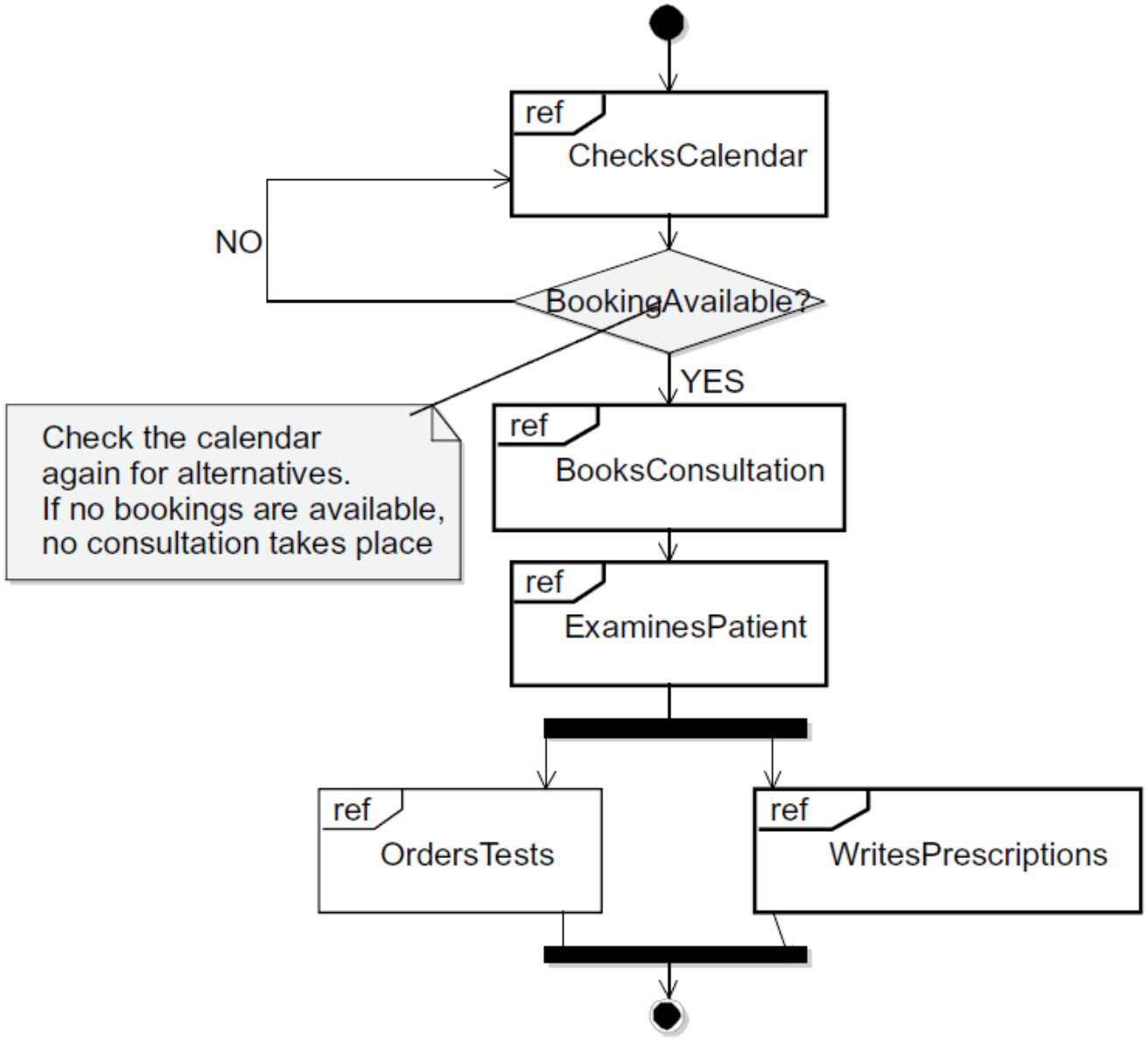
CODE EXAMPLE: FOR ERROR HANDLING

```
Class Patient {
Public:
Class RangeError { int badIndex; }
Schedule getSchedule {int index} const;
};
Schedule::getSchedule (int index) const {
        If (index < 0 || index > 10) //only 10 schedules allowed
            Throw RangeError (index); //error is thrown
        Return contents[index];
}
Void useSchedule() {
     Try {
     Patient aPatient = new Patient;
     Schedule s = aPatient.getSchedule(11);
      }
      Catch (const Schedule::ErrorRange&) {
          cout << "Index out of range. Wrong index is " index;
     }
}
```

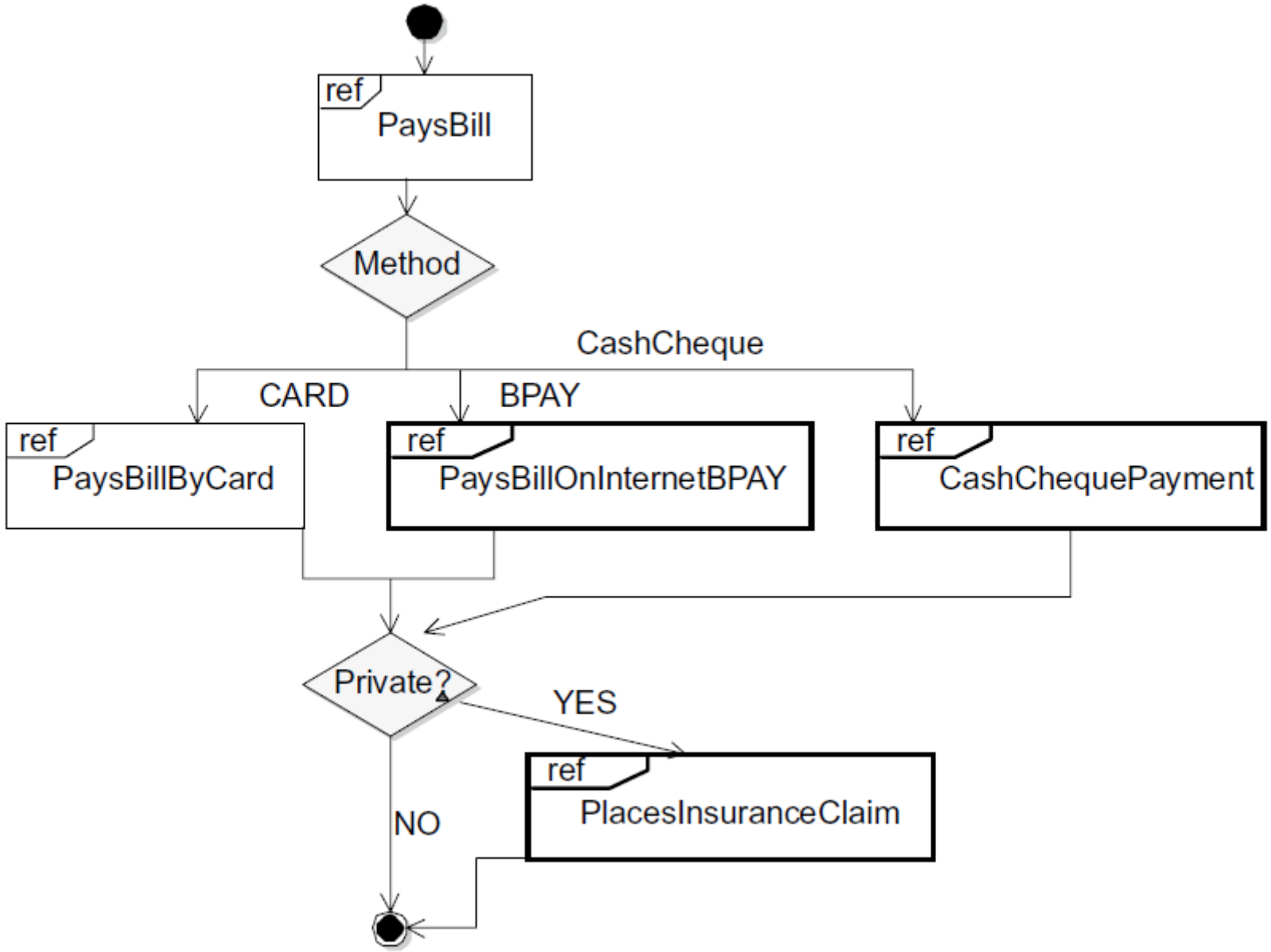
Notations of an interaction overview diagram



Consultation details interaction overview diagram



Accounting interaction overview diagram



Interaction Modeling With (System) Sequence Diagram

: Patient

The Actor

aPatient :Patient

Object

Timeline

Focus of control

1: enquiresAvailability

Message

Self message

Return message

Asynchrnous message

Destroying an object

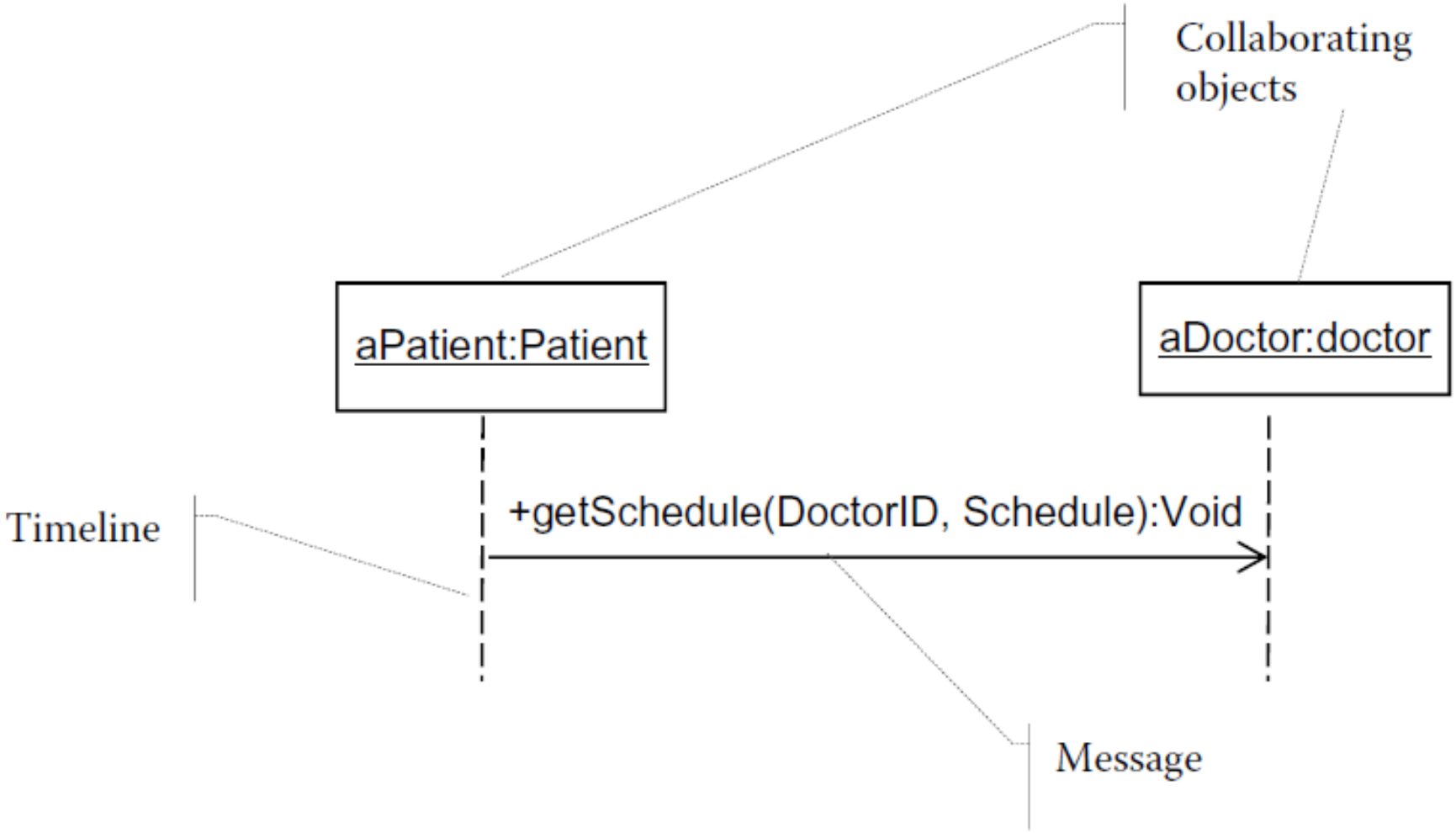
Patient Checks Availability of Doctor

Steps in the sequence

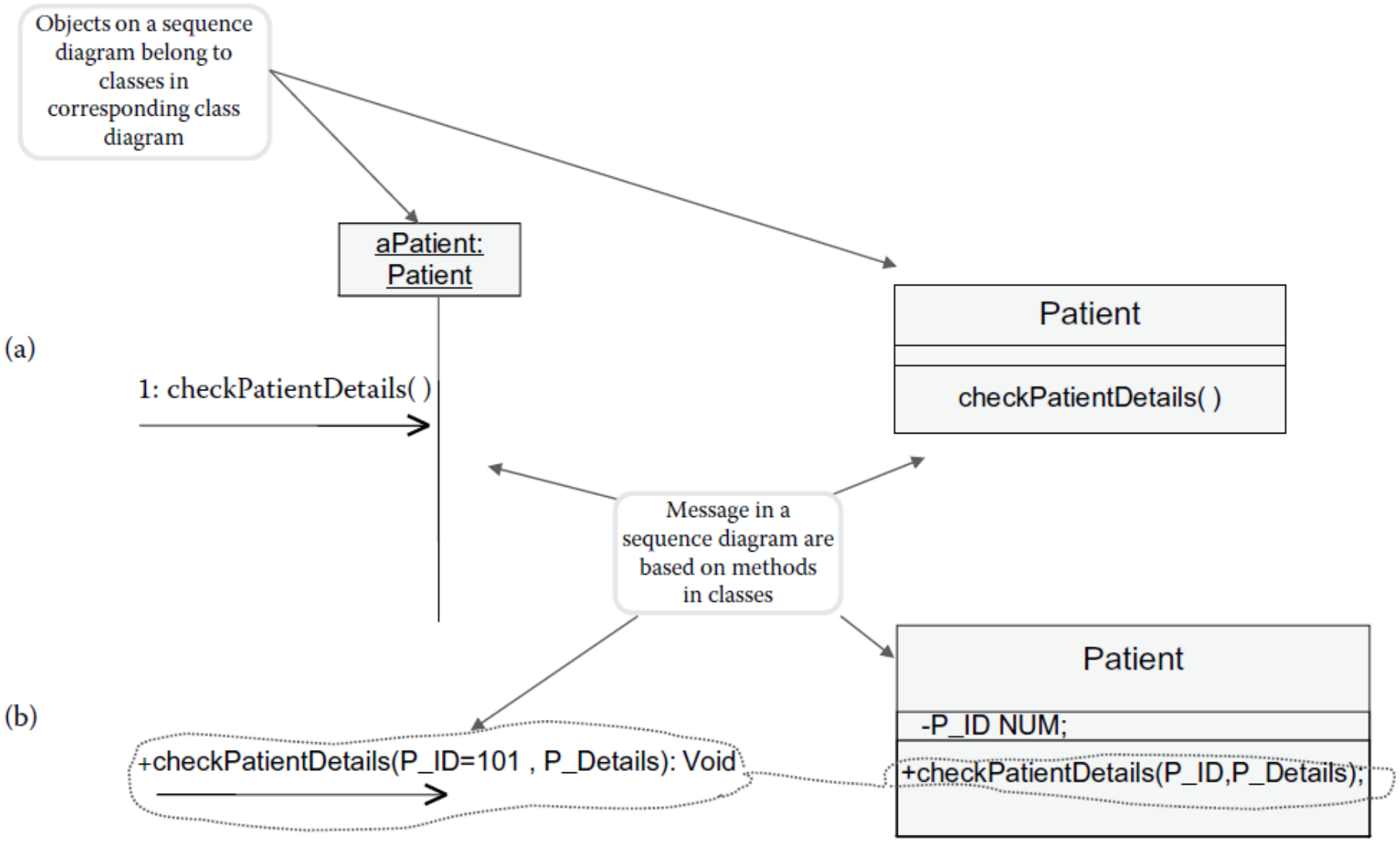
This is an explanatory note

Notes

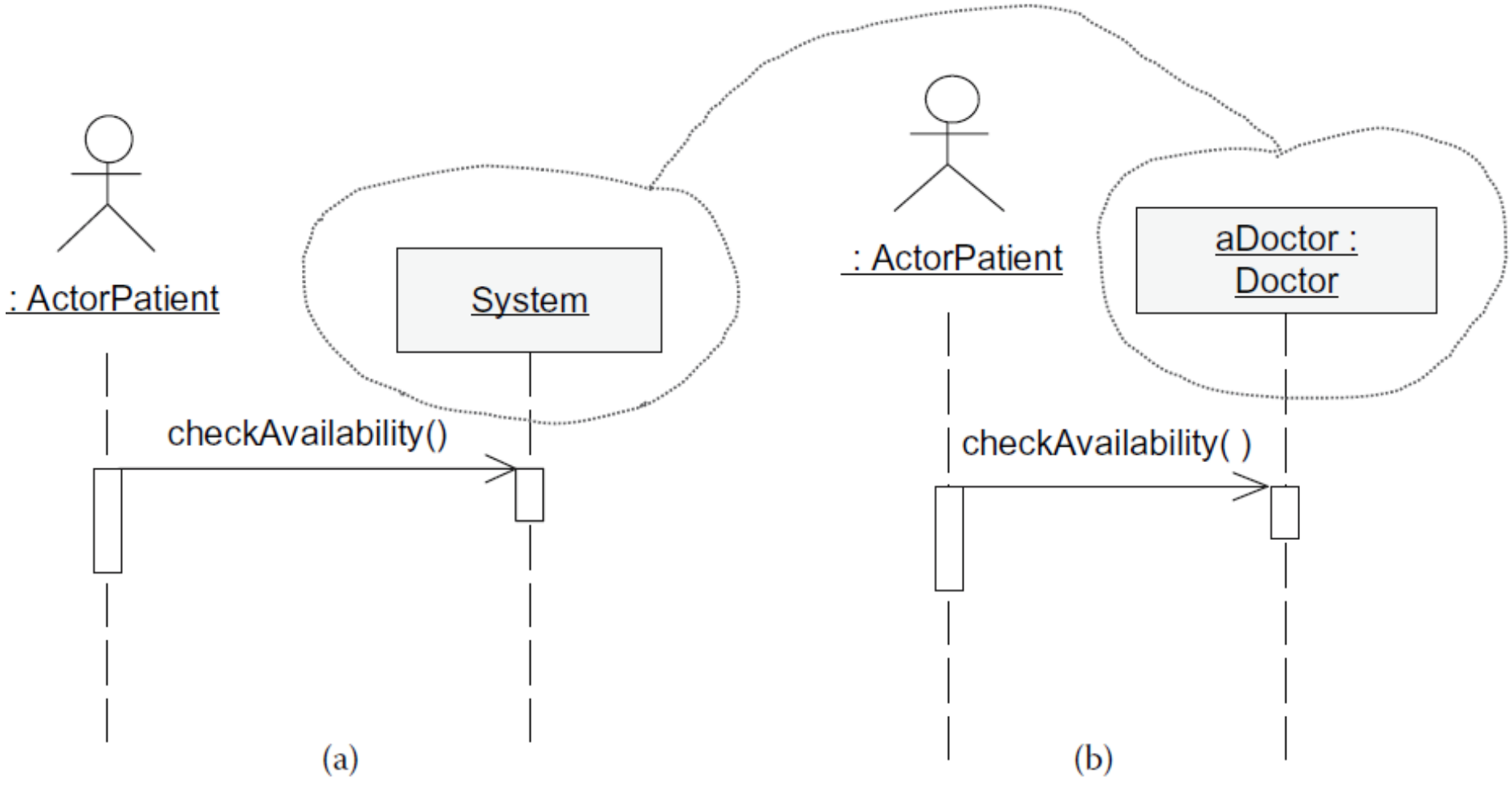
Basics of a sequence diagram



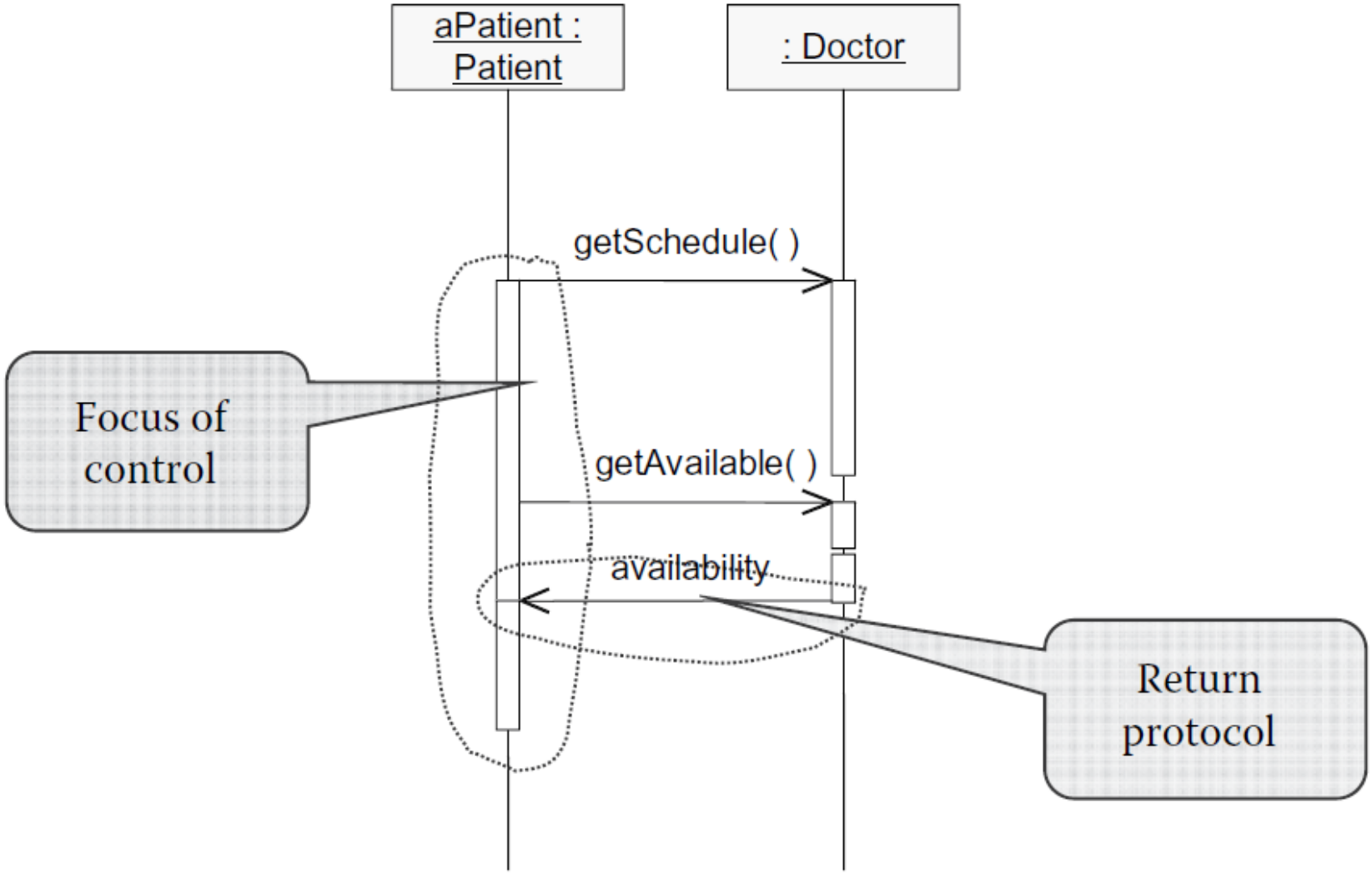
Relating sequence diagrams to class diagrams



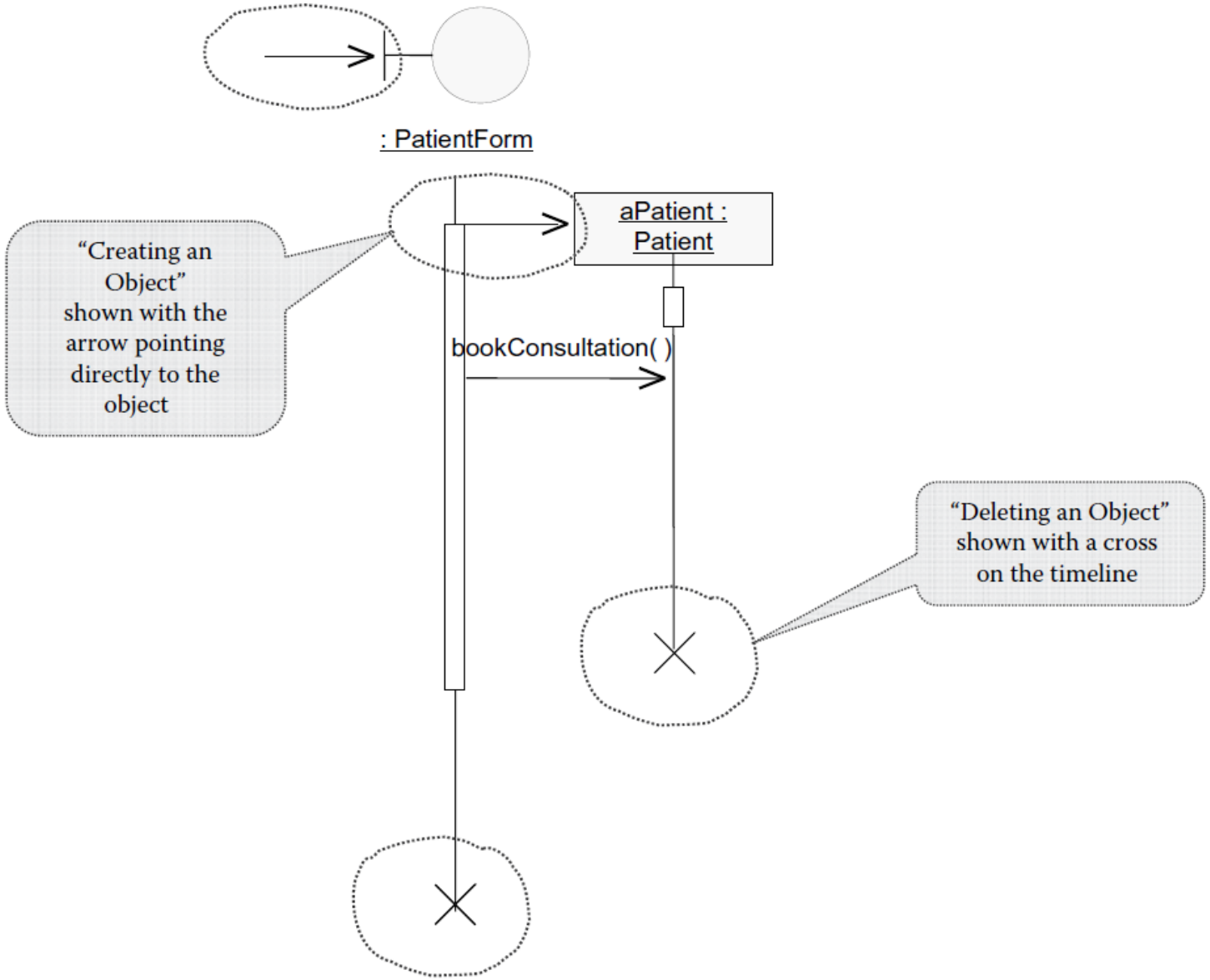
Advancing the sequence diagrams from analysis to design: (a) Analysis level Sequence Diagram shows the entire System as an object; (b) design level sequence diagram specifies an object within the System—in this case aDoctor:Doctor.



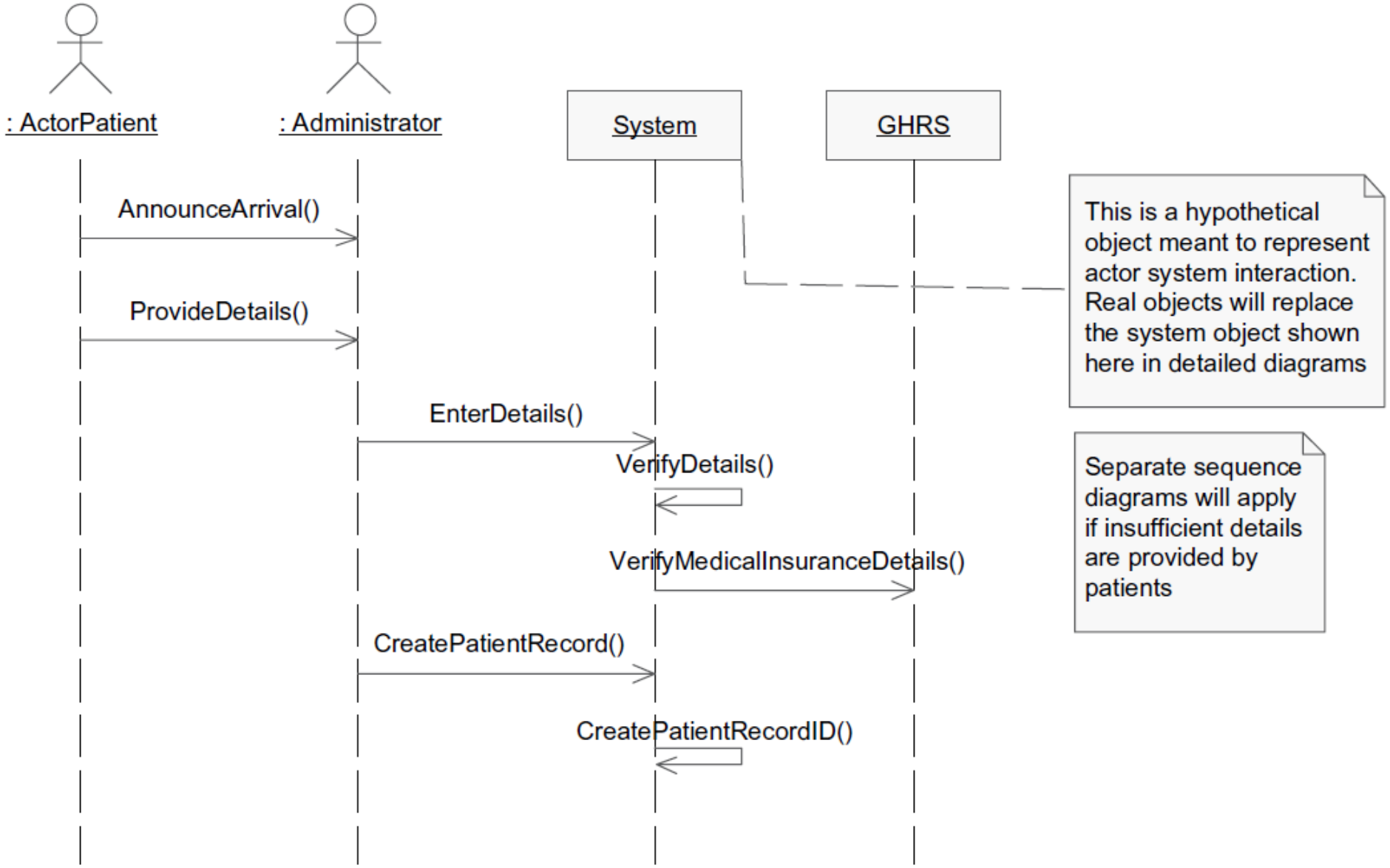
Understanding focus of control and return message



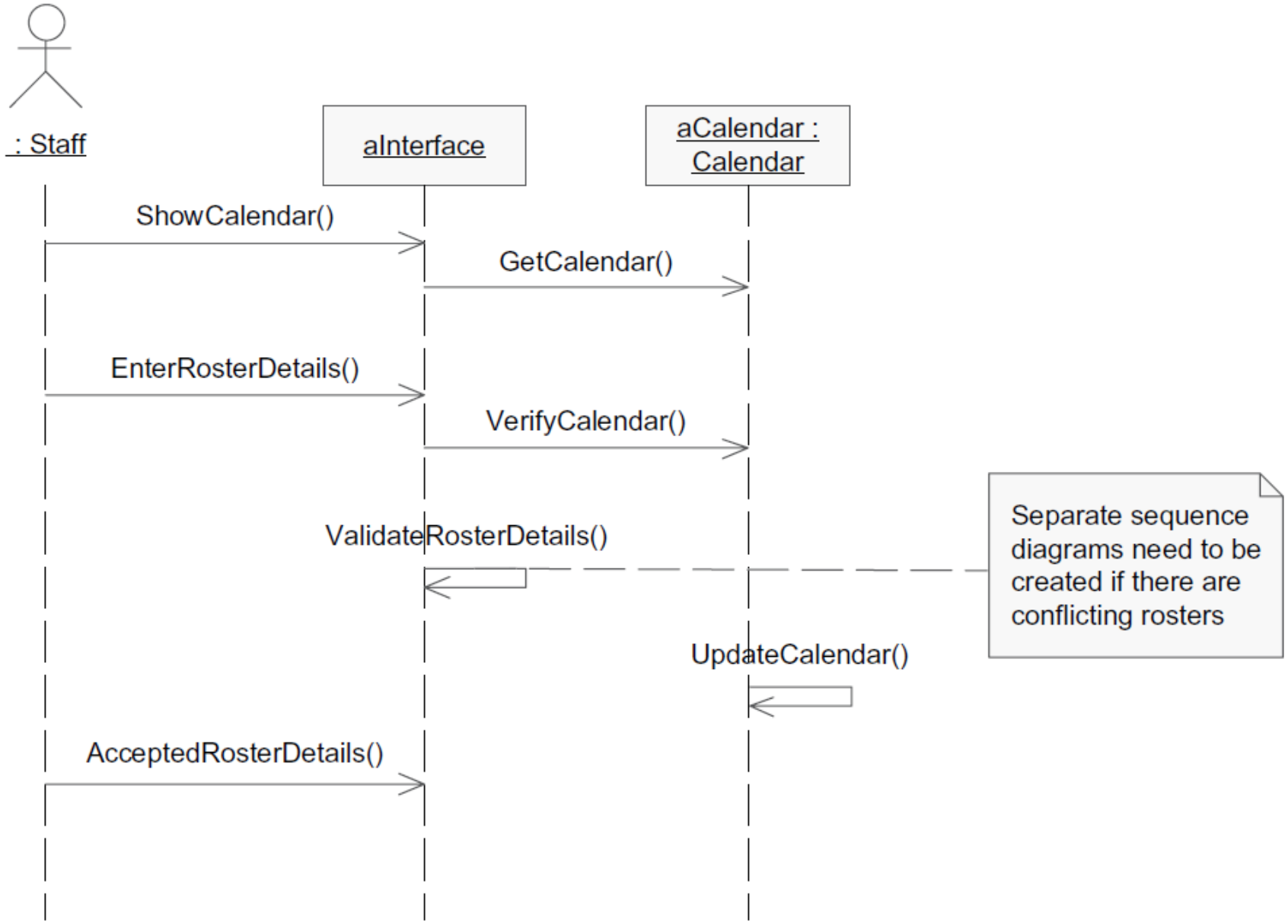
Creating and destroying an object



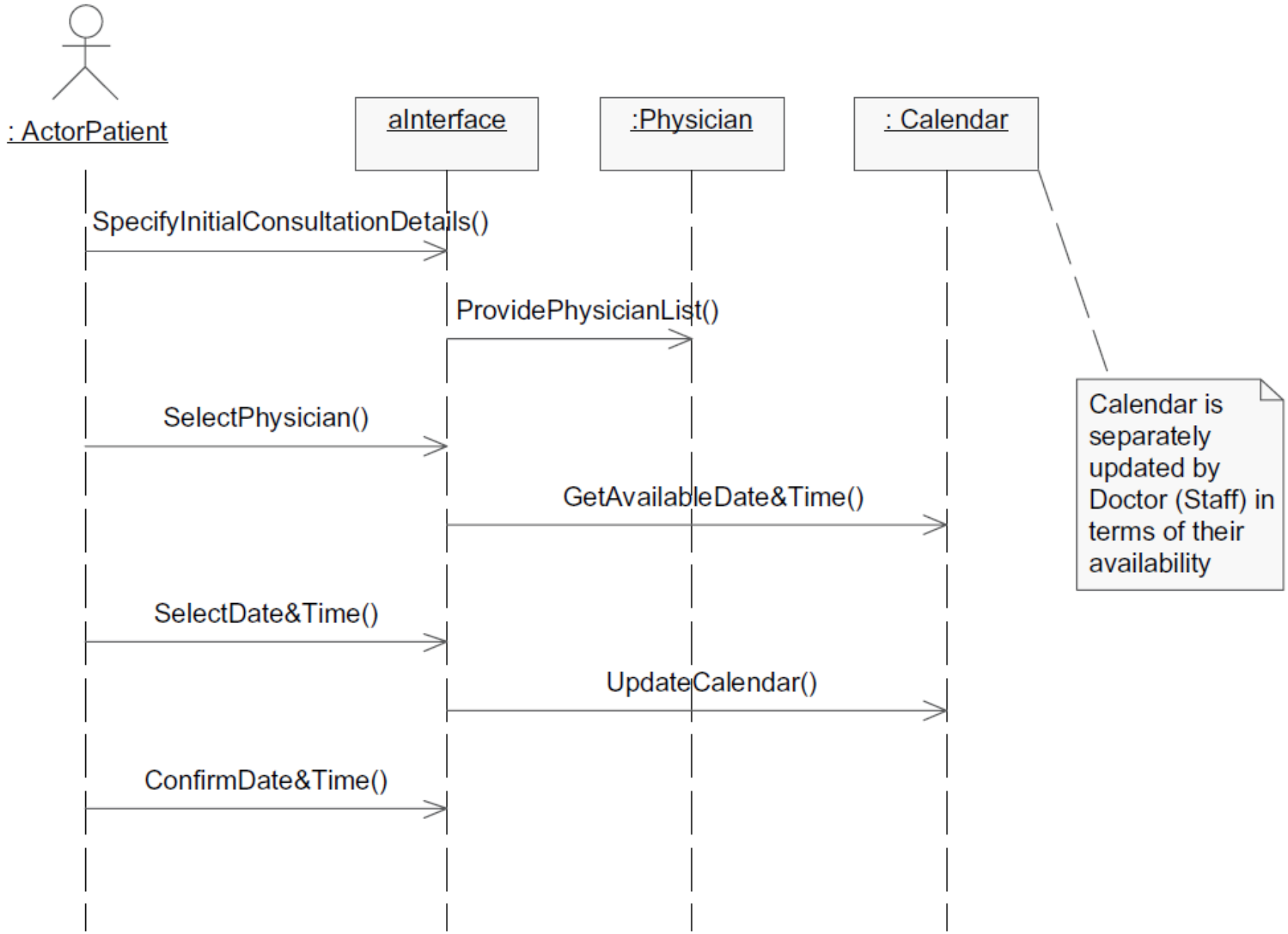
“Registering a Patient” sequence diagram



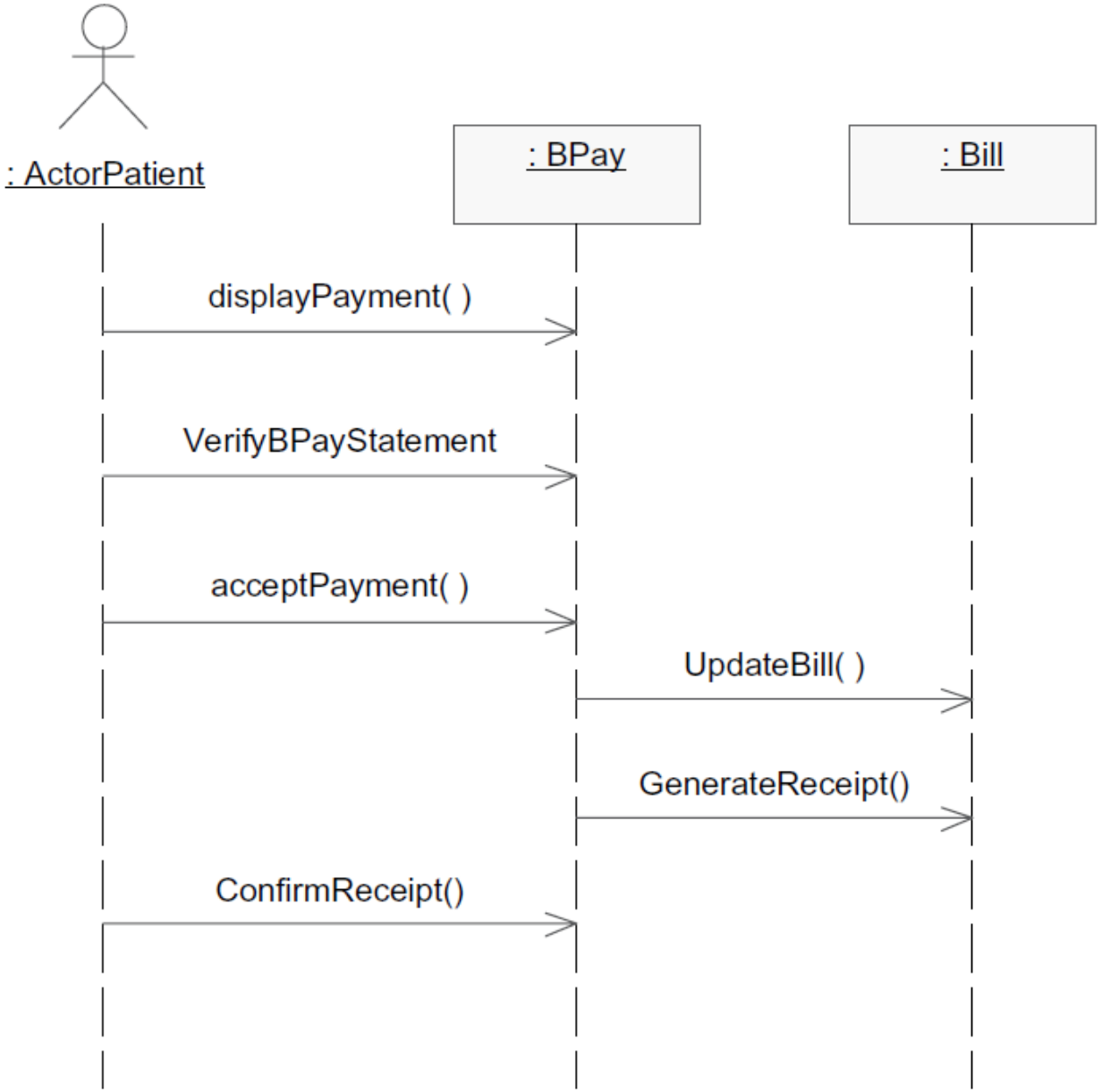
“Updating a Calendar” sequence diagram



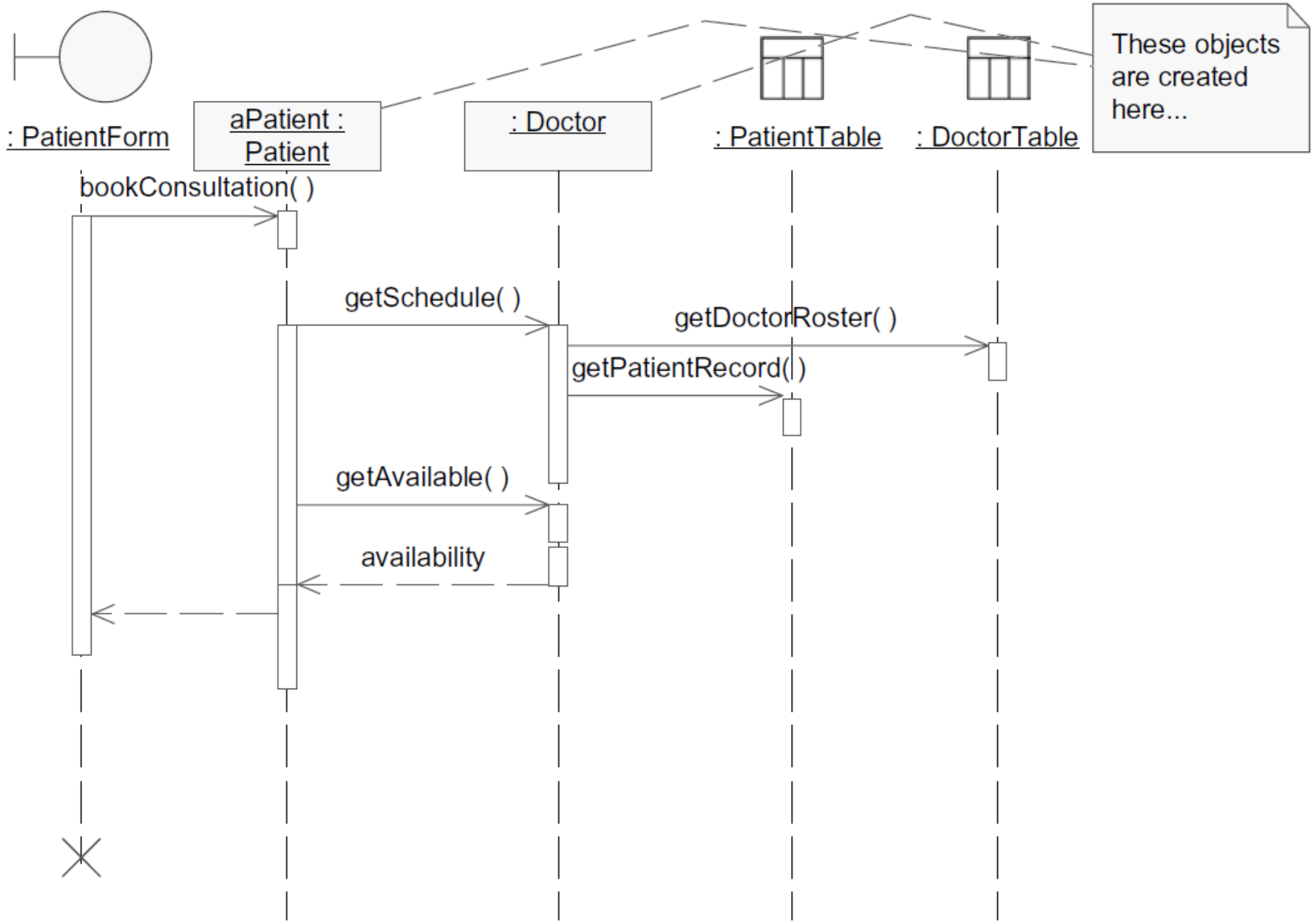
Sequence diagram for “Booking a Consultation.



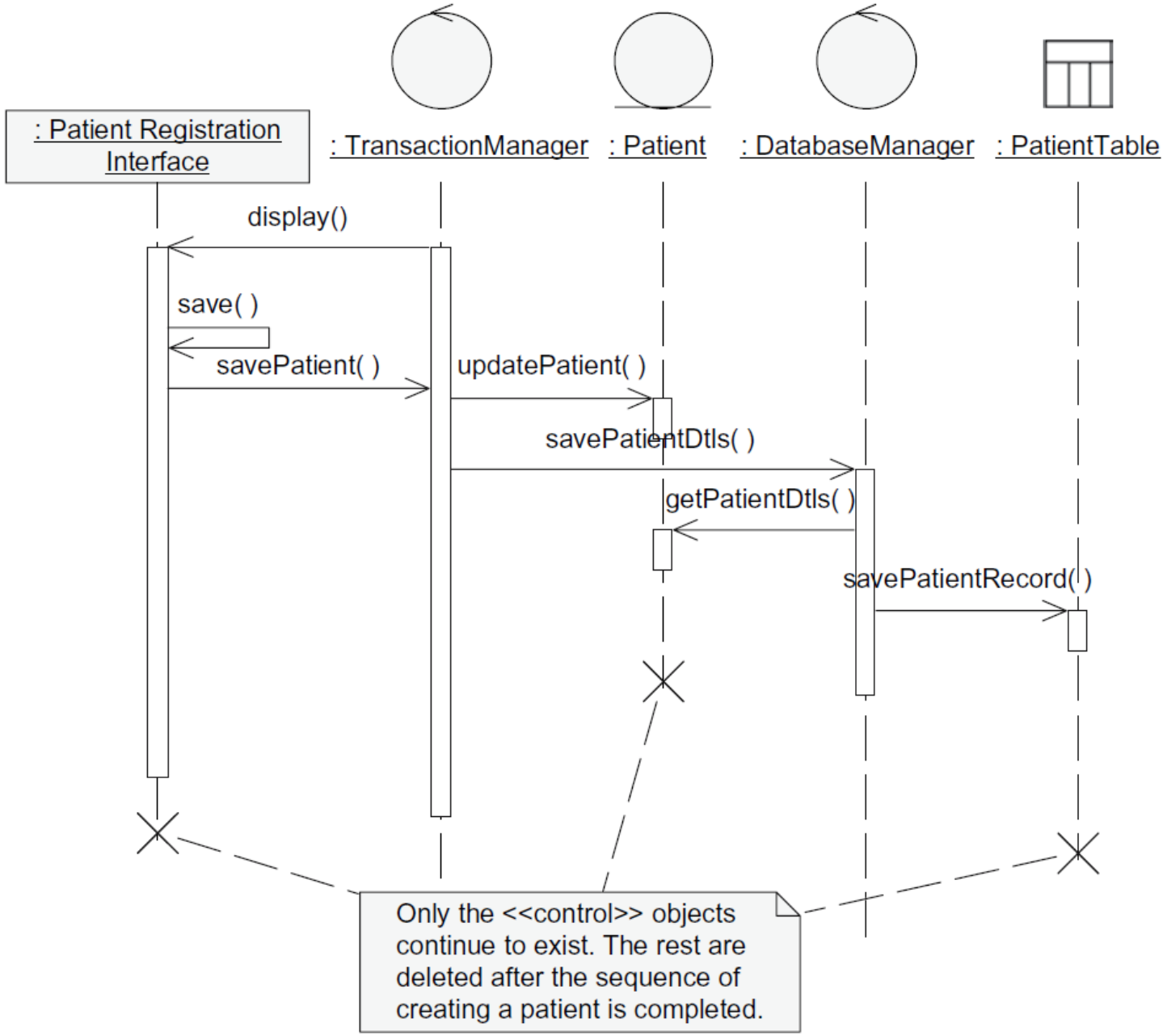
“Paying a Bill” sequence diagram



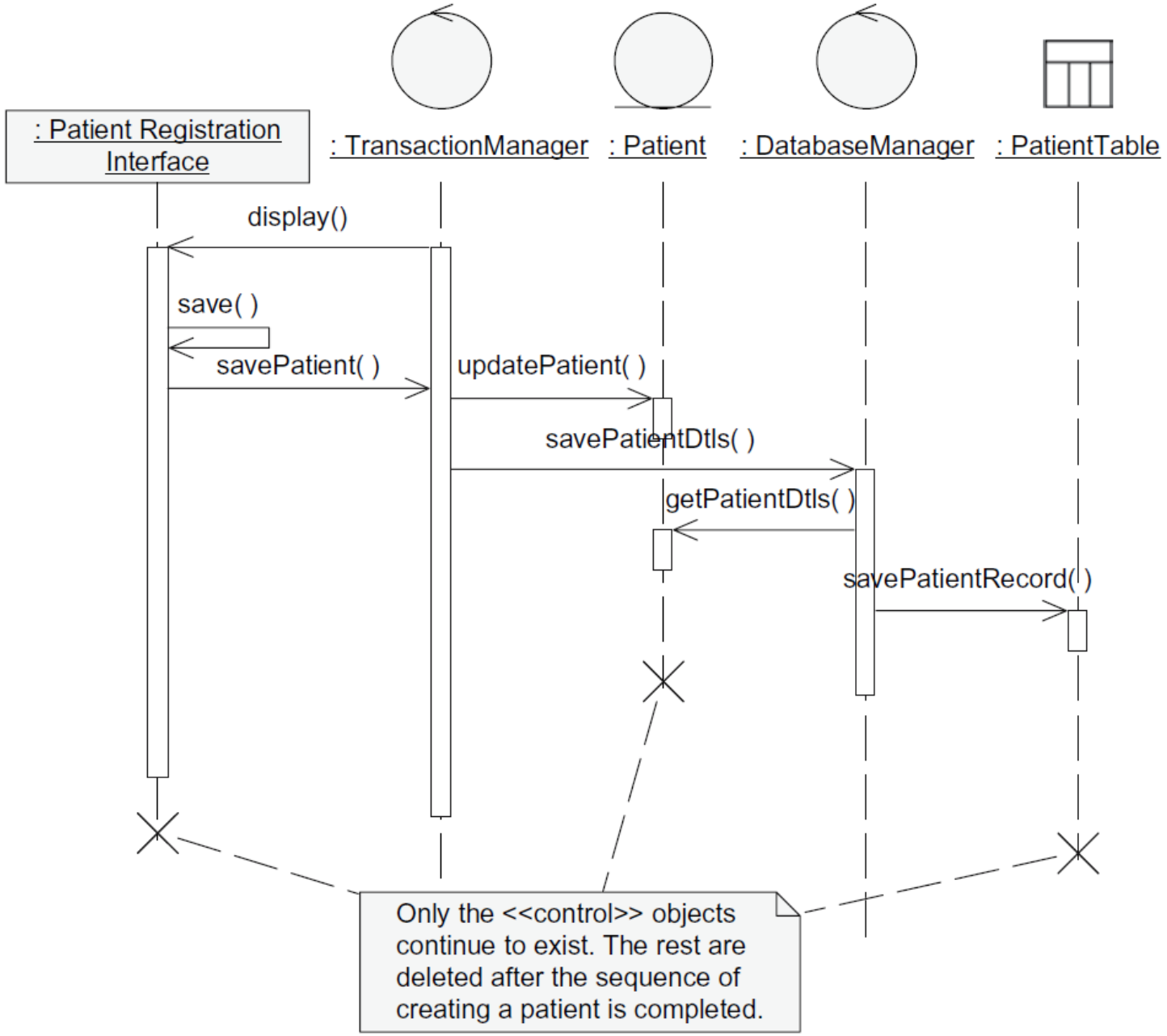
A design-level sequence diagram



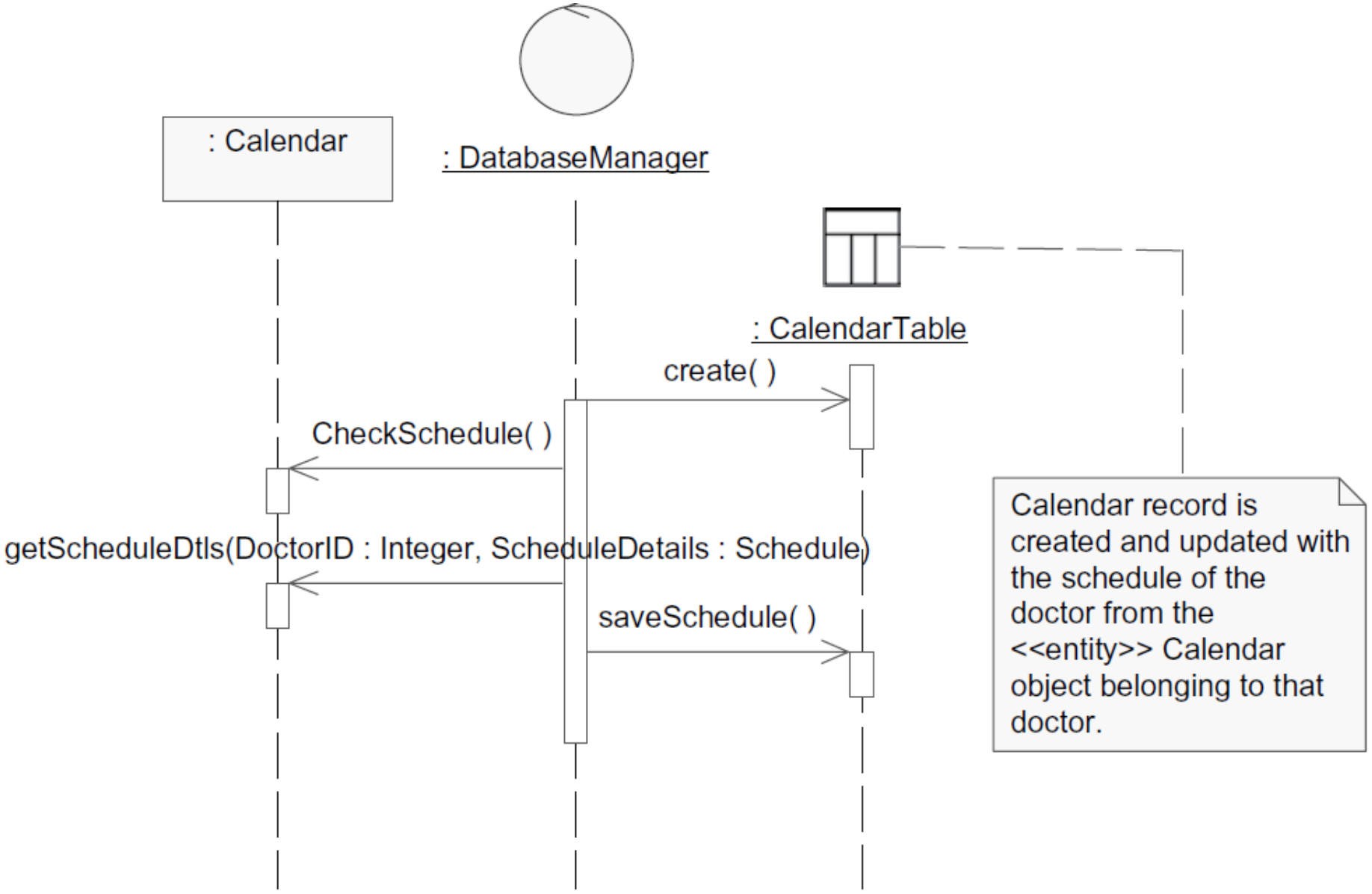
“Registering a Patient” sequence diagram in design



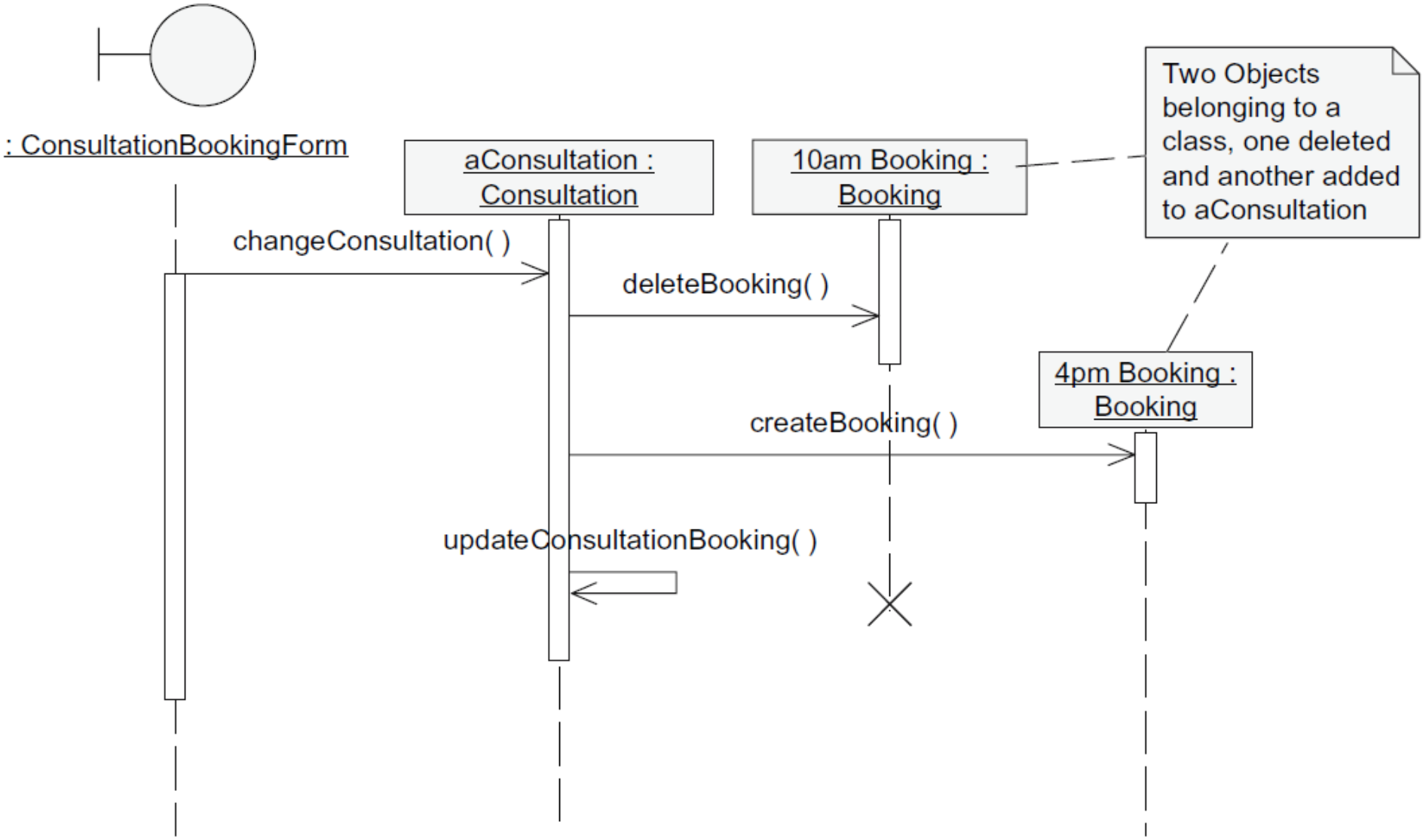
“Registering a Patient” sequence diagram in design



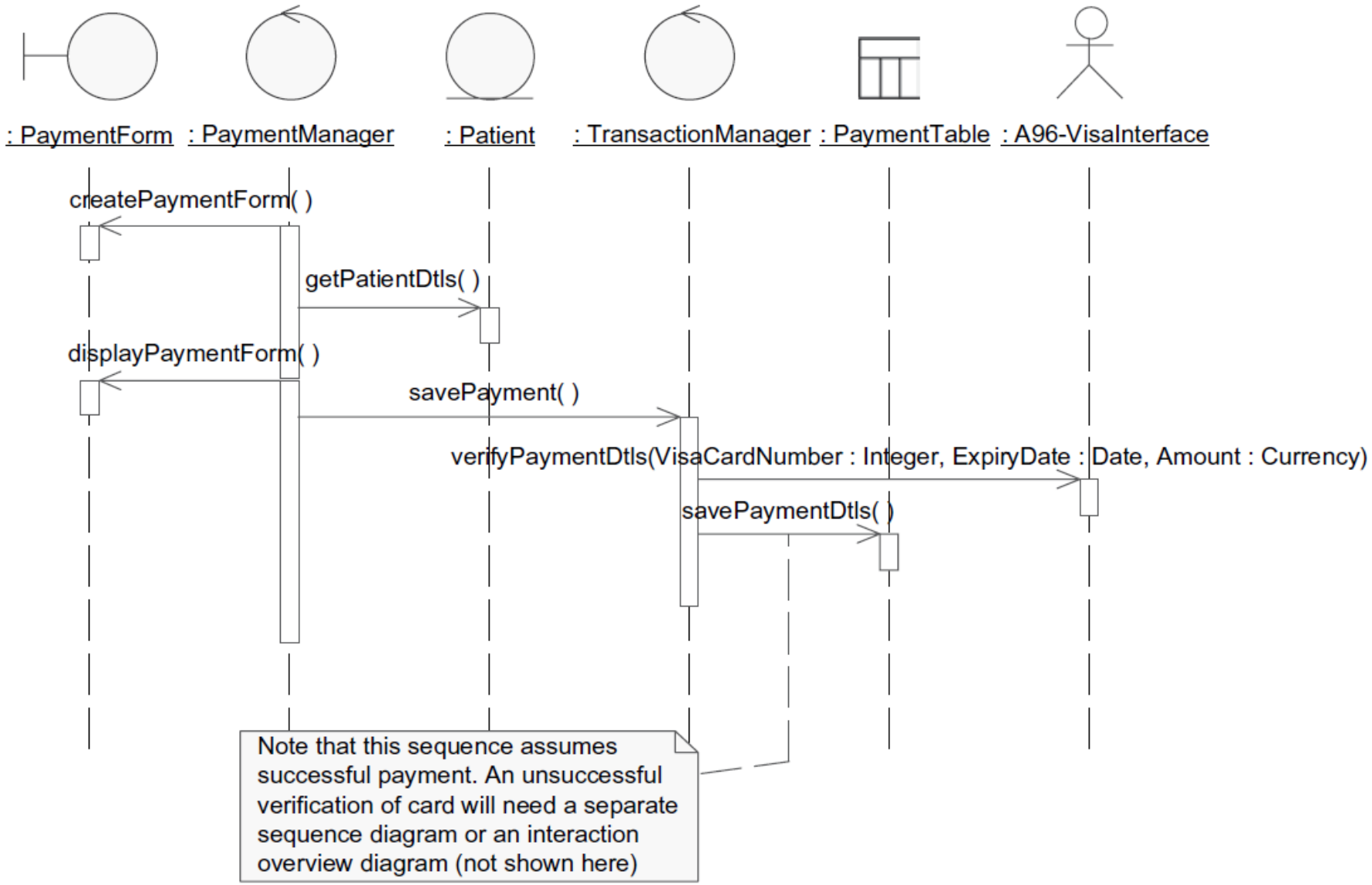
“Updating a Calendar” sequence diagram in design



“Changing Booking Times for a Consultation” sequence diagram in design



“Paying a Bill” sequence diagram

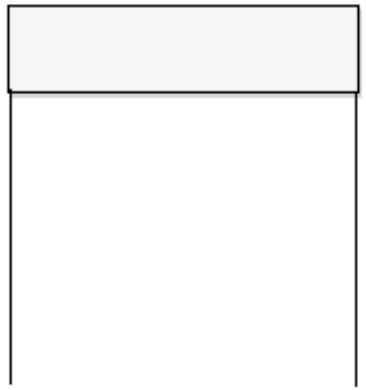


Notations of Activity Diagrams

Start activity

Stop activity

Flow of activities



Partition

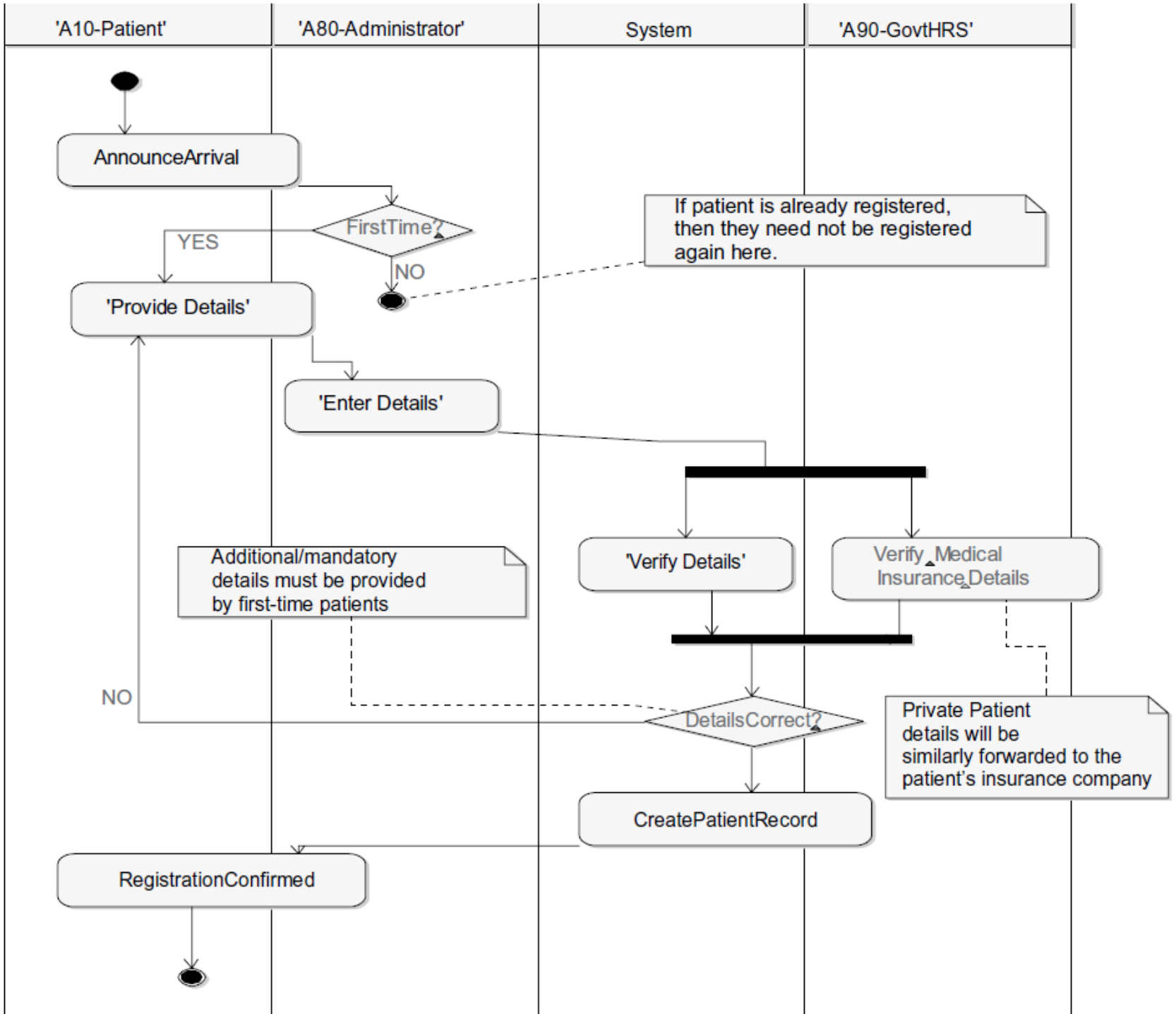
Activity

Fork/Join

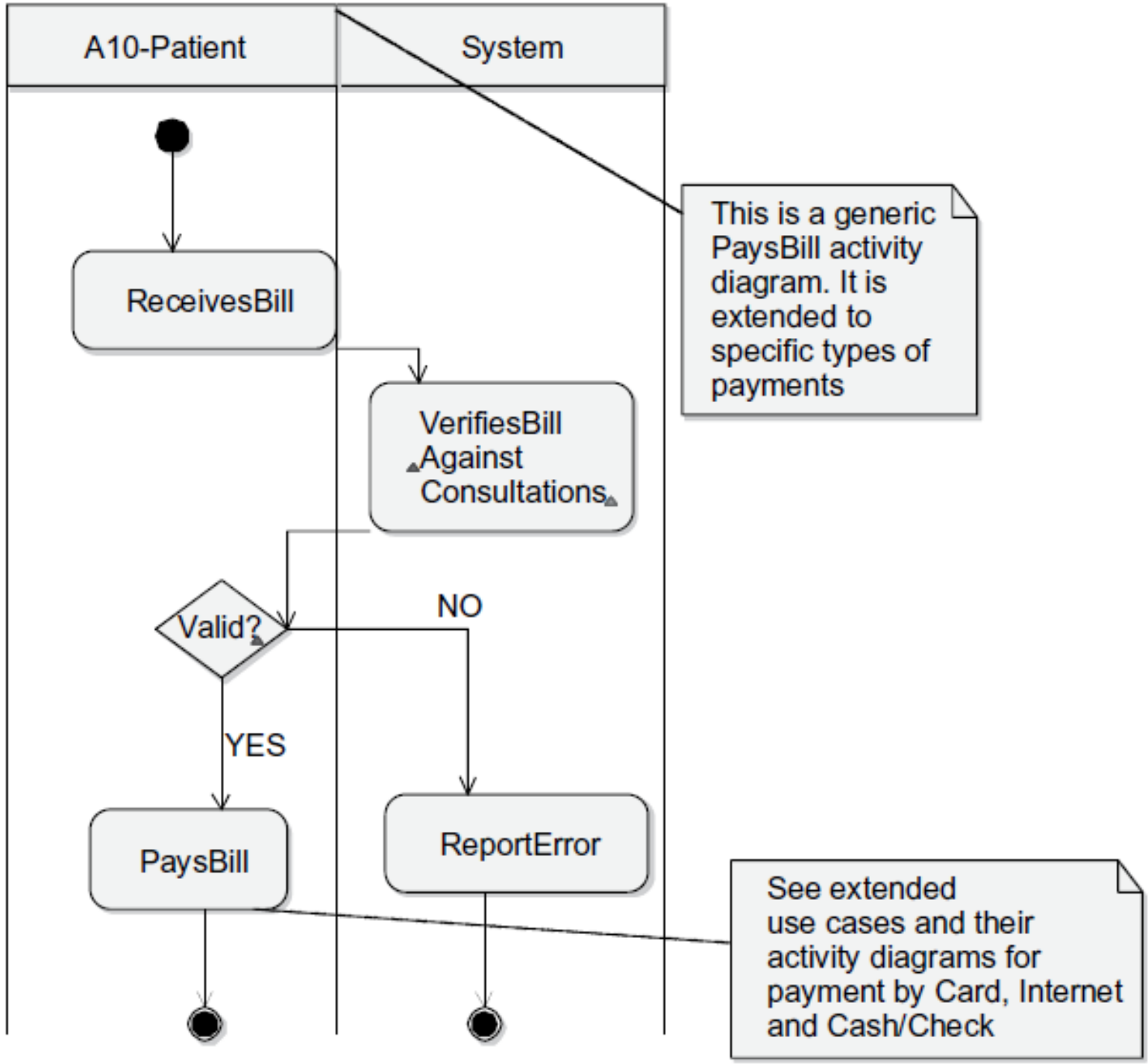
Decision point

Notes

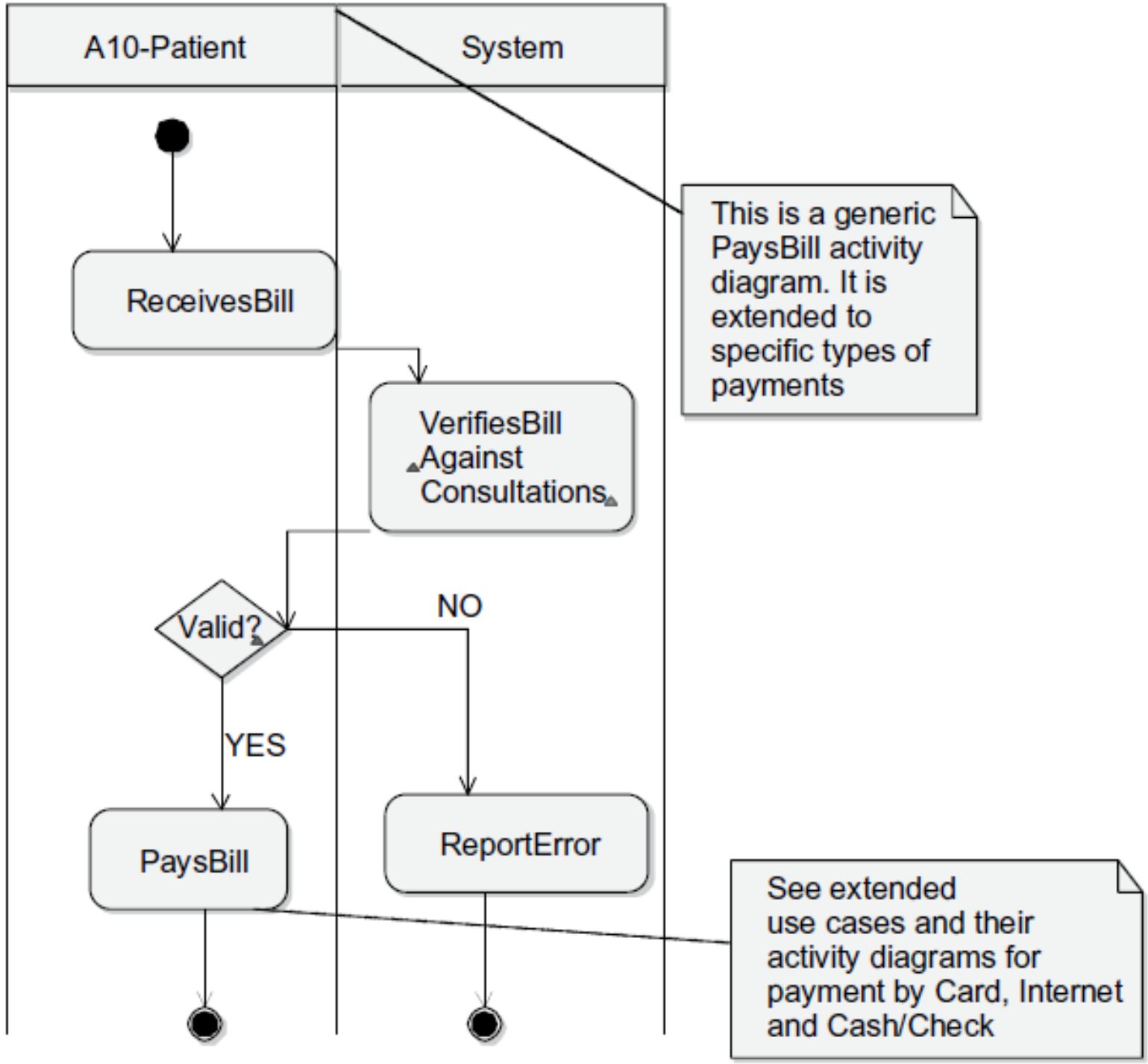
“RegistersPatient ” Activity Diagram



“PaysBill” activity diagram



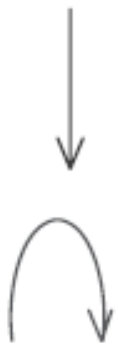
“PaysBill” activity diagram



Notations in a state machine diagram

Start State

Stop State



Transition
and
Self-Transition

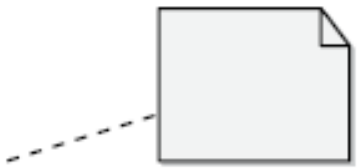
[CorrectDetails]
/ acceptPolicy

Guard Condition/
Action

State

Nesting

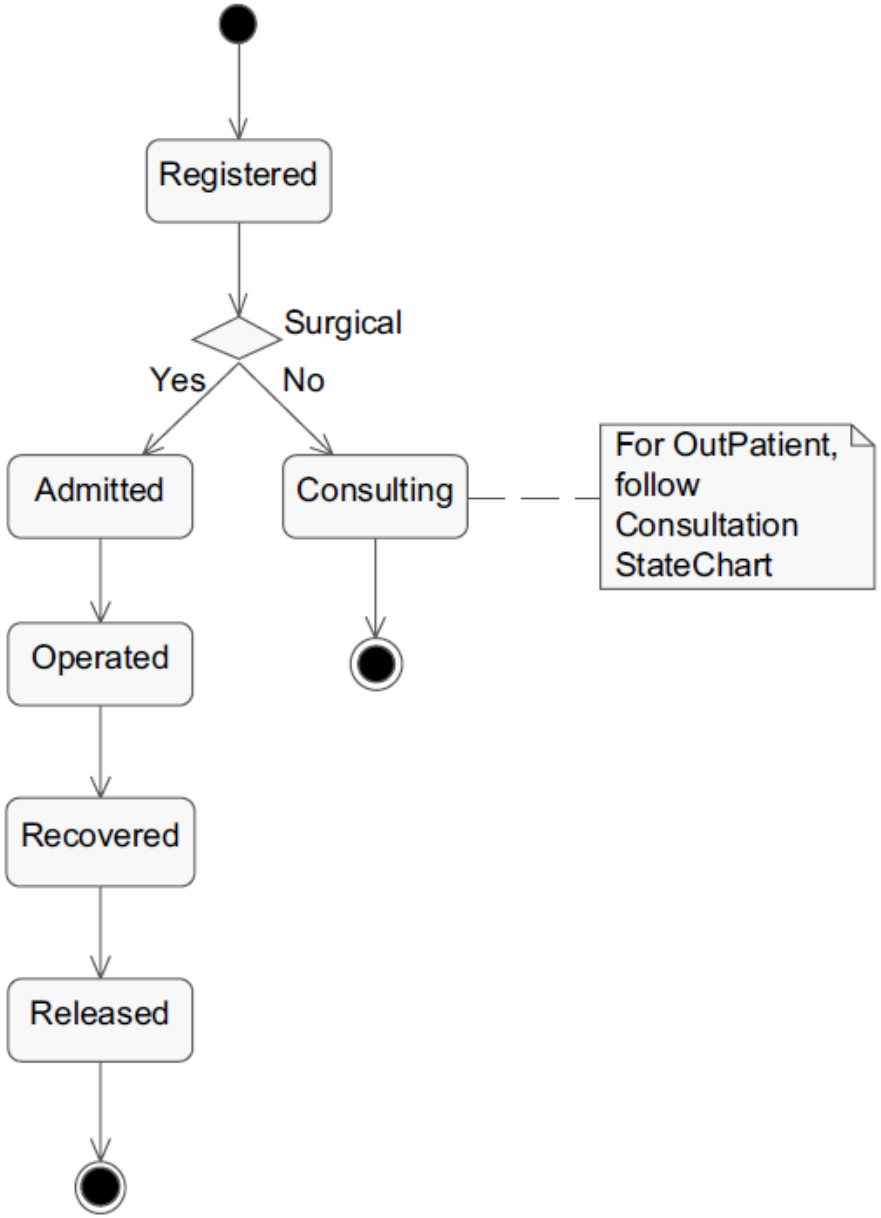
Decision
Point



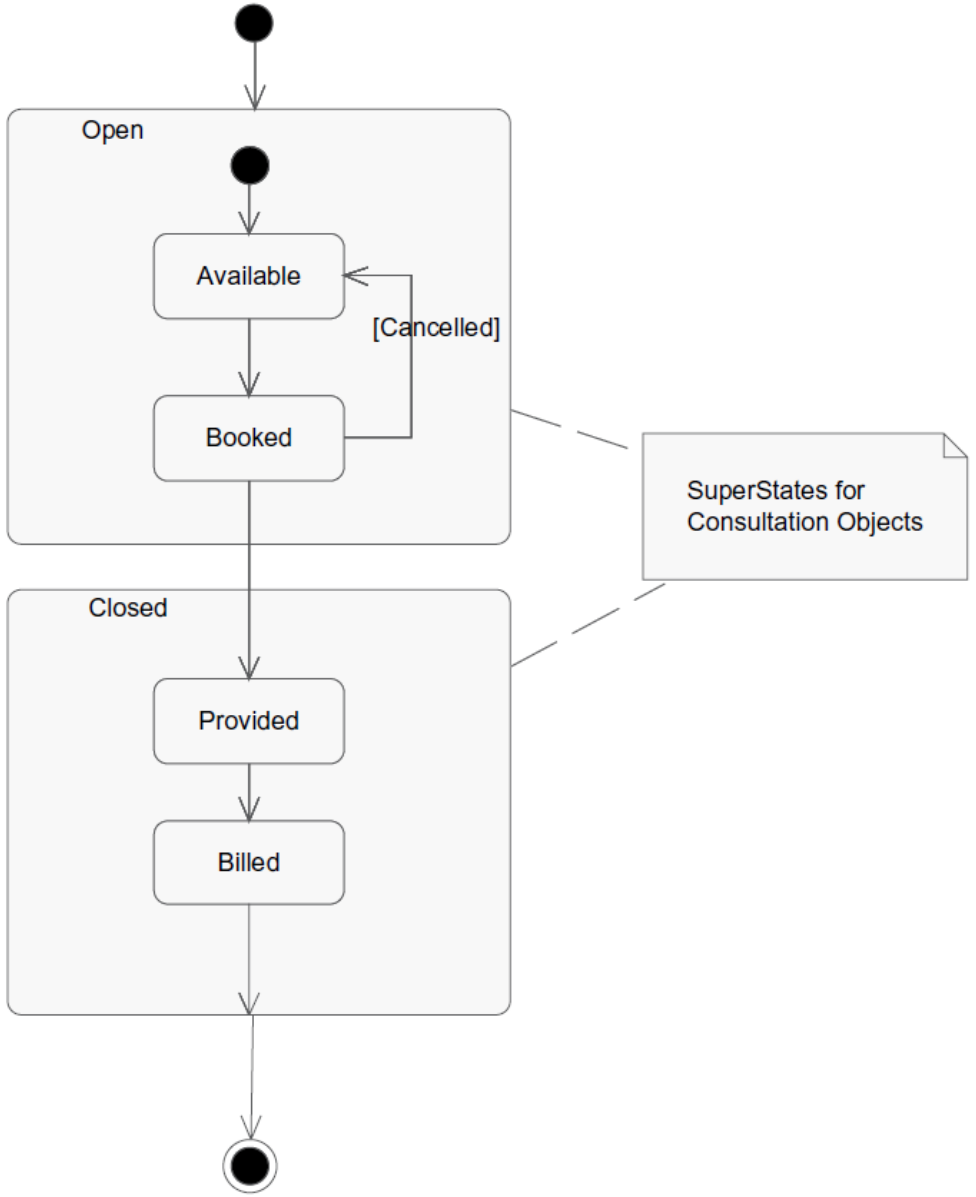
Notes

Patient state machine diagram

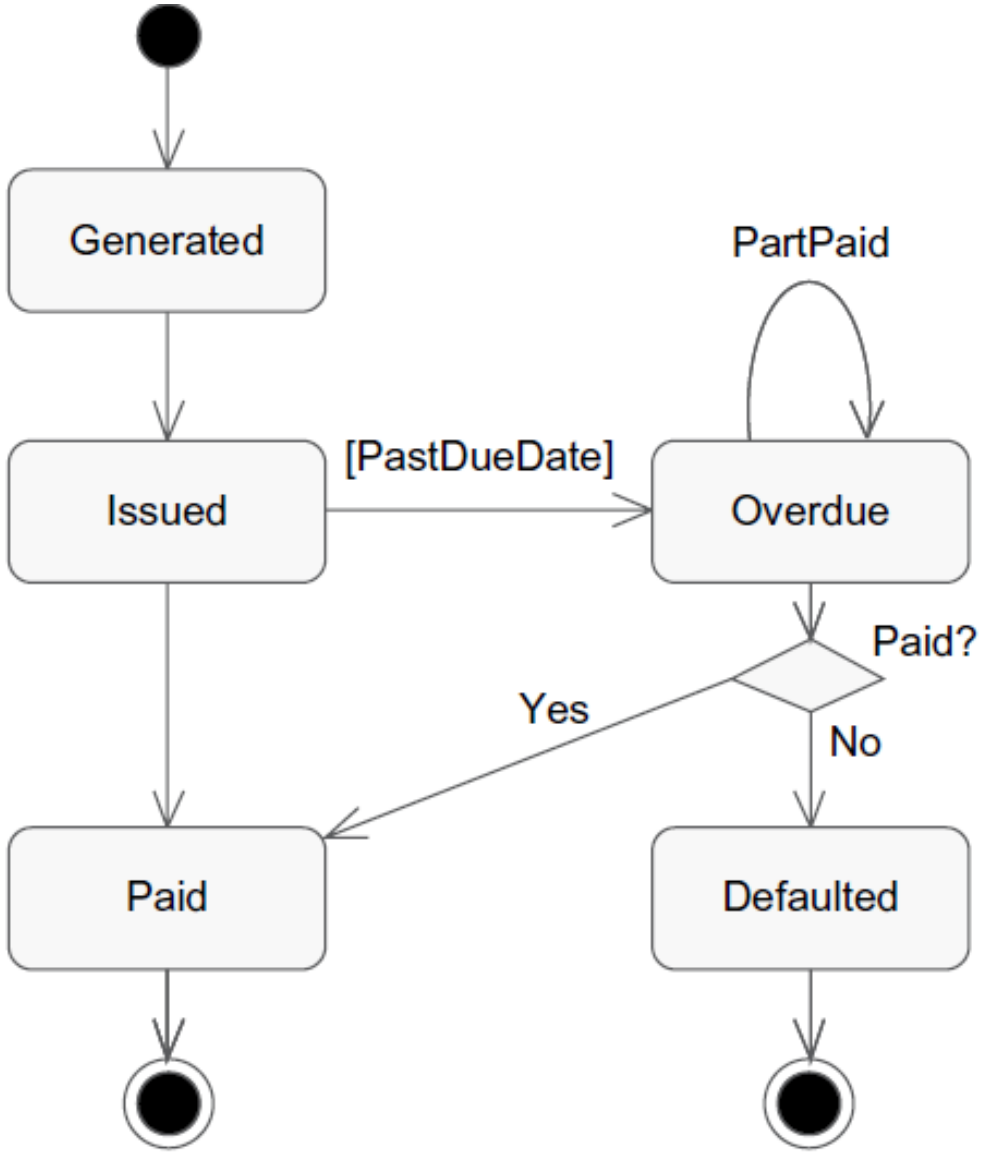
State Identifier	State Name
0	Registered
1	Admitted
2	Operated
3	Recovered
7	Consulting
9	Released



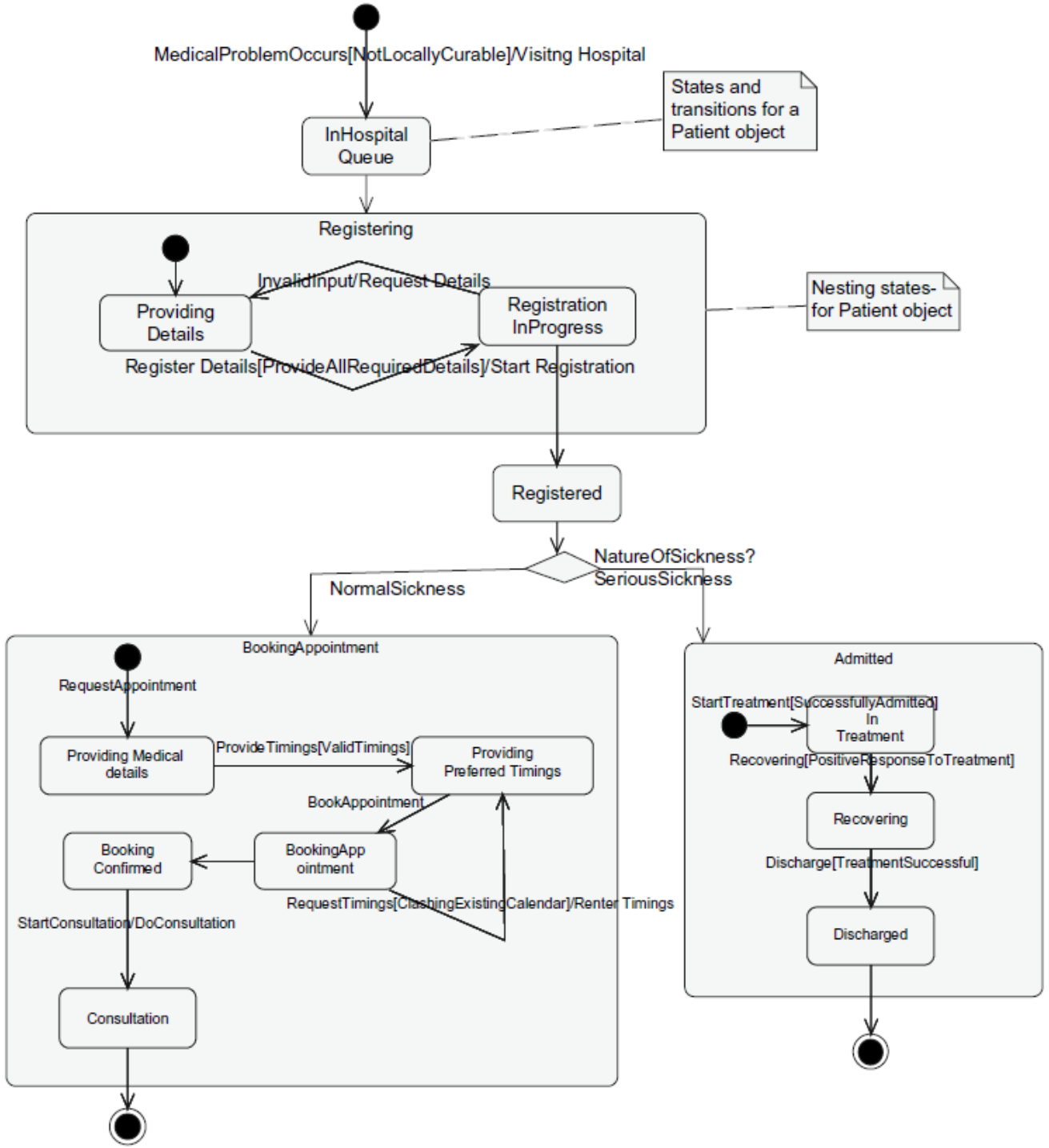
“Consultation” state machine diagram.



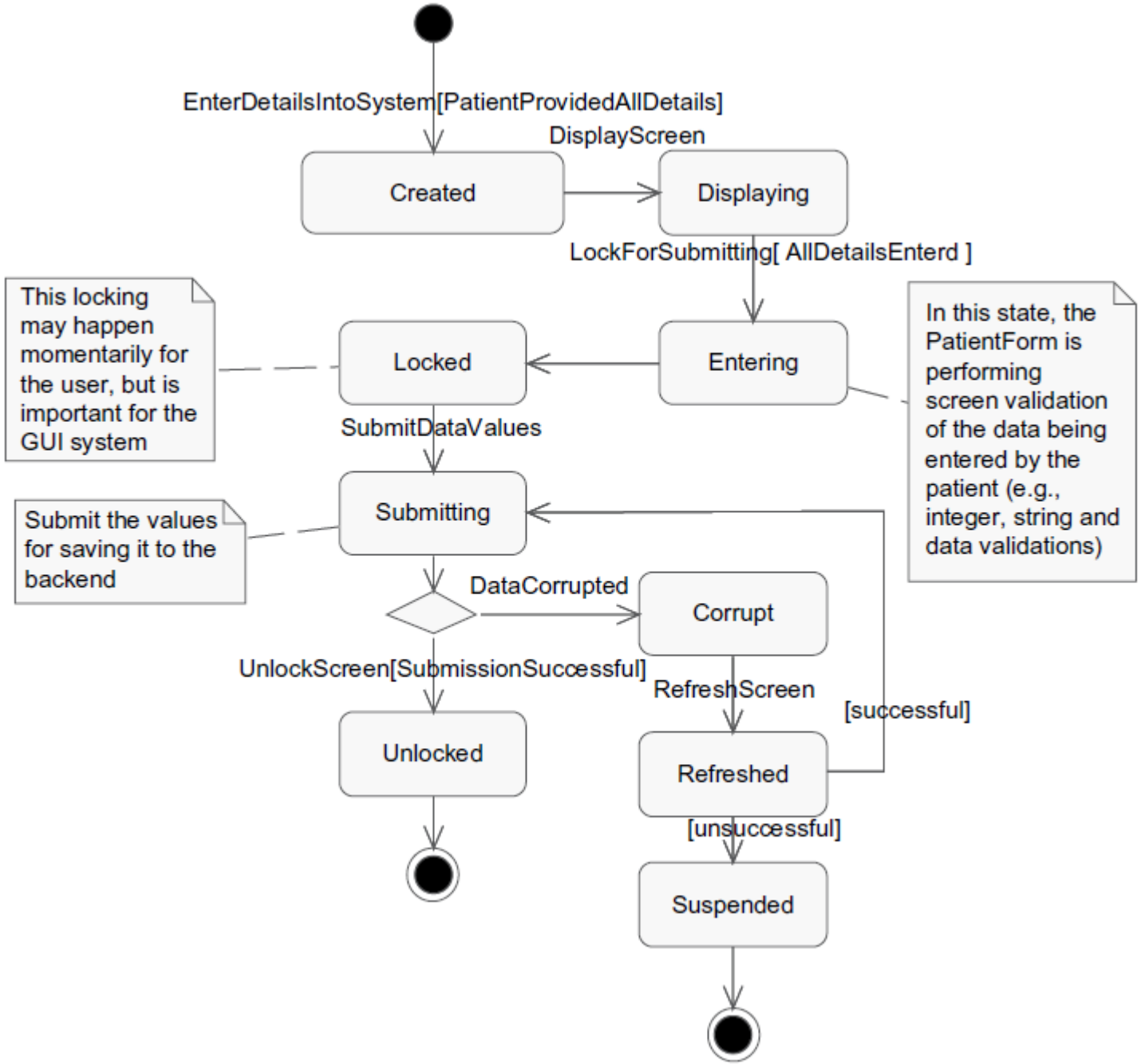
Bill payment state machine diagram



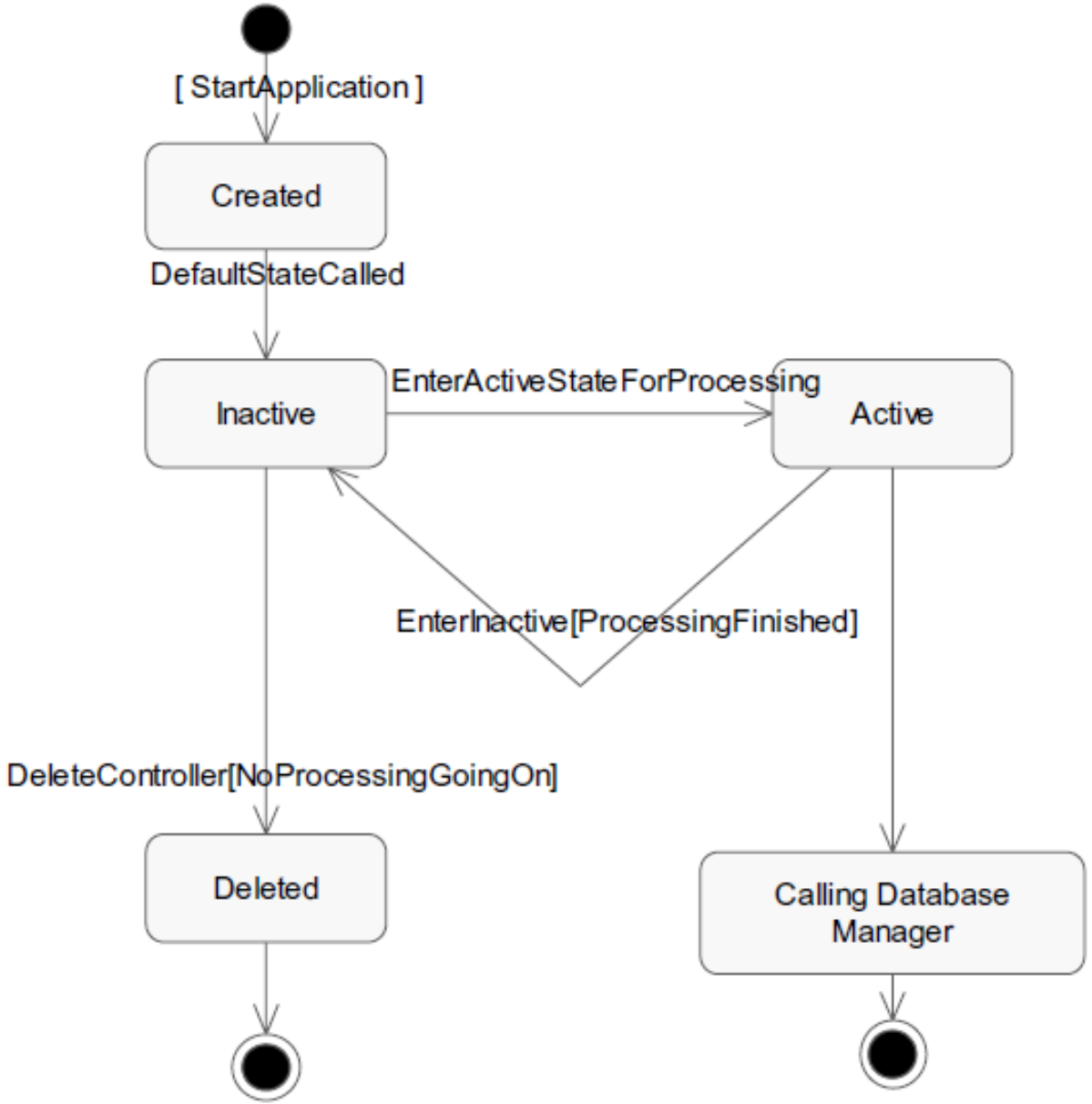
Advanced state machine for “Patient” in solution space.



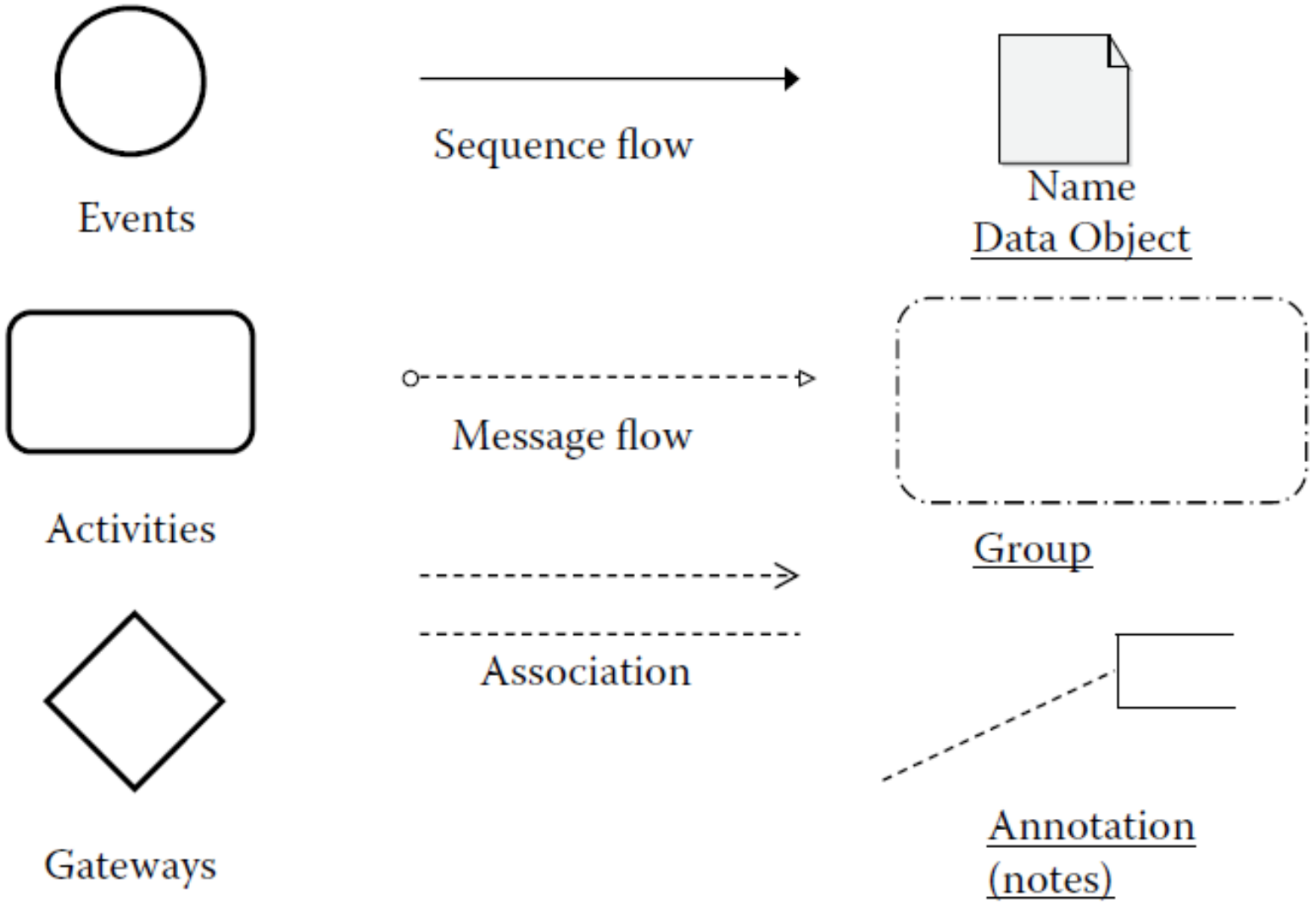
State machine diagram for “Patient_Form” <<boundary>> object



State machine diagram for “Consultation Manager” <<Control>> object



BPMN notation

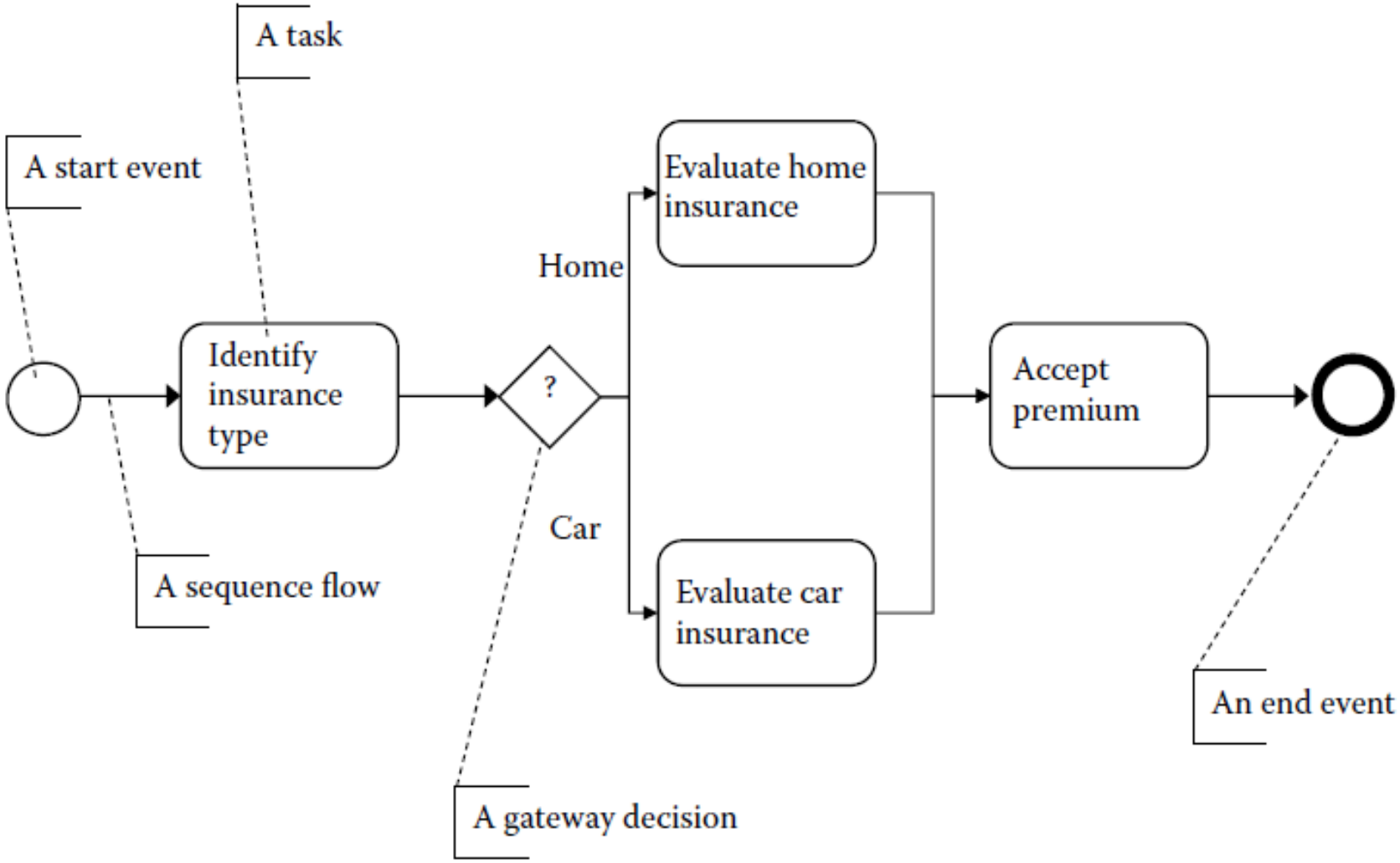


BPMN – grouping with pods and lanes

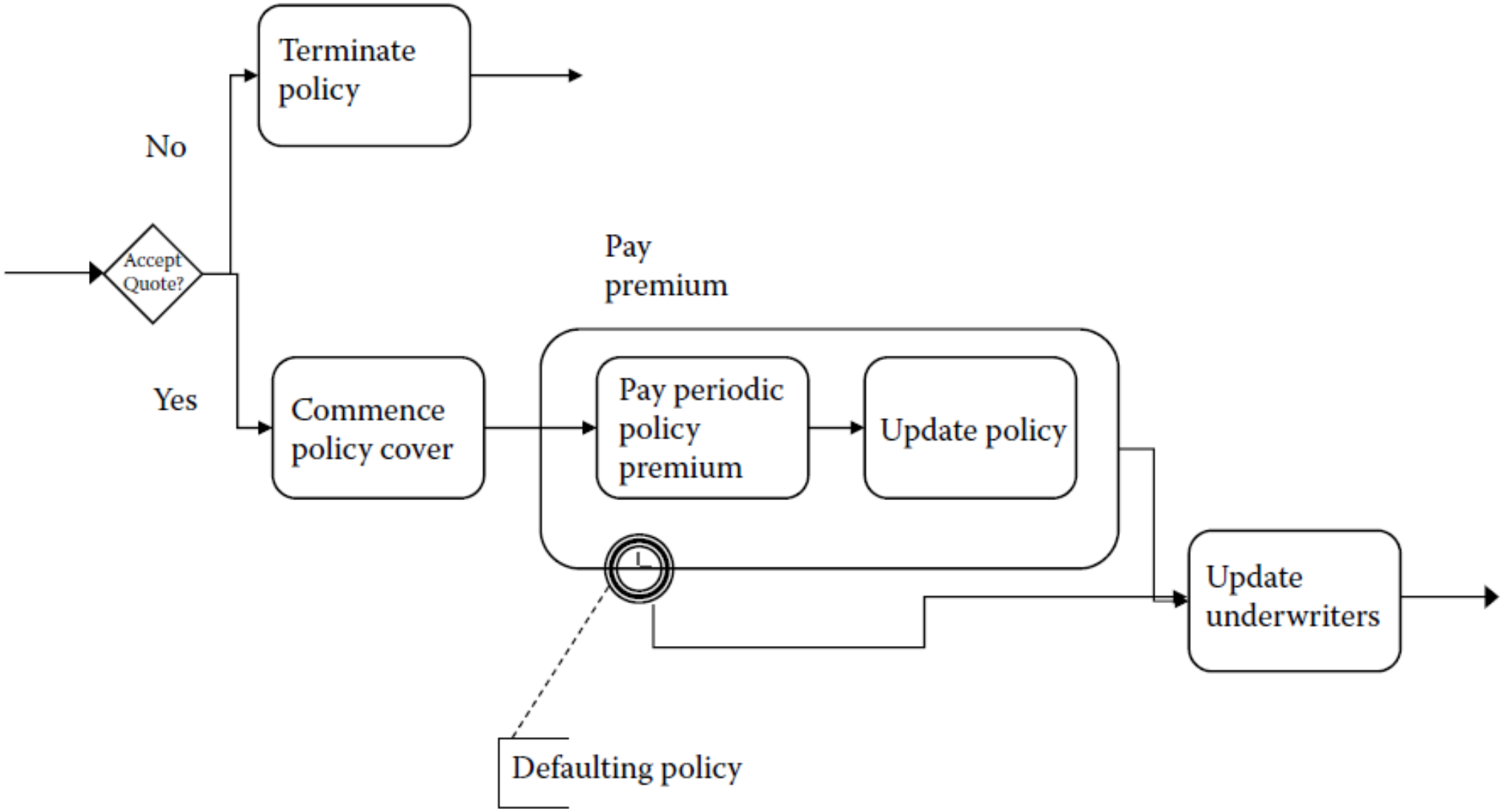
Pools

Lanes
(within Pools)

A basic process model (Insurance example)



Another basic process model (with additional details).



Package Diagrams

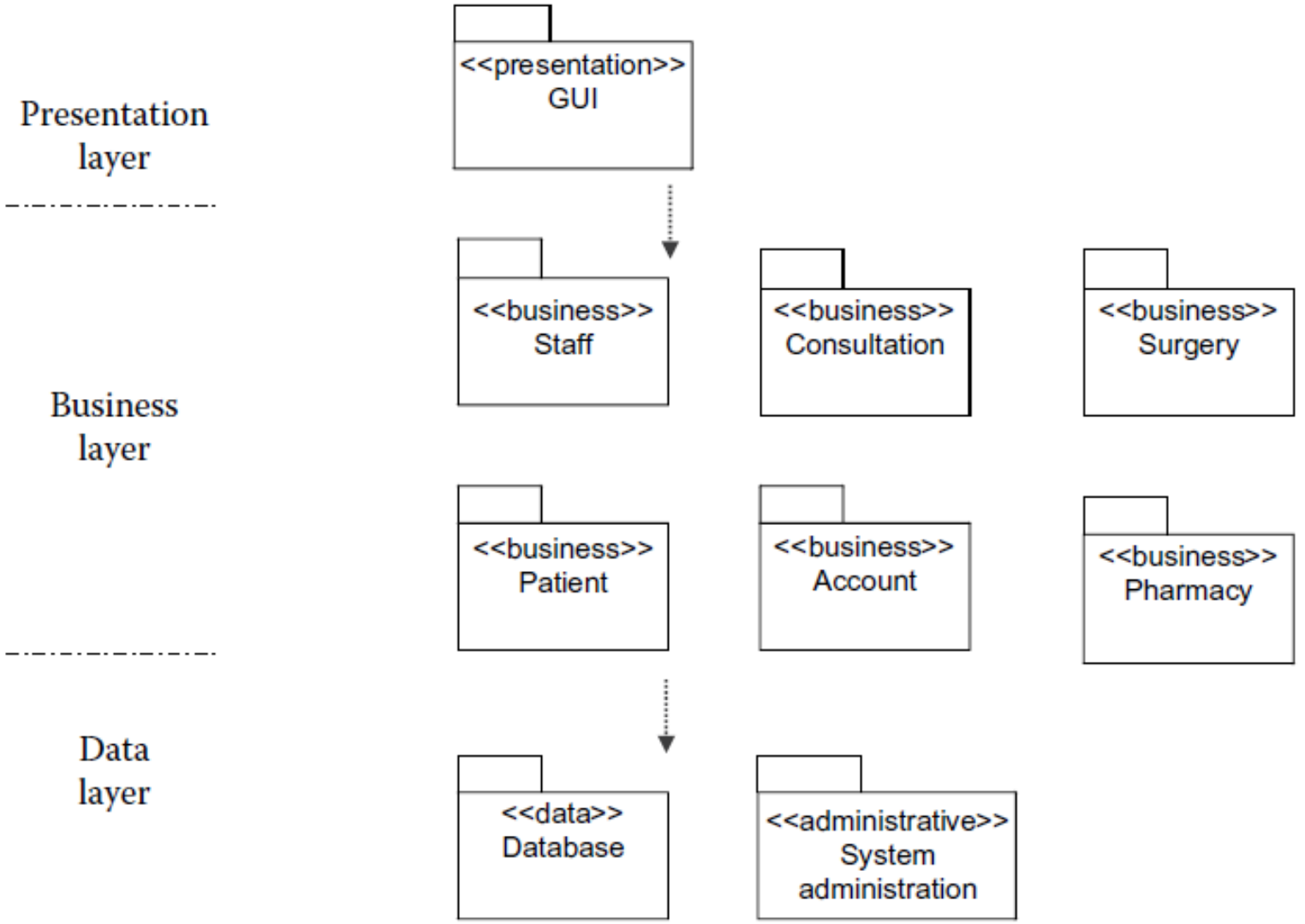
A package in UML represents a logical collection of artifacts and models. Therefore, a package is going to contain classes, components, use cases, and all other related constructs belonging to that particular subsystem.

Package–
represents a subsystem

Dependency–
optional

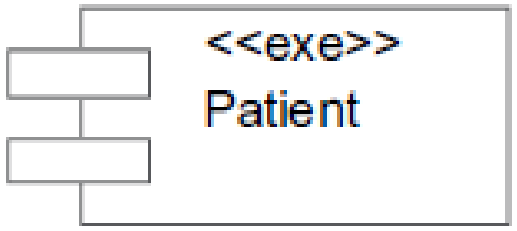
Note–
clarifies the diagram

A package diagram for hospital management system with the three layered architecture of the system

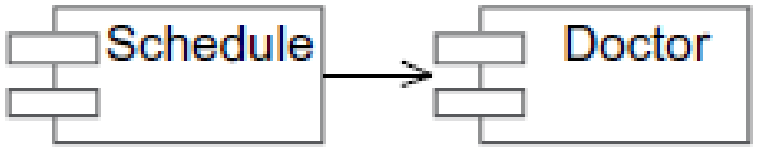


Representing Components with UML

A component is a large and cohesive collection of classes in implementation. A component in a physical module of code comprises many classes that are orchestrated to carry out functions.

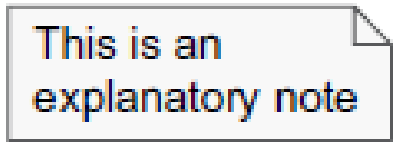


Component



Dependency

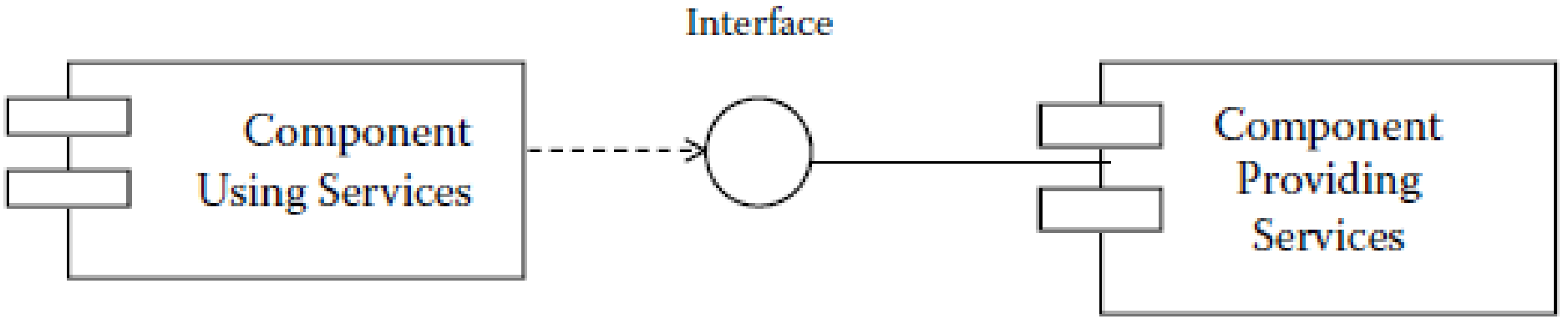
Interface

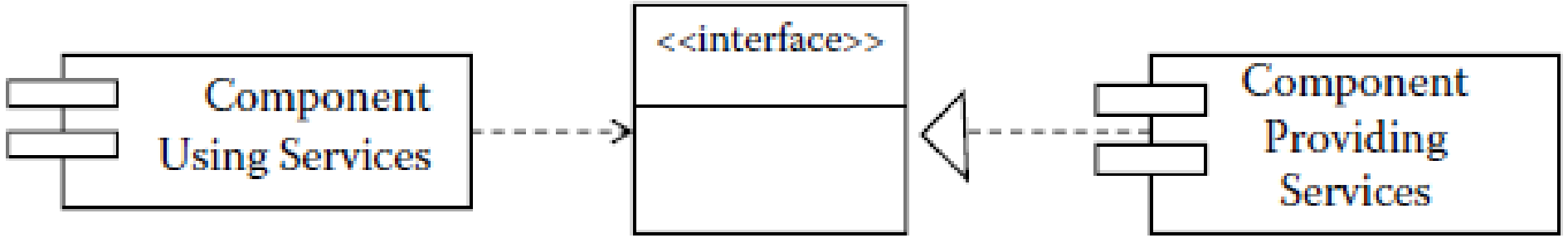


Notes

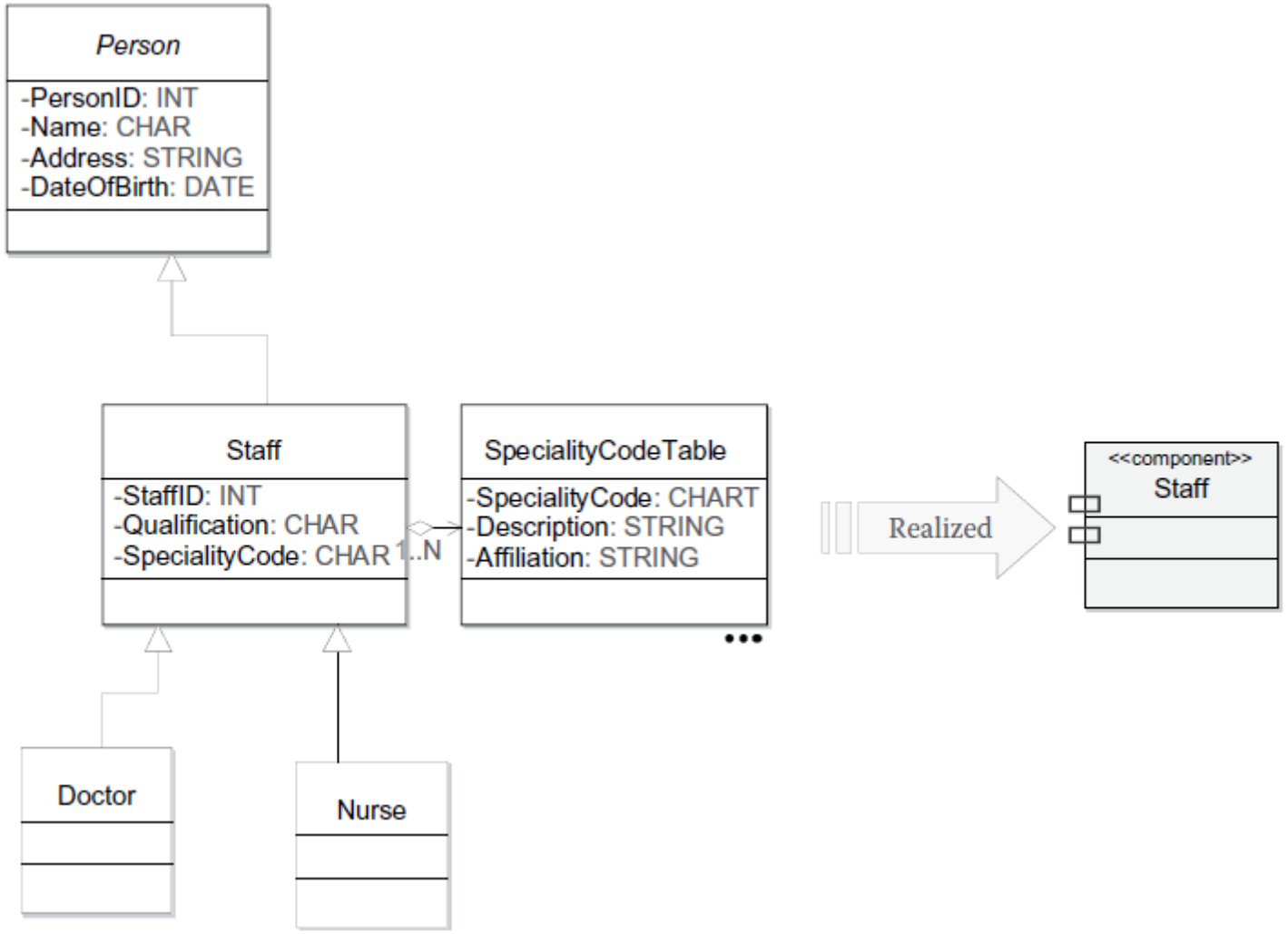
A Patient component will become Patient.EXE or Patient.DLL

Component relating through interfaces.

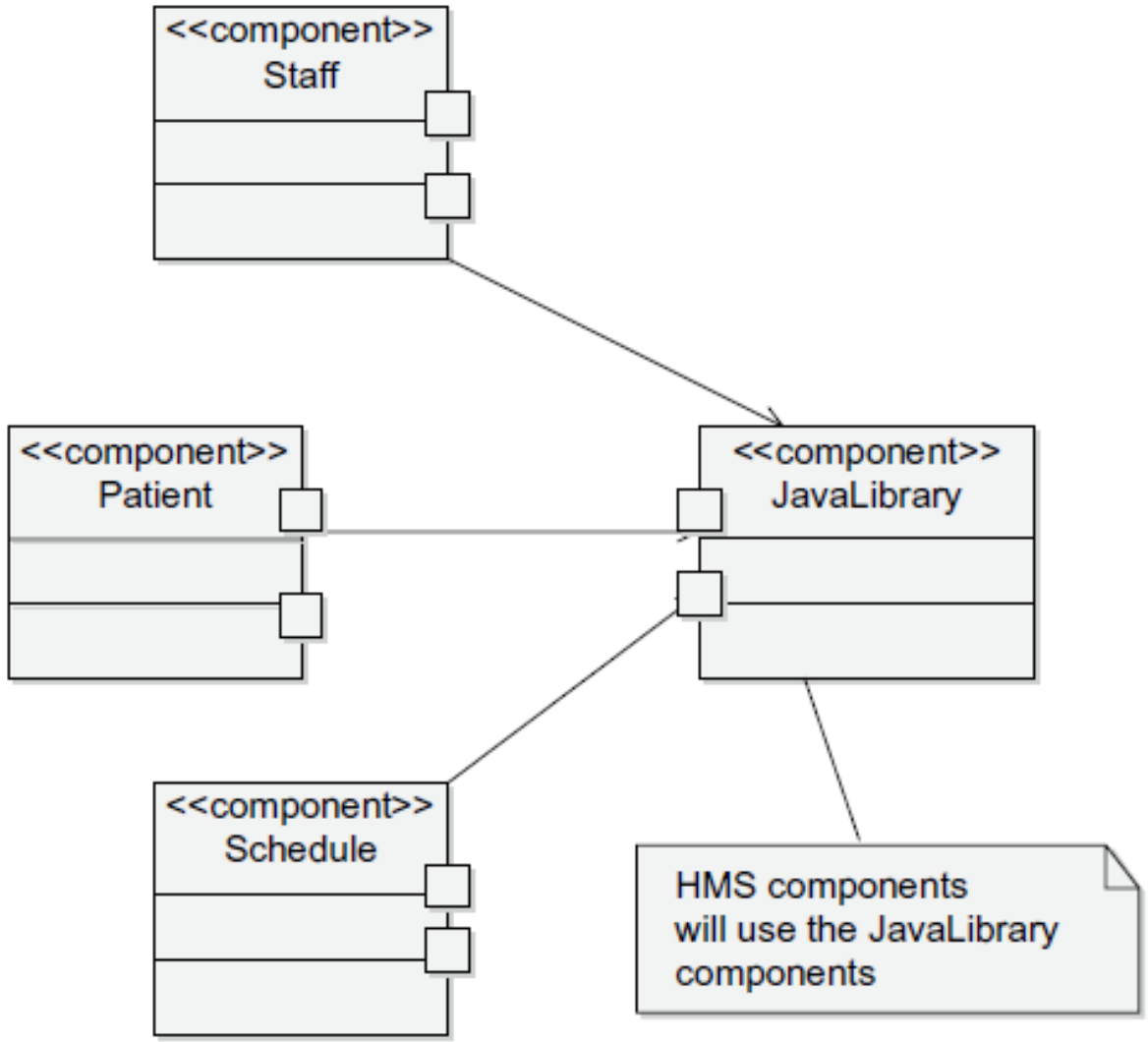




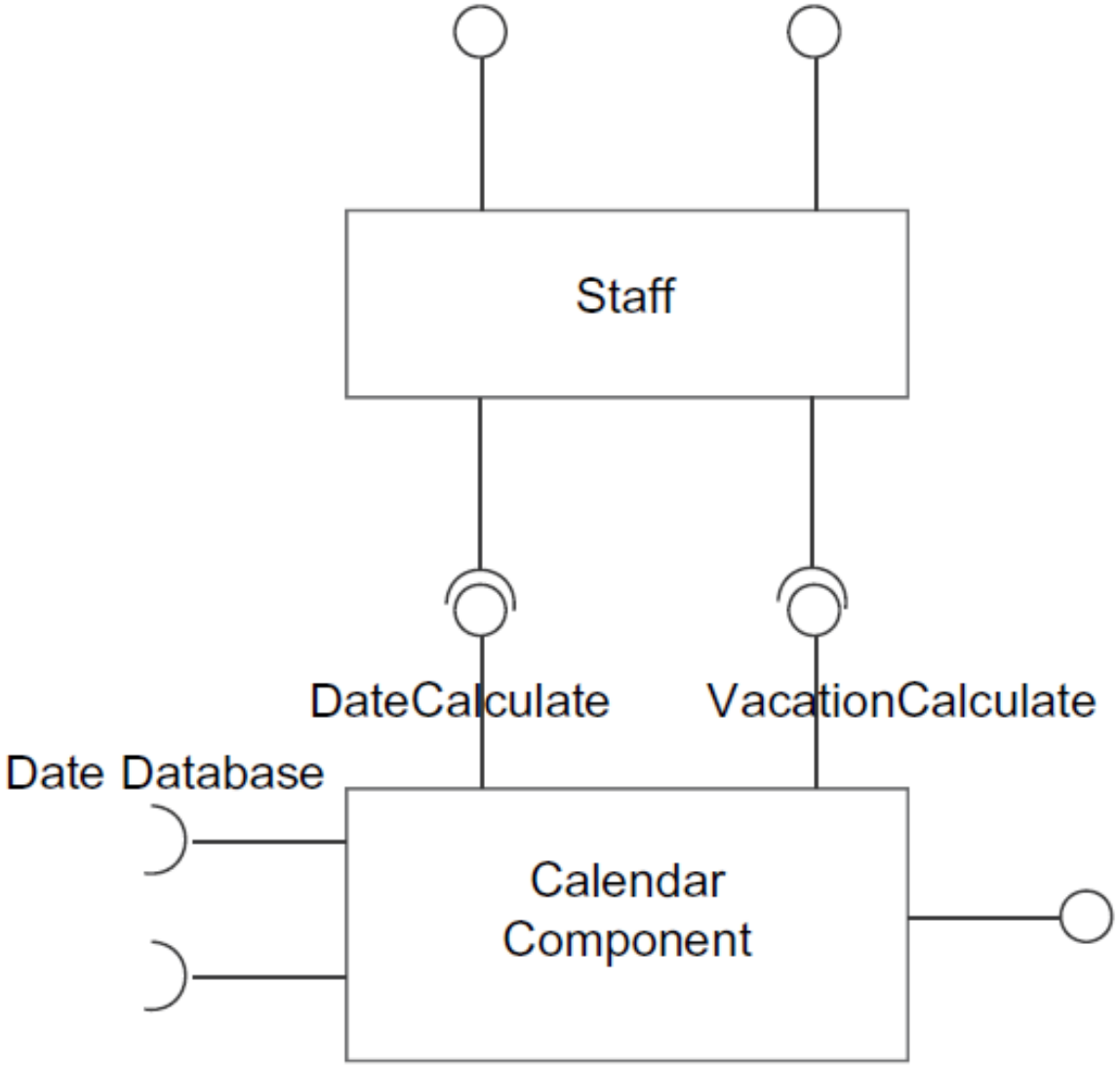
Practical Component Diagram Showing Interdependencies and Packages for HMS



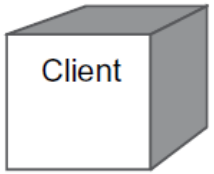
Practical component diagram for HMS showing Java library dependency



Composite Structure Diagram

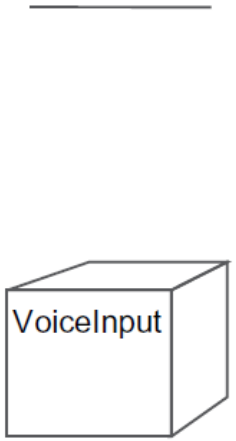


UML notations on a deployment diagram and a sample deployment diagram

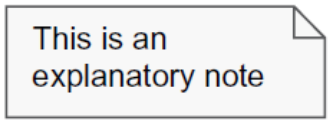


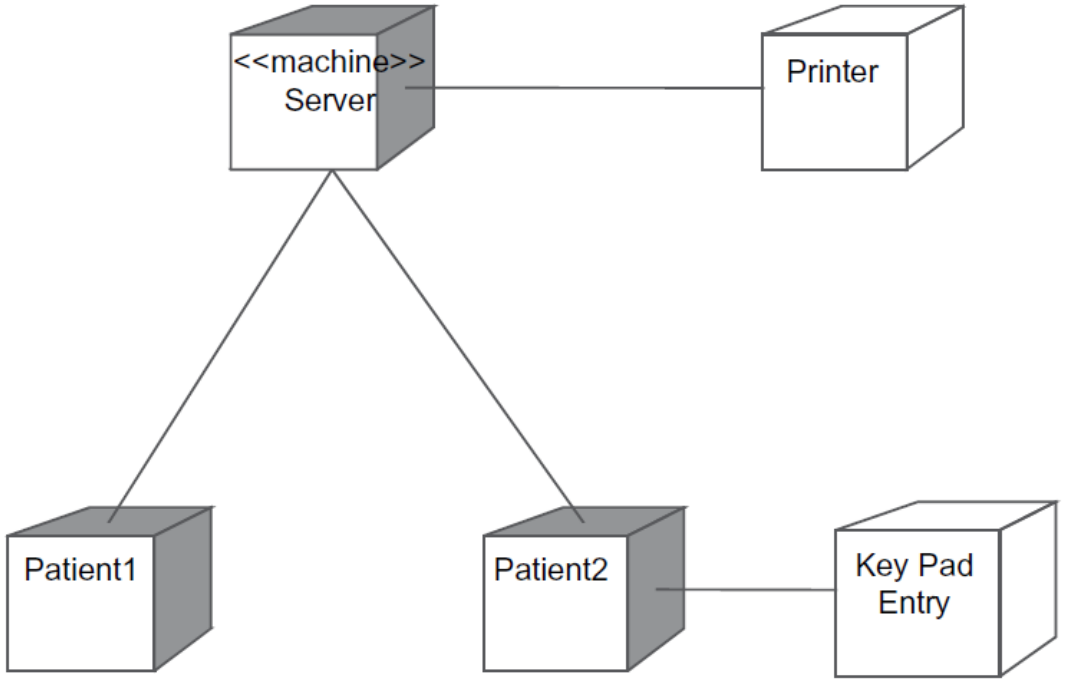
Processor

Links

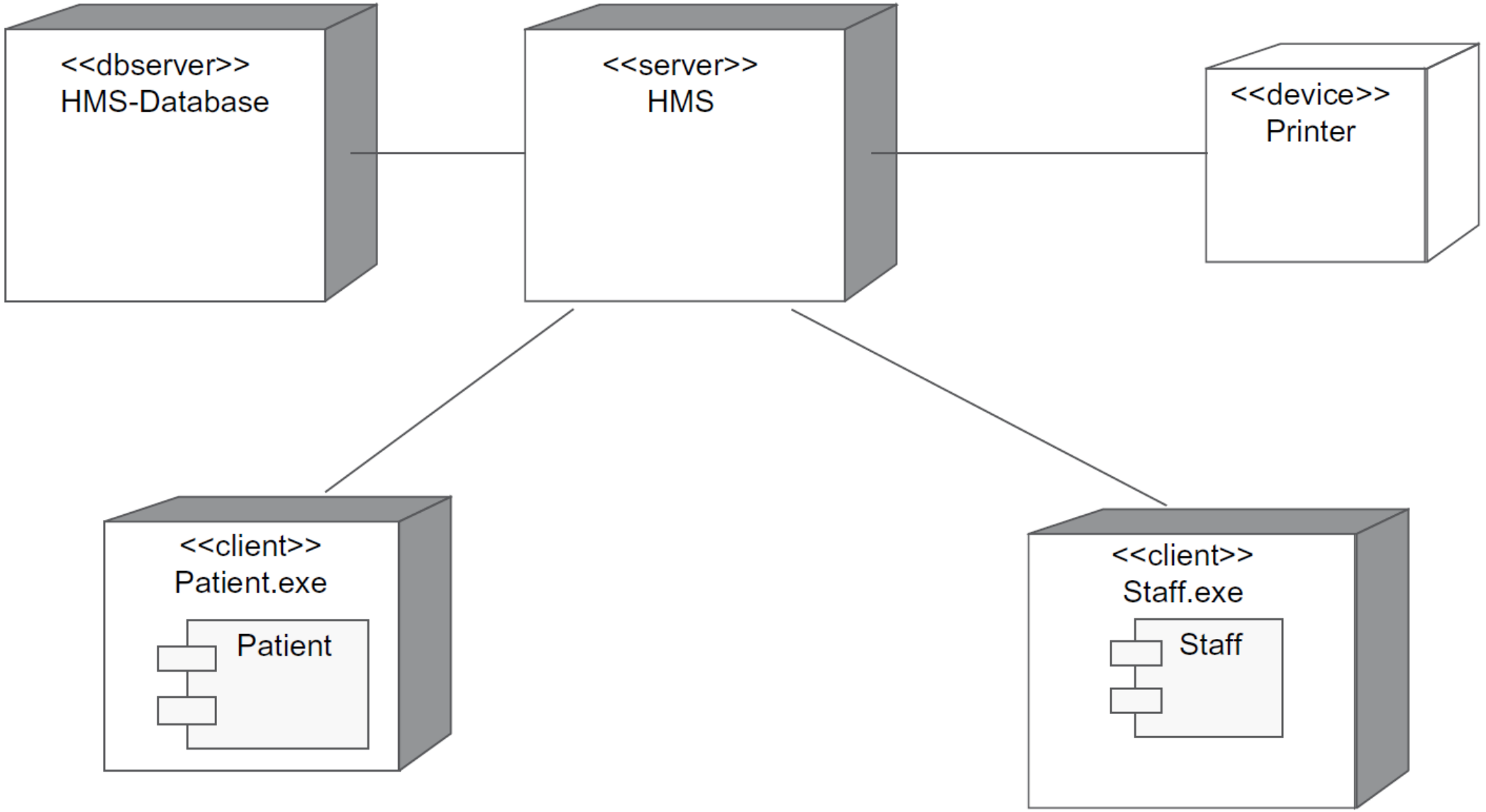


Node

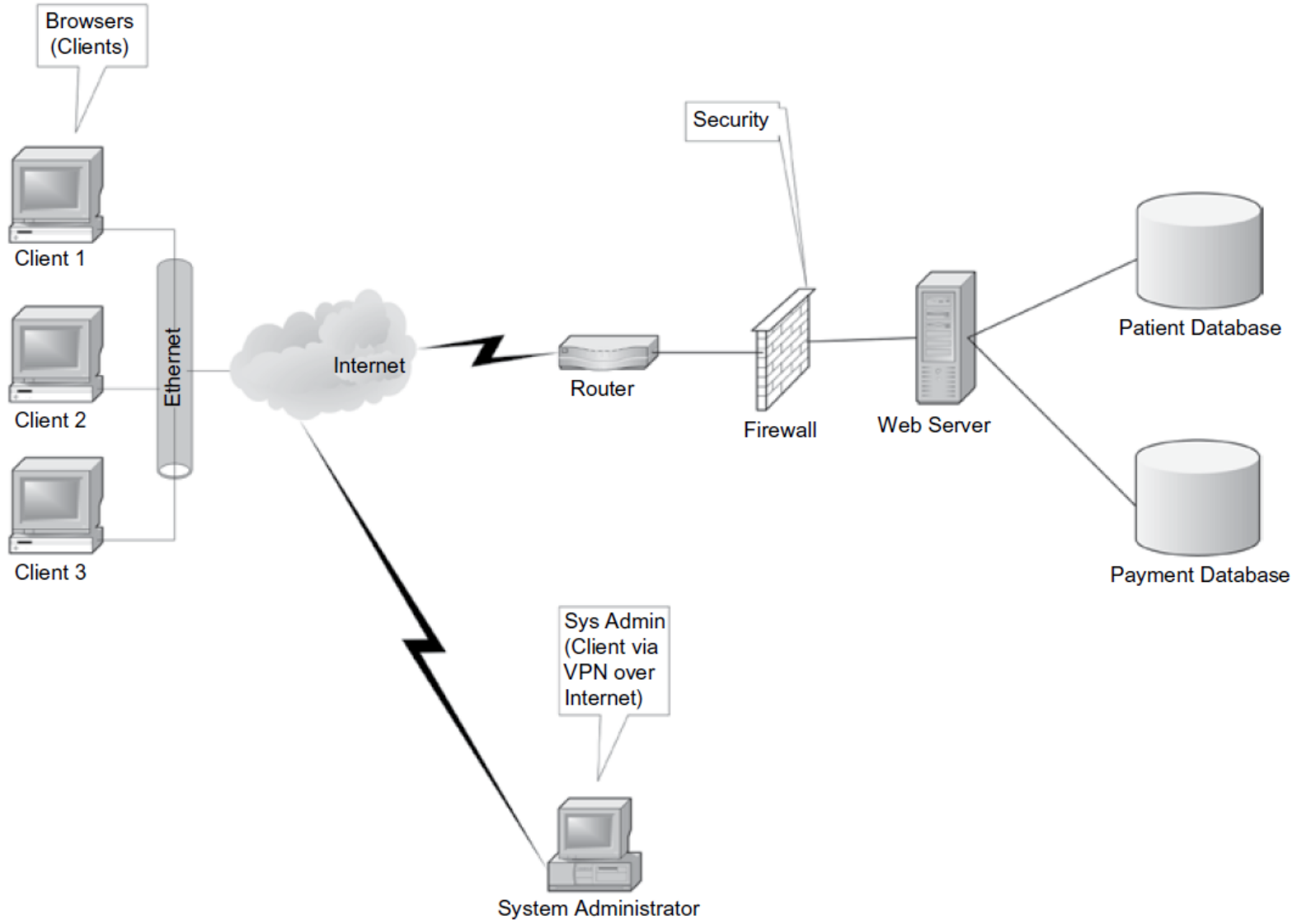




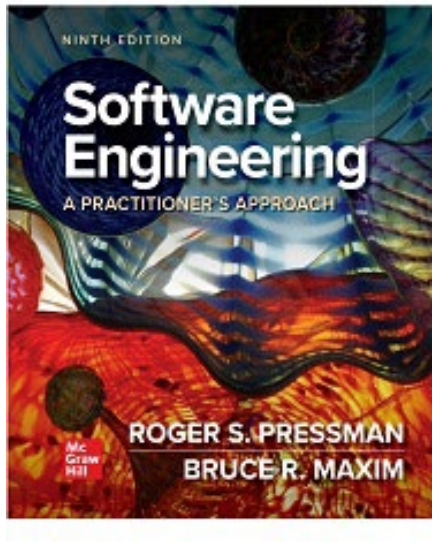
Deployment diagram with HMS components and their distribution

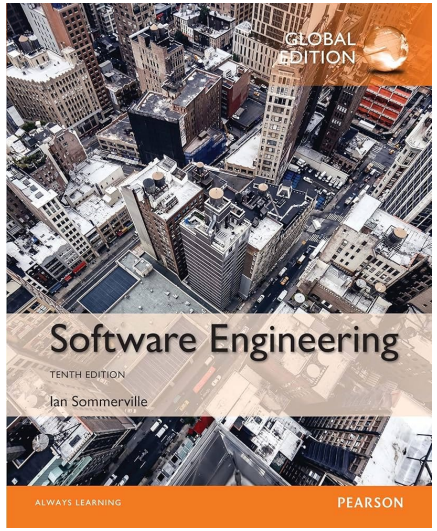


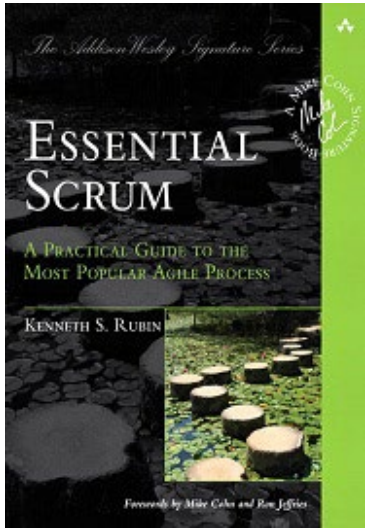
Deployment diagram—Another example

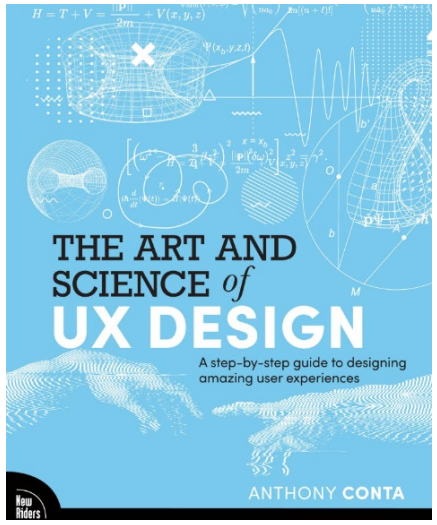


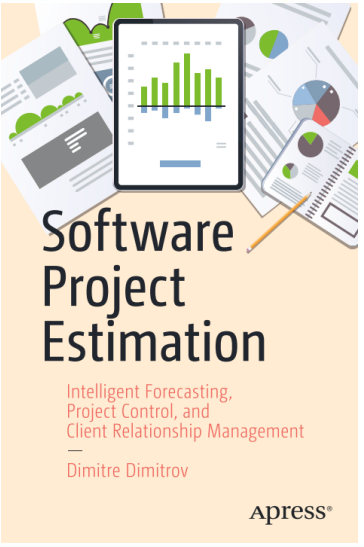
Course References











Course References

- ***Designing User Interfaces***, Michal Malewicz & Diana Malewice, 2020
- ***UI Design Styles: Trends and Design Patterns***, Michal Malewicz & Diana Malewice, 2020
- ***What UX Is Really About :Introducing a Mindset for Great Experiences,*** Celia Hodent, CRC Press, 2022
- ***Lean UX: Designing Great Products with Agile Teams 3rd Edition***, Jeff Gothelf & Josh Seiden, O'Reilly, 2021
- ***Laws of UX: Using Psychology to Design Better Products & Services,*** Jon Yablonski, O'Reilly, 2020
- ***Designing and Prototyping Interfaces with Figma***, Fabio Staiano, Packet Publishing, 2022

Course References

[1] Green, M. D. (2016). Scrum: Novice to Ninja: Methods for Agile, Powerful Development, SitePoint.

[2] Ockerman, S. and S. Reindl (2019). Mastering professional scrum: A practitioner's guide to overcoming challenges and maximizing the benefits of agility, Addison-Wesley Professional.

[3] Martin, R. C. (2019). Clean Agile, Pearson Education.

[4] Hall, G. M. (2017). Adaptive Code: Agile coding with design patterns and SOLID principles, Microsoft

[۵] سامانی‌پور، علی. (۲۰۱۸). آموزش اسکرام برای مدیریت چابک فرایند توسعه اپلیکیشن های وب و موبایل، فرادرس